



Object Oriented Analysis and Design

Block-1_Unit-1

Introduction to Object Oriented Modeling

Topics to be Covered

- Introduction
- Introduction to Object Orientation
- Basic Philosophy of Object Orientation
- Principles of Object Orientation
- Basic Constructs in Object Orientation
- Identifying Classes
- Benefits of Object Orientation
- Introduction to Object Oriented Analysis & Design Tools
- Important Topics
- Summary

Introduction

- Object Oriented Analysis and Design (OOAD) is an approach for developing software systems using object-oriented software engineering concepts.
- OOAD is about having lots of options. There is never only one right way to solve a problem; more options may exist. The more options you have, the better chance you will find a good solution to every problem

Introduction to Object Orientation

- The OOAD approach is sequential in nature. The analysis phase establishes the system requirements and performs domain analysis from the customer perspective. Once the analysis phase is complete, the design phase (which is iterative in nature) maps the logic design to the physical design. The prototype is created and tested.
- Object Oriented Modeling is divided into various stages:
 - The OOM of the system passes through the following steps:
Requirement Analysis of System,
 - Designing of system,
 - Detailed designing with a focus on objects in the system, and
 - Implementing the model.

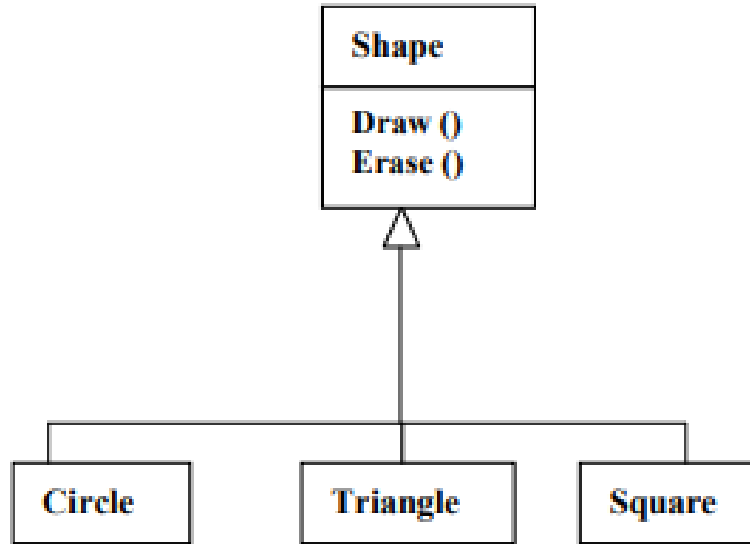
Basic Philosophy of Object Orientation

- Sharing of Structure and Behaviour : One of the reasons for the popularity of object-oriented techniques is that they encourage sharing at different levels.
- Emphasis on Object Structure, not on Operation Implementation In object-orientation, the major emphasis is given to specifying the characteristics of the objects in a system rather than implementing these characteristics.

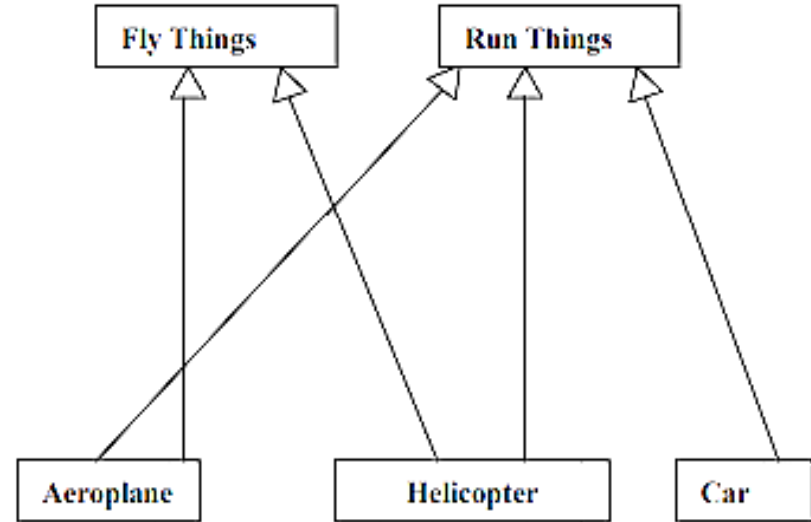
Principles of Object Orientation

- **Abstraction**
 - Abstraction is the process of hiding the internal details of an application from the outer world. Abstraction is used to describe things in simple terms. It's used to create a boundary between the application and the client programs.
- **Encapsulation**
 - Binding data members and functions so that with in a class. Basically, it is a protective shield that prevents the data from being accessed by the code outside this shield.
- **Inheritance**
 - It is a feature of OOAD which allows a child class to have features of parent class.

Principles of Object Orientation



Single inheritance



Multiple inheritance

Basic Constructs in Object Orientation

- Polymorphism
 - “Poly” means many and “morphism” means form, so we can say that Polymorphism is ability of any data to be processed in more that one form.
- Class & Object
 - Class is blueprint of object, it defines how a object should look like.
 - Object is instance of class, we can create as many objects we want.
- Links and Association
 - Links and Association represent the **relation** between **objects** and **classes**. Link is used to establish the relation between the objects in the object diagram. Association is used to represent the relation between the classes.

Basic Constructs in Object Orientation

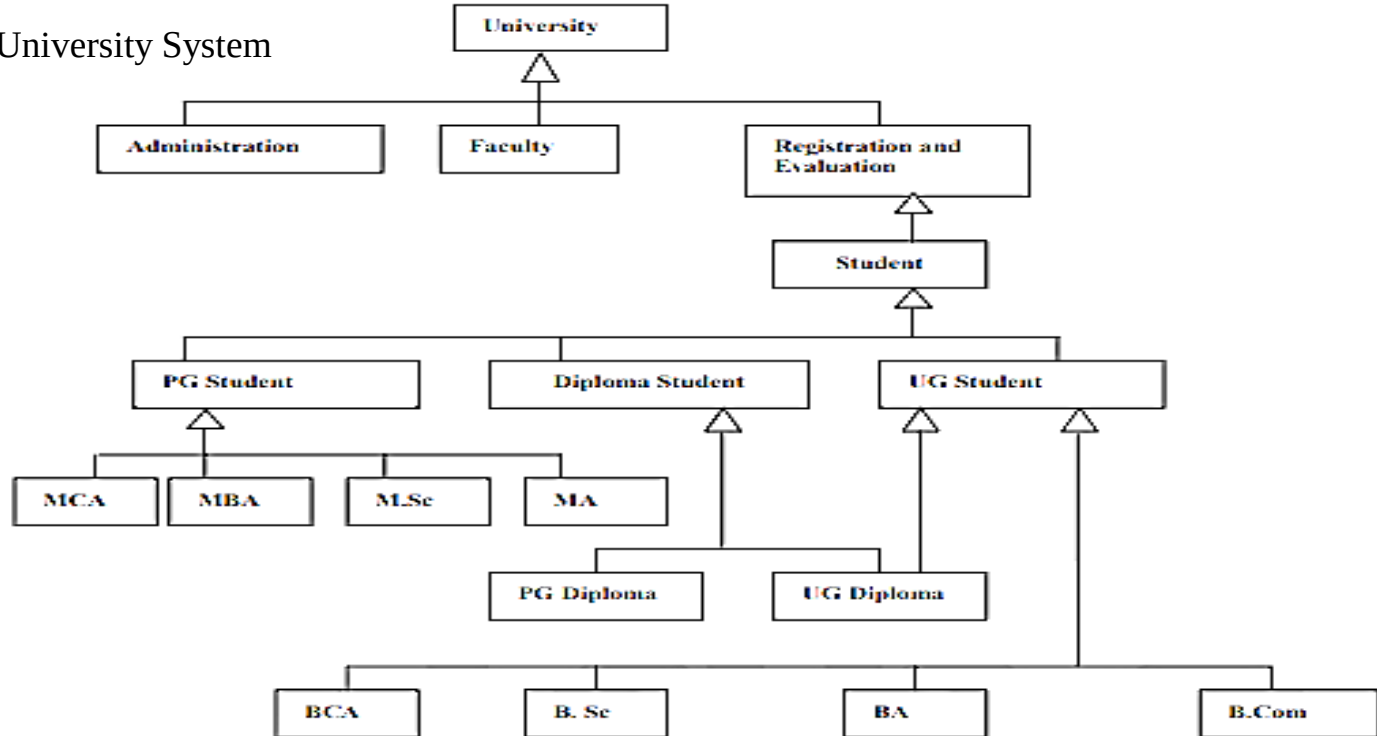
- Generalization and Inheritance
 - Generalization and inheritance are powerful abstractions for sharing the structure and/or behavior of one or more classes.
 - Generalization is the relationship between a class, and it defines a hierarchy of abstraction in which subclasses (one or more) inherit from one or more super classes. Generalization and inheritance are transitive across a subjective number of levels in the hierarchy.
 - Generalization is an "is-a-kind of" relationship, for example, Saving Account is a kind of Account, PG student is a kind of Student, etc.

Identifying Classes

- In object oriented modeling, system understanding and on the basis of that identification of classes is one of the key activity. Establishing relationship among different classes in the system are the first and foremost activity.
- For finding the classes which could be representative of the problem domain and are useful in your problem solving.
 - Find the nouns
 - Evaluate the nouns to find classes
 - Define the purpose

Object Model

Object model for University System



Benefits of Object Orientation

- Faster development
- Increased Quality
- Easier maintenance
- Reuse of software and designs, frameworks
- Reduced development risks for complex systems integration
- Classes
- Objects
- Inheritance
- Association

Introduction to Object Oriented Analysis & Design Tools

- **Object-Oriented Analysis (OOA)** is the first technical activity performed as part of object-oriented software engineering. OOA introduces new concepts to investigate a problem. It is based on a set of basic principles, which are as follows-
- The information domain is modeled.
- Behavior is represented.
- The function is described.
- Data, functional, and behavioral models are divided to uncover greater detail.
- Early models represent the essence of the problem, while later ones provide implementation details.

Important Topics

- Principles of Object Orientation
- Constructs in Object Orientation
- Benefits of Object Orientation

Summary

- The basic philosophy of object orientation, and the need for object oriented modeling.
- The concepts of abstraction, encapsulation, inheritance and polymorphism.
- The basic constructs of object orientation which makes it possible to develop systems in such a way that the system is flexible and open for reusability.
- Basic constructs of object orientation such as class, object, links and association, and generalization are explained in this unit.

*Thank
You*



Object Oriented Analysis and Design

Block-1_Unit-2

Structural Modeling using UML

Topics to be Covered

- Introduction
- Introduction to UML
- Hierarchy of UML diagrams
- Basic Structural Modeling
- Advanced Structural Modeling
- Advanced Classes
- Advanced Relation
- Interfaces Type and Roles
- Important Topics
- Summary

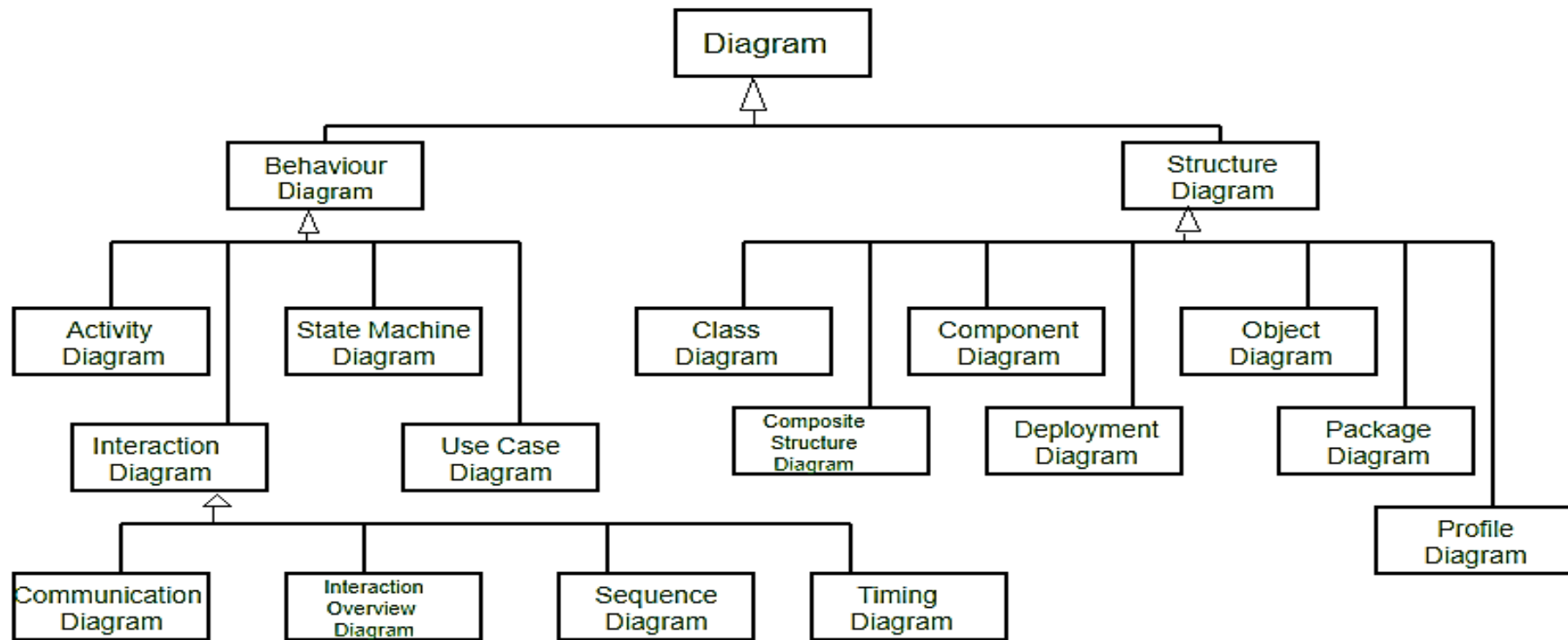
Introduction

- The software application development process consists of many stages. For the development of enterprise applications, there is a team such as Analyst, designer, coder, tester, technical writer and each member of the team works on a different stage and needs a different level of detail.
- This session explains you how to construct the class diagram. Before defining a class diagram, in this session explains about the basic concepts classes, relationships and some important terms or notations that are useful in designing UML diagrams.

Introduction to UML

- UML stands for uniform modeling language.
- It is a general purpose modeling language.
- The main aim of UML is to define a standard way to **visualize** the way a system has been designed.
- UML is **not a programming language**, it is rather a visual/pictorial language. We use UML diagrams to portray the **behavior and structure** of a system.
- UML is used to design models for large, medium as well as small applications.
- UML was created by the Object Management Group (OMG).

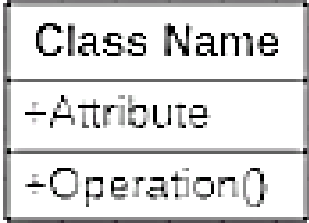
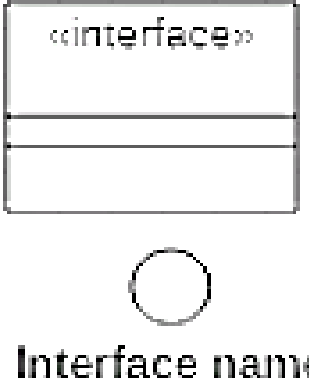
Hierarchy of UML diagrams



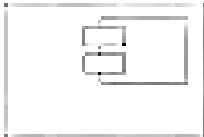
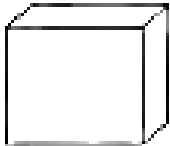
Basic Structural Modeling

- Modeling means creating a diagram for a system that includes identifying the elements that are important to your particular module.
- The structural modeling gives a structural view of a system that highlights the structure of the objects including their classifiers, relationships, attributes and operations.
- In the previous Unit, you have already got awareness about the basic principles and constructs in object orientation paradigms which are helpful for understanding the concepts of structural modeling.
- The class diagram is one of the most important structural model in UML.

Core elements of the structural modeling

Construct	Description	Syntax
Class	Class is a 'blueprint' for creating objects. It contains a group of objects that share the same properties, operations, and relationships. It contains three sections as name, attributes and operations.	
Interface	It is a collection of named operations that describe the behaviour of an element. You can neither define any attributes nor any implementation of the operations in the interface.	

Core elements of the structural modeling

Component	A component diagram is used to split the actual system into various modular parts. Each component has one clear aim within the whole system.	
Node	It is a run-time physical object that denotes a computational resource such as server, router.	
Constraint	It is a condition for a UML element	

Advanced Structural Modeling

- A class diagram commonly contains classes, interfaces and relationships such as dependency, generalization and association.
- Class diagram gives static design view of the system. The class diagram and object diagram are examples of the structural model.
- A class diagram demonstrate what the objects contain in your system in the form of members and their capacity in terms of methods.
- An object diagram shows how objects will affect each other in your system at some state of the object and what values those objects contain in this state.

Advanced Classes

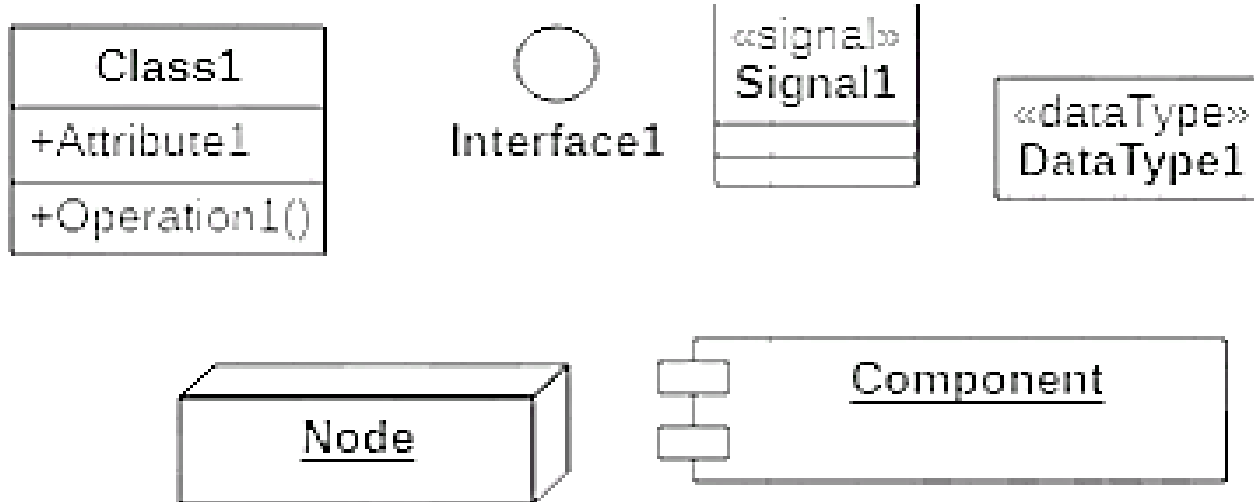
- Classifier : A classifier is a kind of UML element that is used to define some common features in terms of structural (attributes) as well as behavioral (operations).
- A class is an important type of classifier in UML, which describes a group of objects that share the same features. Some more classifiers are given as follows:
 - Interface: It is collection of named operations that describe the behavior of an element.

Advanced Classes

- **Datatype:** its values have no identity, including primitive built-in type (eg. number, string) and enumeration types (eg. Boolean).
- **Node:** It is a run-time physical object that denotes a computational resource such as a server, router etc.
- **Signal:** An asynchronous event communicated between objects.
- **Use case:** It is a list of stages that explain how a process will be carried out in a system.
- **Component:** A component is used to split the actual system into various modular parts.

Advanced Classes

Graphical notations of the classifier

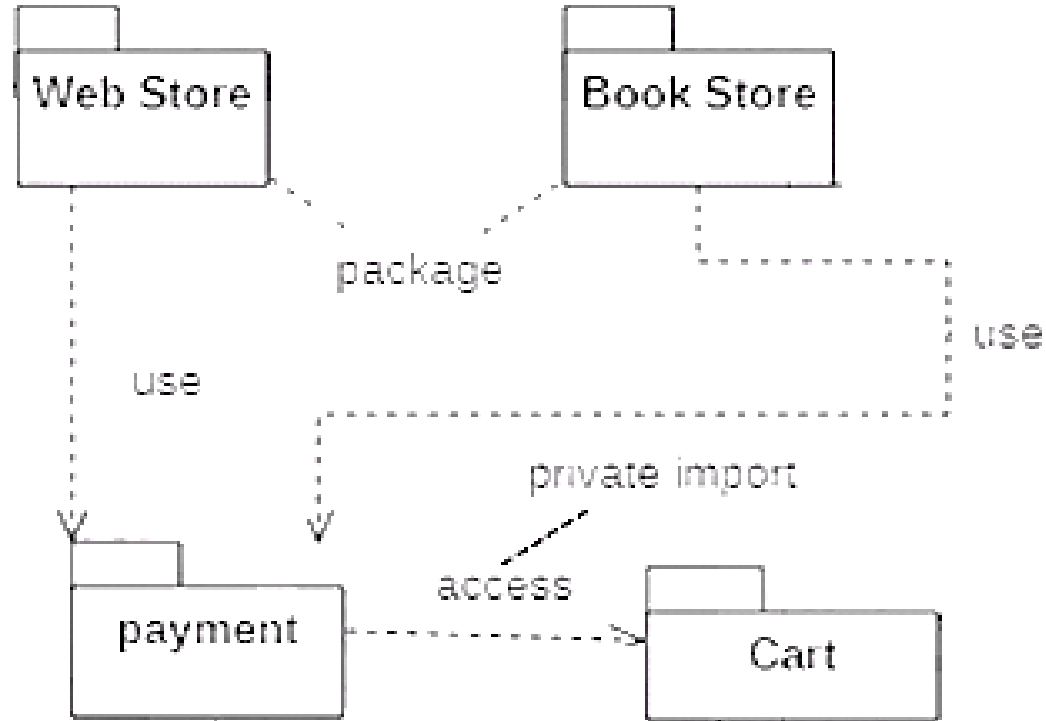


Advanced Relation

- In the UML diagram, the elements are connected with each other through relationships. So, relationships are the connections between elements.
- Dependency Relationship
 - Dependency describes the relationships among elements in which one element is dependent on another element. It means that if changes occur in one element, it will affect the other element.

Advanced Relation

Dependency
Relationships in
Packages :

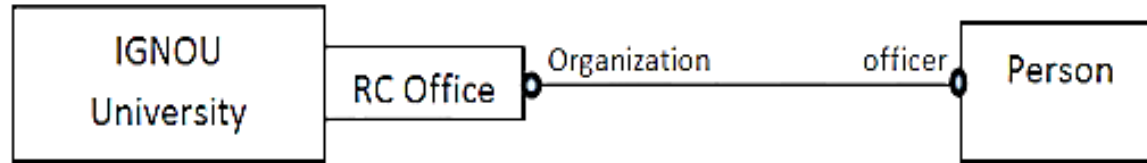


Advanced Relation

- Generalization Relationship
 - Generalization relationship is a relationship in which one element (the child) is based on another element (the parent).
 - Generalization relationships are used in class, component, deployment and use-case diagrams
- Association Relationship
 - An association is a structural relationship; it is used to specify that objects of one element are connected to objects of another element.
 - Aggregation is another form of association, which specifies a whole-part relationship between the aggregate (whole) and the component part.
 - For example, University has many schools. It is an example of aggregation.

Advanced Relation

This example represents qualified many-to-many association :



Many-to-many Qualification

Interfaces, Type and Roles

- **Interface**

- An interface is a collection of operations that are use to specify a service of a class or a component.

- **Interface relationships**

An interface may participate in generalization, association, dependency and realization relationships.

Note: Interfaces may also be used to specify a contract for a use case or subsystem

- **Role**

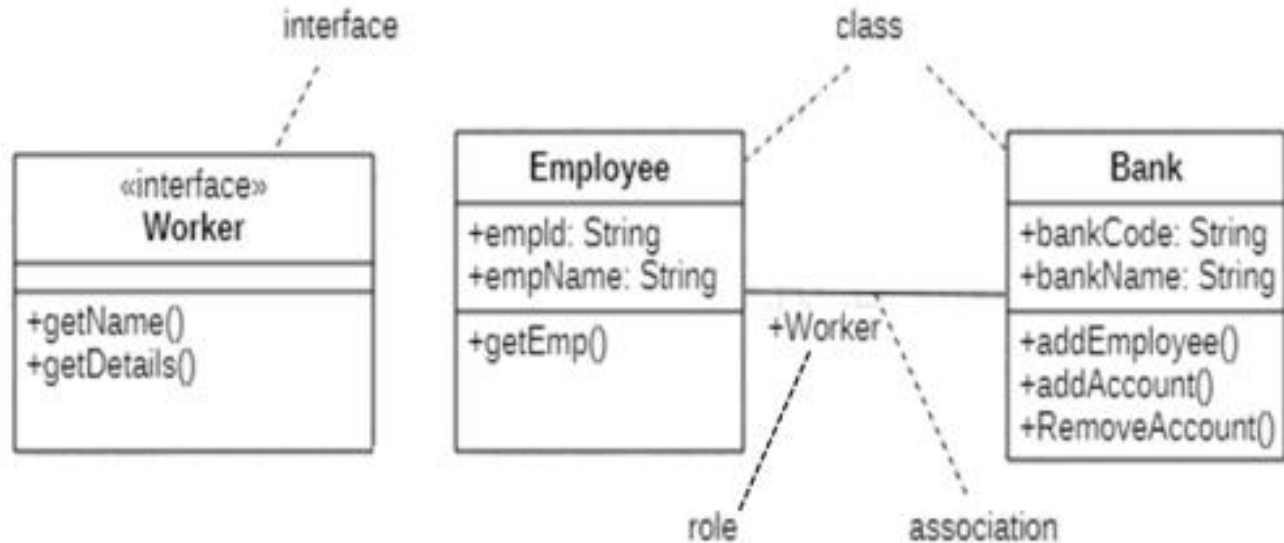
A role names(indicates) a behavior of an entity participating in a particular context. Or, a role is the face that an abstraction presents to the world

Interfaces, Type and Roles

Type Example:



Interfaces, Type and Roles



Interface and Role

Important Topics

- Basic Structural Modeling
- Basic Structural Modeling
- Unified Modeling Language

Summary

- This session explained unified modeling language (UML) and their diagrams used in designing a model.
- Model is a blueprint of the real system that needs to be built. UML is a standard modeling language and is different from the other programming languages.
- The static or structural view focuses on the system's static structure using objects, attributes, operations, and relationships
- The dynamic or behavioral view represents the dynamic nature of the system.
- There are three basic types of relationships in UML: Dependencies, Generalization, and Associations.

*Thank
You*



Object Oriented Analysis and Design

Block-1_Unit-3

Behavioral Modeling Using UML

Topics to be Covered

- Introduction
- Basics of Behavioral Modeling
- Interactions
- Use Cases and Use Case Diagram
- How to Draw Use Case Diagram
- Where to use a Use Case
- Diagram
- Interaction Diagram
- Activity Diagram
- CRUDE Analysis
- Important Topics
- Summary

Introduction

- Object-Oriented Modeling (OOM) is used for modeling the application from the initial stage of the software development. There are different types of objects used in modeling.
- Objects are created and executed by describing them inside the program. Each object is defined in two ways: system-defined objects and user-defined objects.
- Every object has its class and instances. A class of objects is defined by declaring the instances of the class.

Introduction

- The structural classification part is the structural classification, which describes the relationship between one object to another object; while the classifier describes the Static view, Use Case view, and Implementation view.
- Dynamic behavior depicts the behavior of a system with time. Behavior can be changed with time and can be represented using a State Machine, Activity, and Interaction Diagrams.
- Model management is the third part that describes the organization in a hierarchical structure

Basics of Behavioral Modeling

- The behavior model helps to narrate the internal parameters of the entire system, which supports all the activities of an organization.
- It also expresses the system's internal logic without determining how the process is to be executed.
- The objective of the behavioral models is to display the fundamental objects in a problem domain that will work together to form a collaboration to support each of the use-cases.

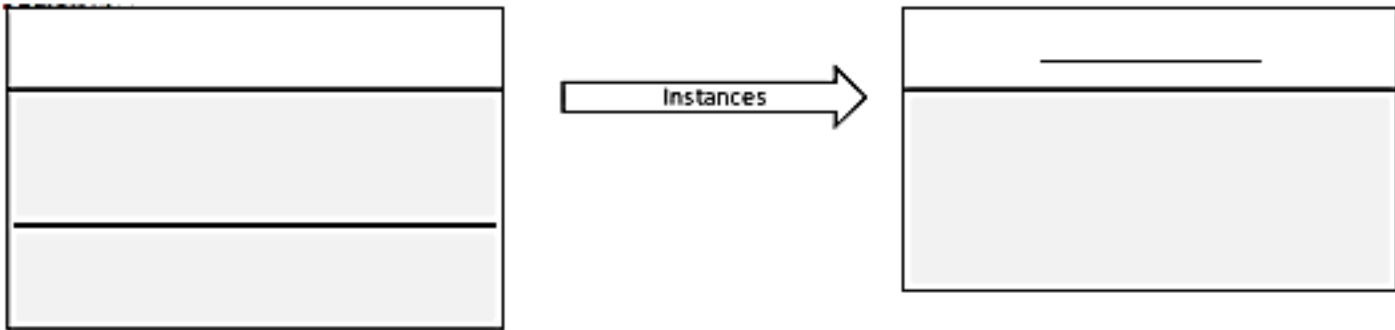
Interactions

- An interaction is a behavior that contains a set of messages that substitute a set of objects to accomplish the defined objectives.
- OBJECT DIAGRAM
 - If we use an object diagram, then the unlimited number of instances are available in nature and need to be considered its impact on the system.
 - Following things should be considered and understood clearly during the construction of the object diagram.
 - Objects are used to construct the object diagrams.
 - Links are used for connecting the objects.
 - Objects and links are used to construct an object diagram.

Interactions

- INSTANCE

- In UML, every block is represented by the class. It is used to create instances or objects of the class. Every object is associated with the class.



Class and Instance

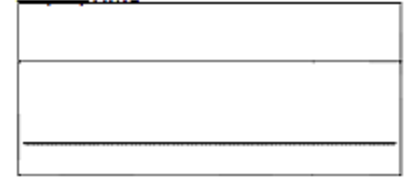
Interactions

- **ACTION:**
 - It is a named element that represents a movement from one stage to another stage. It represents a single step within the operation. Every action has some input and output. It is represented by round-cornered rectangles
 - Some of the action names are given below:
 - Fill Order
 - Check-In
 - Course Enrolment
 - Display Error



Interactions

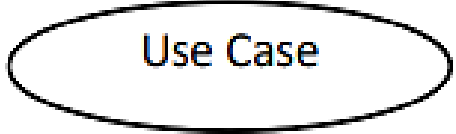
- OPERATION:
 - Features of a class are represented in the form of attributes and operations.
operation is represented in a form of block.
 - The top rows represent the name of the class,
 - the middle row represents the class's attributes, and
 - the third row represents operations of the class.
- LINK:
 - It is used for establishing a relationship between objects and classes of the system.
 - Link is represented by a line.



Use Case Diagram

- **Objectives Of Use-case Diagram**

- To identify the requirement of a system.
- To get the external perspective of a system.
- To identify the internal and external factors that influence the system.
- Show the relationship between actors.
- It also describes the flow of the event



Use Case

Use Case Diagram

- It is used to describe a part of the system's functionality that enables access to the functionalities. It is represented using the ovals. Label the ovals with verbs that represent the system's function. The objective behind use-cases is to describe interactions. The flow of batch processing, which generally does not include interactions, can also be used as a use-case.
- Actor
 - An actor is a user or external system that moves a system. The role of an actor is equated with determining the behaviour of the associated system.



How To Draw A Use Case Diagram?

- Use Case diagrams are used to define the functionalities of a system. When the diagrams are considered for high-level analysis, objects, relationships, and their guidelines should be clearly defined
- We should consider the following items while drawing a use-case diagram –
 - Identify all the Use Cases.
 - Identify all the actors.
 - Generalize the relationship between Use Cases and actors.

How To Draw A Use Case Diagram?

- Following guidelines should be followed to draw a Use Case diagram –
 - Use Case is used to identify the requirements.
 - Use Case name reflects the functionalities of the object.
 - We need to have appropriate names for actors.
 - Relationships and dependencies are specified in the Use Case diagram.
 - Name should be defined in such a manner that it reflects the functionalities it performs.
 - Relationships and dependencies are clearly defined in the Use Case diagram.

Where To Use A Use Case Diagram ?

- To analyze the requirement analysis of a system.
- To prepare the model of a system.
- Requirement analysis and high-level design.
- Forward and reverse engineering.
- Modeling the basic idea behind the system.

Interaction Diagram

- **OBJECTIVES OF INTERACTION DIAGRAM**
 - It is used to monitor the dynamic behaviour of a system
 - It represents the structural aspects of various objects in the system
 - It represents the sequence of interactions within a system
 - It provides the means of visualizing the real-time data.
 - It can be used to explain the architecture of an object-oriented or distributed system.
 - It shows the communication and sequence of messages passing in the system.

Interaction Diagram

- There are two types of Interaction Diagram
 - SEQUENCE DIAGRAM
 - sequence diagram describes the sequence order of interaction between objects. It also describes how performance is enhanced of the objects in a system. It flows from left to right.
 - COLLABORATION DIAGRAM
 - It is used to show the relationship between the objects present in a system. Both the sequence and the collaboration diagrams represent the same information but differently. Instead of showing the flow of messages, it depicts the architecture of the object residing in the system as it is based on object-oriented programming. The collaboration diagram, is also known as a communication diagram, is used to portray the object's architecture in the system.

Activity Diagram

- Activity diagram is basically a flowchart which is used to represent the flow from one activity to another activity. The activity can be described as an operation of the system.
- The control flow is drawn from one operation to another. This flow can be branched, sequential, or concurrent.
- An activity diagram consists of the following behavioral elements:
 - Initial State/Start Point: It represents an activity diagram's initial state or start point. It is denoted by a small, filled circle.



Activity Diagram

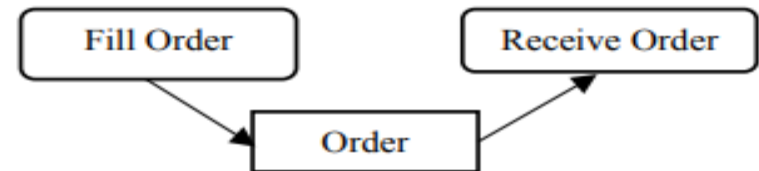
- Activity or Action State: It represents the uninterruptible action of objects. It is represented by a rectangle with rounded corners or edges.

Activity/Action State

- Action flow or Control flows: Action flow shows the edges and paths from one action state to another action. It is represented by an arrowed line.

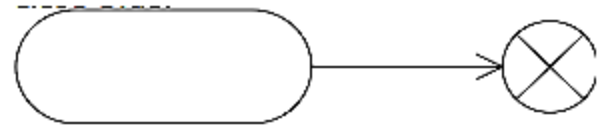


- Object Flow: It is used for the creation and modification of objects. An object flow arrow shows an object actions, including the transition from one active state/object to the other object.

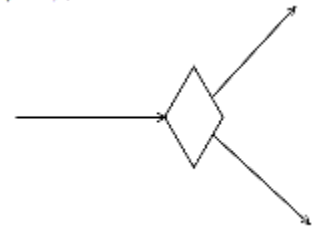


Activity Diagram

- Flow Final Node: It shows a circle with a cross inside. It shows the end of a single control flow. The final activity node denotes the end of all control flows within the activity.

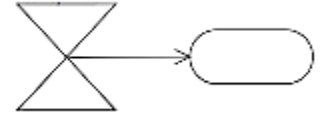


- Decision: It is represented by a decision with alternate paths. It is like a flowchart where the decision is to be made by diamond, with the options written on the sides of the arrows issuing from the diamond.

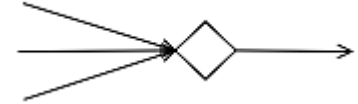


Activity Diagram

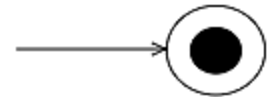
- Time Event: It refers to an event that stops the flow for a time.



- Merge Event: It is used to merge multiple flows that are not concurrent.



- Final State/End State: An arrow is used to indicate the circle, and the inside filled circle shows the end state or final state of the diagram.



CRUDE Analysis

- The CRUDE analysis is used to perform various activities of an object in the project domain and collaborate in support of Use Case analysis. CRUDE matrix is used to represent each interaction among objects, it is labeled for the various purpose, which is as follows:
 - Create - to create and store new data.
 - Read - to retrieve and read data.
 - Update - to change or modify then store the data.
 - Delete - to delete or remove the data.
 - Execute - to execute the data.

Important Topics

- Behavioral Modeling.
- Use Case Diagram
- Sequence Diagram
- Collaboration Diagram
- Activity Diagram
- CRUDE Analysis

Summary

- Modeling is used for the developing application/services of the software. Different types of objects are used in modeling.
- A Use Case diagram is a dynamic behaviour in the system to represent the dynamic expressions of the system.
- An interaction diagram is used to represent the dynamic behaviour, and it consists of objects, relationships and messages.
- The activity diagram represents the activities of dynamic behaviour. To capture the various activities of objects and permissions within a system is represented by the CRUDE matrix.

*Thank
You*



Object Oriented Analysis and Design

Block-1_Unit-4

Advanced Behavioral Modeling Using UML

Topics to be Covered

- Introduction
 - Events and Signals
 - State Machines
 - Processes and Threads
 - Time and Space
 - State Chart Diagram
 - How to draw a State Chart
- Diagram
 - Where to Use State Chart Diagrams
 - Important Topics
 - Summary

Introduction

- This unit provides various existing event approaches to behavioral modeling.
- UML is used to define a wide variety of behavior using the basic features of events, states, signals, and transactions.
- Using these features state machines, and the time-space diagram represents arcs (transactions) and vertices (states) in the behavioral models.
- The relationship with structural elements in these diagrams will be explained in this unit.

Events And Signals

- Events

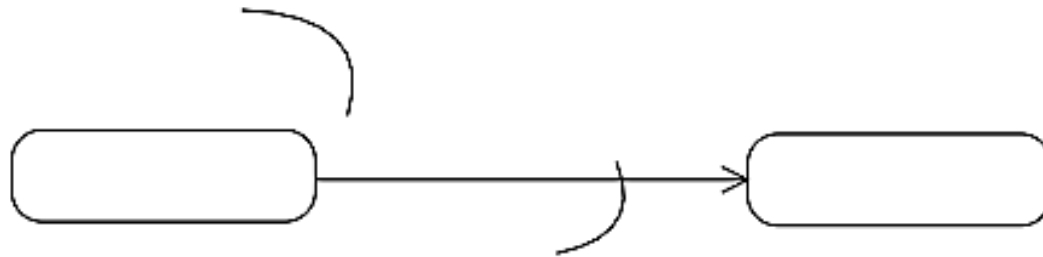
- Events are also known as the “Things that occurred”. UML, event things happened in the form of the model.
- An event is a model specification that causes something to happen in time and space.
 - Events can be internal or external.
 - Events may occur in synchronous and asynchronous forms.
 - The time at which the event executes is an implicit attribute of the event.
 - One event may occur before or follow another, or the two events may be unrelated.
 - Two events that are causally unrelated is known as concurrent event.

Events And Signals

- There are various types of events:
 - Signal Event : It is a way of expression, transmission of information from one object to another object
 - A signal event happens when two or more items happen at the same time.
 - Various characteristics, instances, relationships, and operations are associated with a signal event.
 - A signal's attributes serve as a parameter.
 - A signal event is used for sending and receiving a signal.
 - It is a message between objects, while a signal event is an occurrence in time

Events And Signals

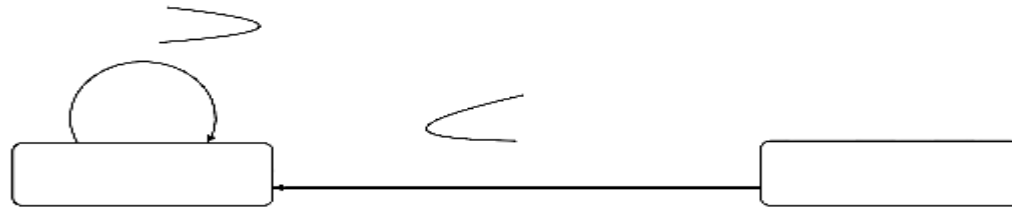
- Call Event :
 - It refers to an operation transmitted from one object to another. It is used as a trigger to change the state of a state machine.
 - In this event, the sender sends a message and waits for the acknowledgement from the receiver's end. For example, a customer or user wait for the OTP at the time of some online transactions.



Call Events

Events And Signals

- Time Event and Change Event :
 - It is an event that occurs at an accurate time or the passage of a given time interval. It is denoted by the “after” followed by the parenthesis expression that measures the time duration.
 - The change event is denoted by “when” followed by a boolean expression in parenthesis.



Time and Change Event

Events And Signals

- Sending and Receiving Event :
 - At least two objects are involved in the calling and signaling event: the object on which the event has been directed for transmitting or initiating the operation.
 - Any class instances (objects) are capable of sending or invoking a signal for a receiving object. When an entity passes a signal, the user transmits the message and afterward proceeds with its control flow without waiting for the receiver to respond.

Events And Signals

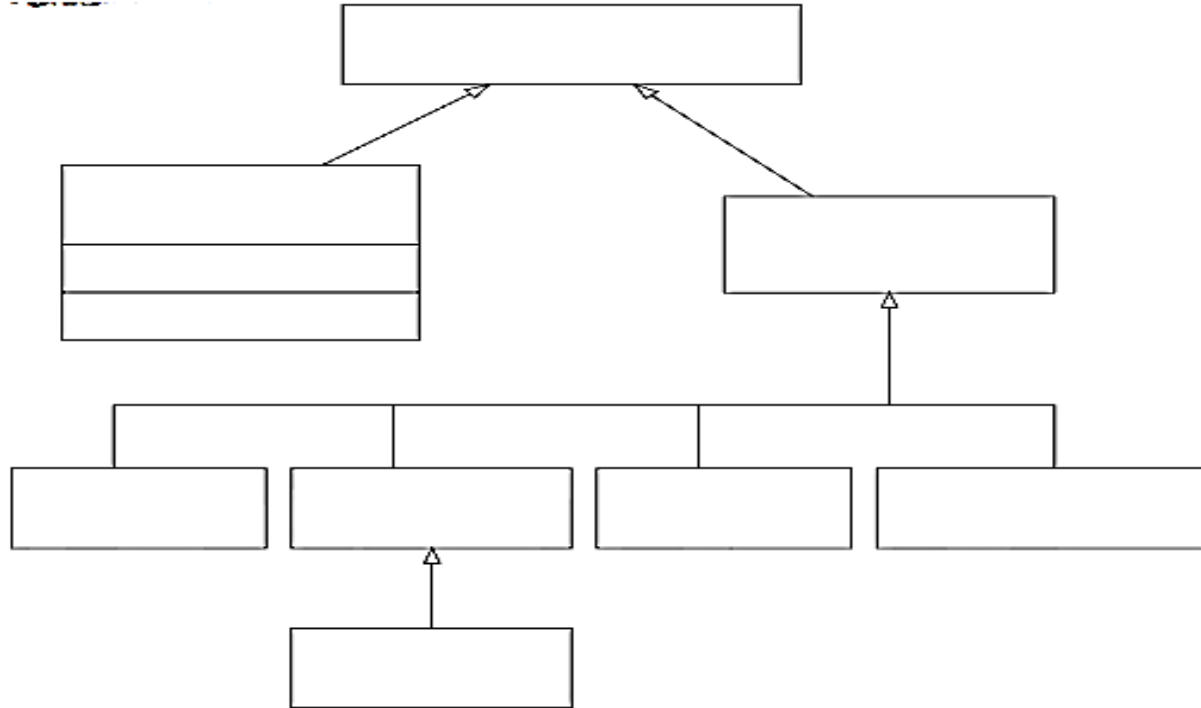
- Signals

- A signal is used to present an object which is transmitted by one object and received by another object asynchronously.
- Every signal has instances, attributes, and relationships that perform various operations. A class instance can receive a class or signal event.
- If the event is a synchronous call event, the sender and receiver are met during the operation. It means the flow of control of the sender is suspended from the receiver flow of the control until the operation is terminated.
- In this case, the sender and receiver do not lock. The sender sends the signal continuously without the wait a response from the receiver end. In such cases, the event may be lost, it may activate the receiver's state machine, or it may invoke a call method.

Common Modeling Techniques

- Modeling a Signal Family
 - Consider all types of signals that can be received by a given collection of active objects.
 - Search for the common types of signals and put them in an inheritance generalization/specialization hierarchy.
 - Search for polymorphism in these active objects' state machines. Adjust the hierarchy when you find polymorphism by introducing intermediary abstract signals
- Modeling Families of Signals Diagram -

Common Modeling Techniques

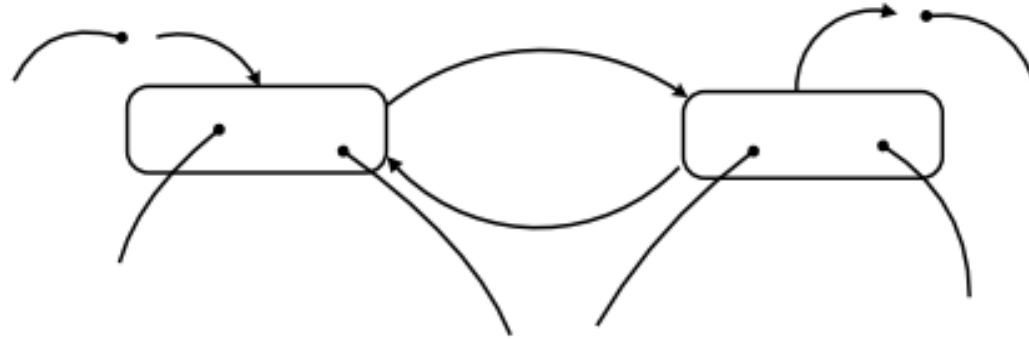


State Machines

- State machine is a specification of the dynamic behavior of a particular class, objects, use-cases, and systems. When you create a state machine, the object attaches to the system to become the master of the state machine.
- State
 - A state contains five parts: name, input/outcome behavior internal transitions, and, transitions performed without any state change.
 - Substates – A state's nested configuration that includes disjoint or concurrent substates.
 - Deferred events - A collection of events that were not processed in this state, but in place of delayed object handling in a different state.

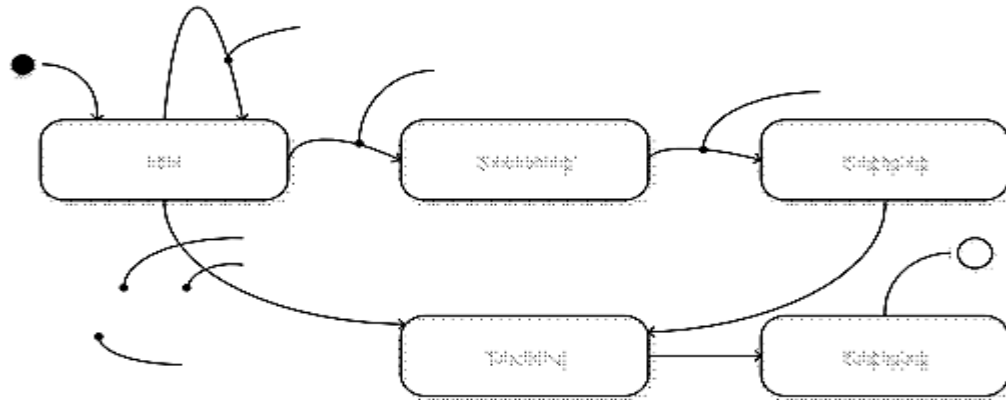
State Machines

- Initial state – It shows the default starting point for the state machine or substate and is seen as a full black circle.
- The final state – In this state, machines or enclosures are closed and are shown as a filled black circle with an unfilled circle.
- Pseudo-states - The first and last states are known as pseudo-states.



State Machines

- Transitions
 - Transition refers to the relation between two states; the first state performs some action and reaches the second state. Transactions are executed when one event is defined, and conditions are met. Parts of a Transition -



State Machines

- Five parts of a transition are:
 - Source state – The transition's effect on the state.
 - Event trigger – A stimulation that can activate a source state to fire when a guard condition is met.
 - Guard condition – Boolean expression is evaluated when the transition is activated by the reception of the event trigger.
 - Action – An executable atomic calculation that may be directly performed on the object that owns the state machine and indirectly on the other objects that are visible to the object.
 - Target state – It is activated after the completion of the transition

State Machines

- Event Trigger
 - In the context of state machines, an event is the presence of a signal that may trigger a state transition.
 - Various changes of state can involve events like signals, calls, and time passing.
 - The symmetric event trigger is also possible in a transition.
- Guard Condition
 - It is a Boolean function wrapped in square brackets after the trigger event.
 - It is calculated after the transition's trigger event takes place.
 - It is only calculated once for each transition as the event occurs, but it can be re-evaluated if the transition is reactivated.

State Machines

- Action

- An action is a discrete measurement that is executable, i.e. it cannot be disrupted by an occurrence and ends.
- Actions involve calling to work, making, removing, and transmitting a signal to an object.
- Other activities can cause an activity to be disrupted.

- Substates

- A substate is a state that is nested within another one.
- A composite state consists of substates.
- A composite state can contain either concurrent (orthogonal) or sequential (disjoint) sub-states.

STATE MACHINES

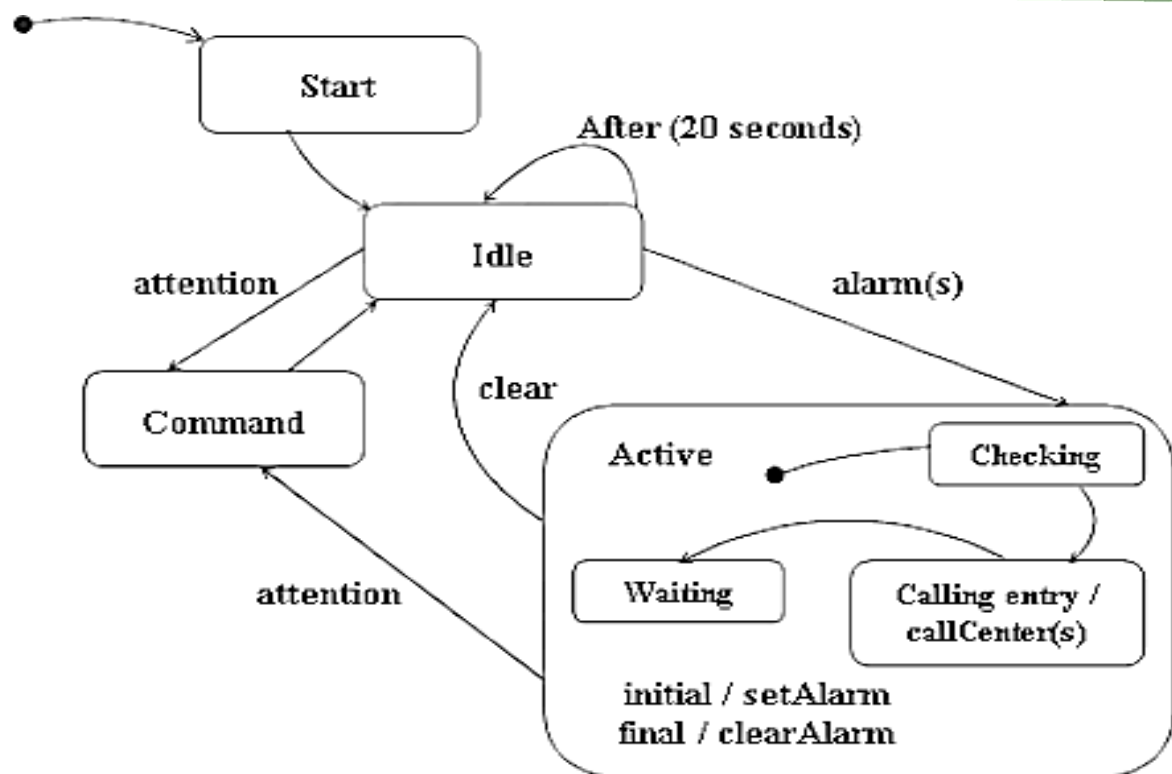
- Sequential Substates
 - Sequential substates are those substates in which each state can effectively exercise an occurrence common to the composite states at any time.
 - Sequential substates split the composite state's, space into disjoint states.
 - There can only be one initial state and one final state in a nested sequential state machine
- Concurrent Substates
 - Concurrent substates specify two or three state machines operating in parallel in the enclosing object context. Execution of these concurrent substates continues in parallel.

State Machines

- While simulating an object's lifetime followings should be considered:
 - Set the original and final entity states. You should state the pre and post conditions of the initial and final states to lead the rest of your model.
 - Determine the events to which this object will respond; if you have already defined these in the interfaces of the object. If you don't already define them, you would have to think about which objects in the setting would communicate with the object and which events they might display.
 - Describe any initial or final actions, especially if the idiom they cover is being used in the state machine.
 - Substates may be used to extend these states as needed.

State Machines

Modeling the Lifetime of An Object



Processes And Threads

- An active class is an object that has a process or thread. The class can monitor the operation of the process, while an active class is a class of instances.
- A process itself is a heavyweight flow that can run concurrently with many other processes
- A thread is a lightweight flow that can run concurrently with several other threads in the same process

Processes And Threads

- **FLOW OF CONTROL**

- In a linear method, there is only one control flow, which means that only one thing will happen at a time.
- Whenever a sequential program is running, control begins at the beginning of the program, and procedures are executed in that order.
- If multiple users executes the concurrent programs, a sequential program would process only at a time.
- The queuing or dispatching of any concurrent external events is known as a flow of control.

Processes And Threads

- **Classes and Events :** There are different types of classes Active class and Normal class:
 - Active class have some special properties. It represents the flow of control while the Normal class has not any such flow control. However, active classes and normal classes behave implicitly called passive classes because they cannot control flow independently.
 - The properties of active classes are the same as those of regular classes. Its instances are created. Attributes and operations are available in active classes. It is used to describe relationships of association, generalization, and connection.

Processes And Threads

- Standard Elements : The UML uses various standard elements, but most of the standard stereotypes which are applied to active classes are –
 - Process : It is a heavy weight process, in which the operating system involves running more than one process at a time. These processes are run independently in a separate and autonomous environment. One process communicates with another process using communication techniques like signals, shared memory, semaphore files, etc. Processes are not nested with each other. If there are several processors on the node, then concurrent processes may execute at the same time.

Processes And Threads

- **Threads** : It is a lightweight process in which threads execute inside a process and share the same address space with other threads of the process. In java programming, a thread is used as a class thread. All the threads in the process are connected to access the same resources inside the same process. Threads do not follow the nested concepts inside one another.
- **Communication** : When objects participate with another object by passing messages from one to the other, it is called communication.

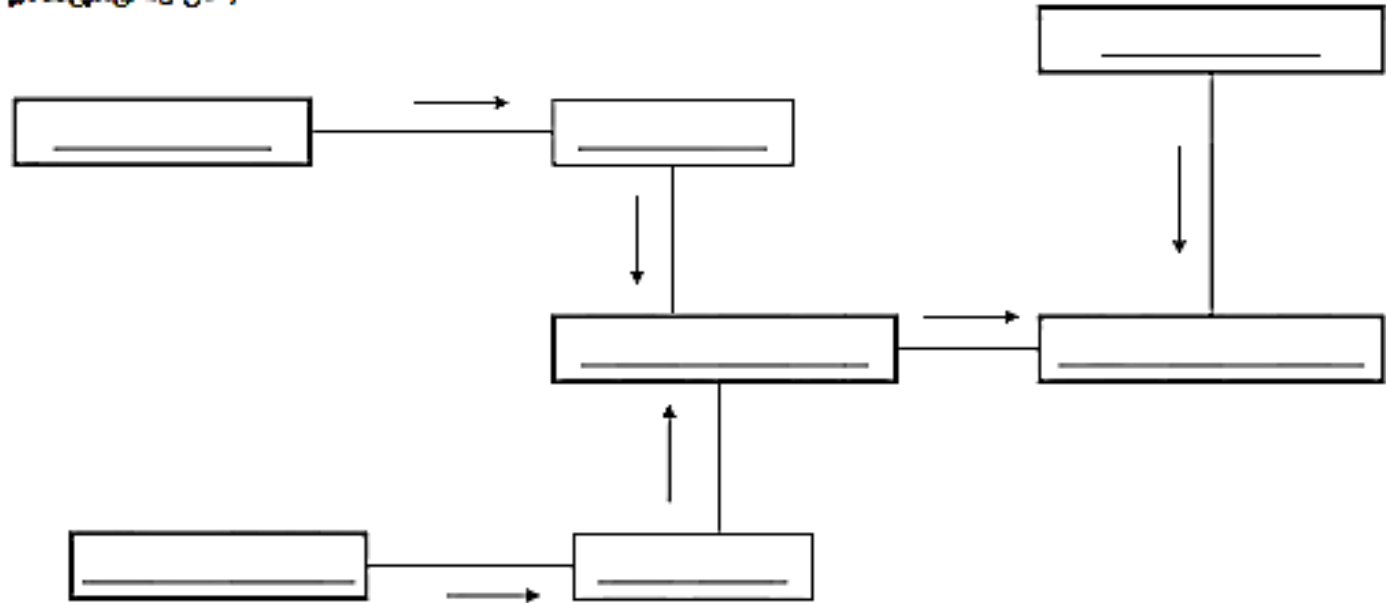
Processes And Threads

- Synchronization : If a flow goes through an operation, we state that the control locus is in operation at a specific time. We may also assume that since the operation is calculated for such classes, the control locus is in one particular class instance at any given moment.
- A single process (and hence in an object) can have multiple control flows, and different operations can have different control flows. But, in a single object, there are always many control flows. Types –
 - Sequential Model,
 - Sequential Model,
 - Concurrent Model.

Processes And Threads

- Common Modelling Techniques
 - Modeling Multiple Flows Of Control
 - Multiple control flows are complex to create. It is challenging to build a system that includes multiple control flows.
 - Multiple control flows are complex to create. It is challenging to build a system that includes multiple control flows.
 - Propose a structured separation of duties between such classes and then discuss the passive and active classes with each participant in the system.
 - The diagram catches those static options in class, which highlights each active class.
 - Consider the dynamic working of each class group.

Processes And Threads



Modeling Flows of Control

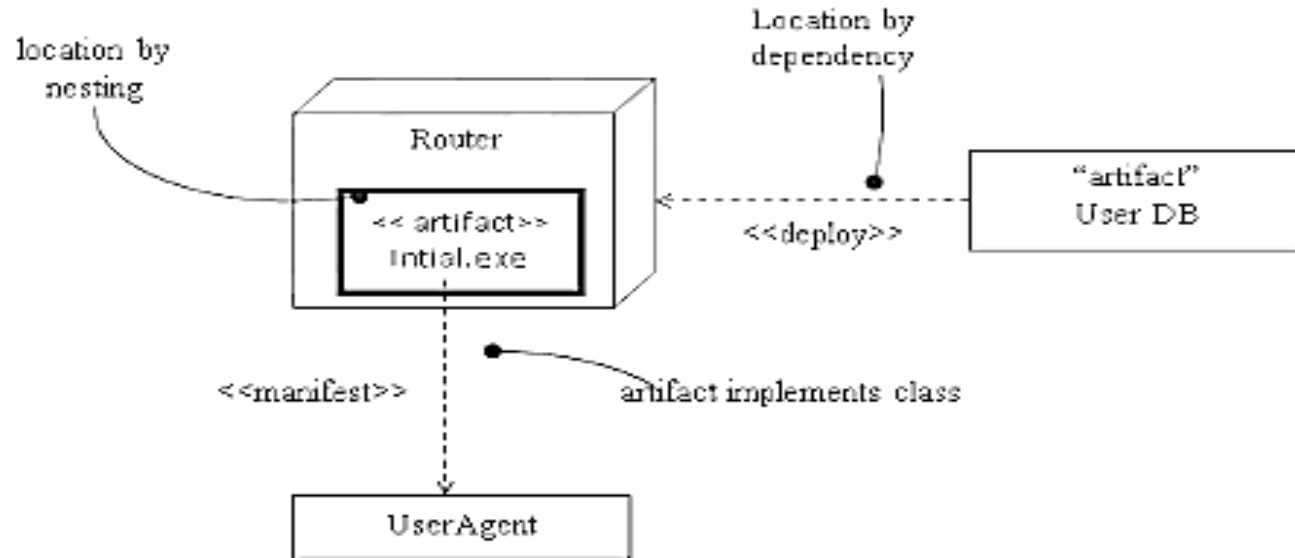
Time And Space

- We need time and space modeling because a distributed framework requires the physical transfer of elements across nodes. These nodes may involve multiple physical processors in the same box or even represent computer systems located half a world away from each other AND,
- It is really hard to have a good model dealing with these distributed and real-time processes, which exposes systems, time, space, and functionality.

Time And Space

- Time : Real-time systems are time-critical systems. In such systems, events may arise at regular or irregular times. The reaction to the event itself may occur at pre-arranged absolute times or at predictable times
- Locations : In distributed systems, elements are physically distributed between the system nodes. For specific systems, components are set, while in some systems, which are loaded, components need to be migrated from one node to other nodes in other systems.

Time And Space



Location

State Chart Diagram in UML

- A State chart diagram explains the different states of a component in a system. It can be used to define the different states of an object. External and internal events control these states
 - It is used to present a state machine that displays the flow of control from one state to another state.
 - It is essential for designing executable systems via forward and reverse engineering.
 - It helps in modeling an object's lifespan.

State Chart Diagram in UML

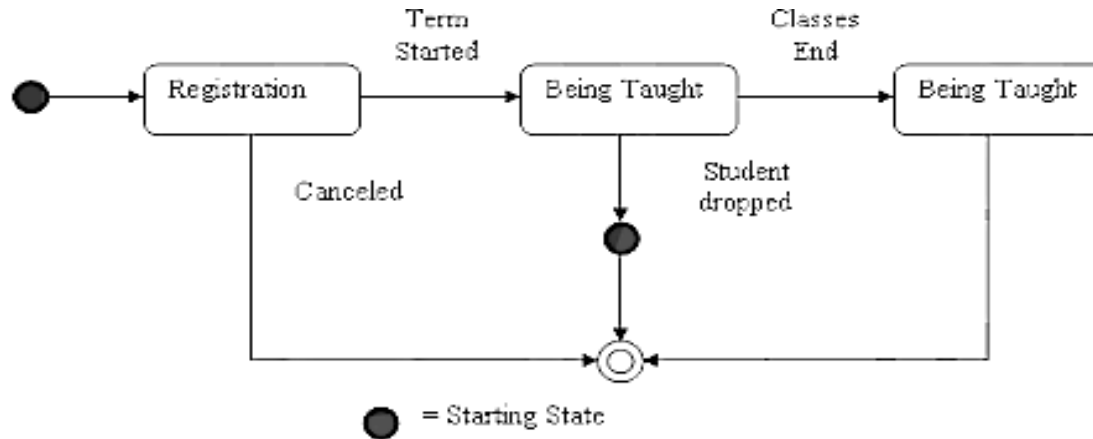
- Purpose of State Chart Diagram
 - The state diagram depicts the flow of control from one state to the next state. The term “state” refers to a situation in which there is an object, and it changes when some event is triggered. The main purpose of the state chart diagram is to design an object’s lifespan from development to termination.
 - State chart diagrams are often required for forward and reverse engineering of a system. It is used to model the reactive system.
- Below are the key goals of using state chart diagrams:
 - To model a system’s dynamic aspect.
 - To model a reactive system’s lifetime.
 - To identify an object’s different states throughout its existence.
 - Define a state machine for modeling an object’s states.

How to draw a State Chart Diagram

- To describe the states, state chart diagrams are very significant. States can be defined as the state of objects when a specific event occurs. Before drawing a state chart diagram, we should clarify the following points:
 - Identify the significant objects to be examined.
 - Define the states.
 - Describe the events.

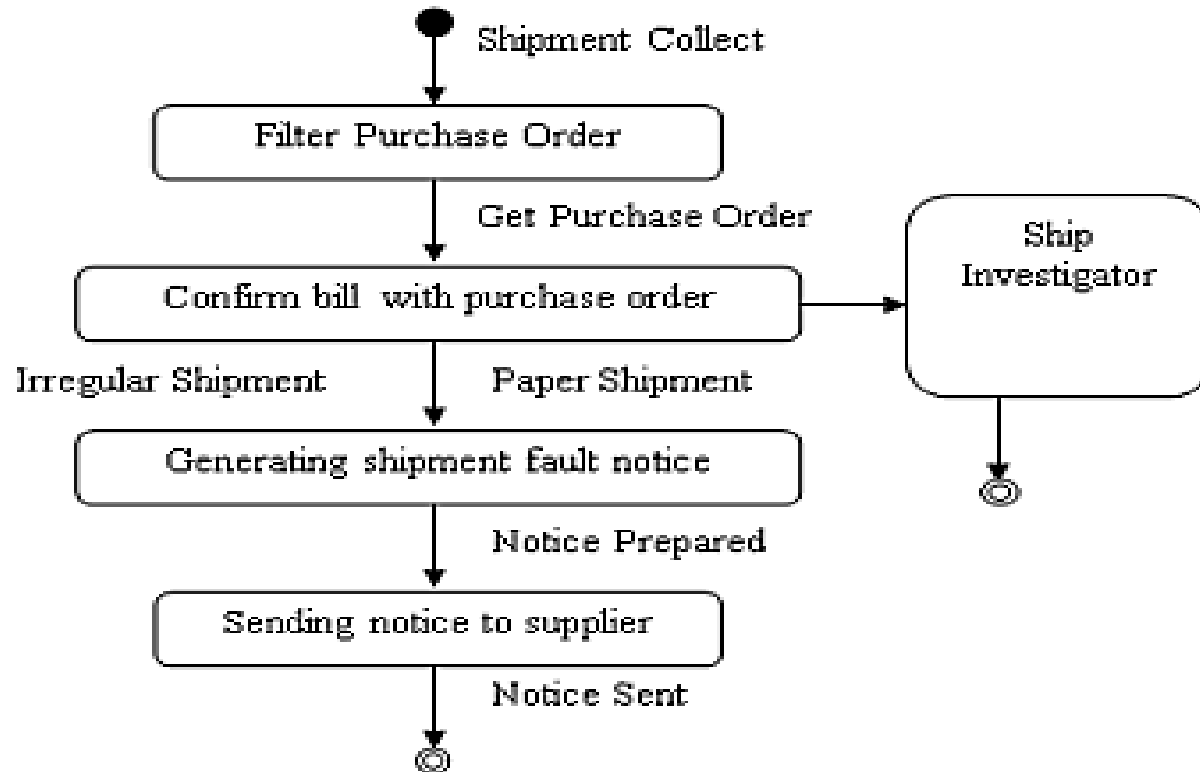
How to draw a State Chart Diagram

Student States in a Semester :



- Let's take an example and draw a state chat diagram of Shipping Process.

How to draw a State Chart Diagram



Where to Use State Chart Diagrams

- The common use of state chart diagram is:
 - Model system object states.
 - The reactive framework can be modeled. The reaction mechanism is made up of reactive objects.
 - To recognize events for improvements to the state.
 - For reverse and forward engineering.
- State chart diagrams are used to simulate the system's states as well as the events that occur. State map diagrams are used to clarify various states of an entity over its lifespan, which is very useful when developing a system.

Important Topics

- Events
- Signals
- Action
- Communication
- Flow of Control
- State Chart Diagram

Summary

- In this session, basic norms, concepts, and style guidelines for constructing behavioral or dynamic UML models are explained. Unit explained sequence, collaboration, state and activity diagrams, and their elements. Behavioral modeling is the modeling of a system's behavioral or dynamic component.
- Further, this session explains the interaction between the components that make up the system functionality, and it's used to identify how a system meets its requirements, parts, and relationships in a specification model for the system, subsystem, and class specification.
- Also, it is explained that the class, object, sequence, collaboration, state, activity diagrams, and their model components make up a specification model.

*Thank
You*



Object Oriented Analysis and Design

Block-1 Unit-5

Architecture Model

Topics to be Covered

- Introduction
- Model
- Architecture Modeling
- Implementation Diagram
- Collaboration
- Important Topics
- Summary

Introduction

- Object-oriented concepts play an important role in the development of the software system.
- We need to model using object-oriented concepts.
- We need to apply the object-oriented concepts appropriately and efficiently to develop error-free software.
- Such modeling provides us with a goal-oriented way of analyzing real-world problems. System modeling is required to better understand the assigned problems and understand the better coverage of the developing stages of the software system

Model

- A model is an abstract representation of a system, which promotes a better understanding of its representation. It helps to give better insight into the development of the system among communication of project team members.
- A model is a way to represent something in the same or in a different medium. It always captures the important aspect of a thing and omits the unwanted attributes or components associated with it.
- We need a convenient medium to express the model. If you want to design a software system model, you need a modeling language such as UML

Model

- **Modeling** is a critical part of any developing software project. It always leads to the final deployment of a good software system. If we want to understand the system in a better way, we need the modeling of the system.
- Modeling helps to visualize and control the system's architecture. So we can summaries the Modeling as:
 - Provides the visualization of the system as per our needs,
 - Help us to reduce the complexity of the system,
 - Facilitate a template for constructing a system,
 - Supports in the testing of the system before its implementation, and
 - Provides communication with the end-users.

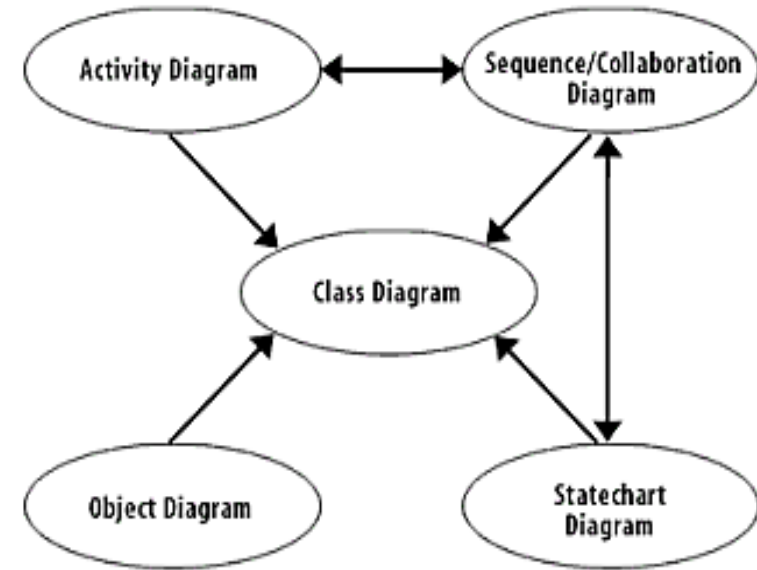
Architecture Modeling

- Architectural model represents the overall framework of the system. This model contains both structural and behavioral elements of the system. It can be defined as the blueprint of the entire system.
- Aspects of The Architectural Modeling –
 - The dynamic aspect of the architectural model is represented by a collection of component and connection configurations which are controlled by nodes.
 - The functional aspect of the architectural model includes the system functions and the data-flows i.e. interaction between them. It defines how the functions will operate together to perform the system goal.
 - The non-functional aspect of the architectural model describes the system's operational capabilities and constraints that enhance its functionality.

Architecture Modeling

- Logical Architecture

- If we want to understand the structure and organization of the system's design, we use an architectural view which is also known as logical architecture or logical view. The logical architecture starts with the end user's functional requirements and provides a top-down abstraction of the system's overall design.



Logical Architecture of a system

Architecture Modeling

- Physical Architectural
 - The physical architectural models are the systems' dynamic (run time) architecture. It shows the configuration of the topology of software components on the physical layer as well as the physical connections between these components.
 - This model provides detailed information about how elements or components will be deployed across the system infrastructure.

Implementation Diagram

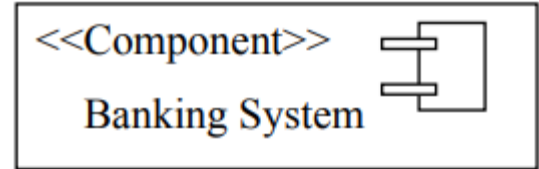
- The diagram, which is closely correlated with the Use Case, is the implementation diagram.
- The purpose behind this coupling is to highlight which types of design elements will implement the functionality of Use Case in a novel system.
- This diagram provides the physical environment of our system
- Types of implementation Diagram are :

Implementation Diagram

- **Component Diagram:** Component diagrams are used to show how the physical components in a system have been organised. It shows the set of components and their relationships in the system.
 - Graphically, a component diagram is a collection of vertices and arcs. The essential components of component diagrams are :
 - components,
 - Interfaces
 - ports and
 - connectors

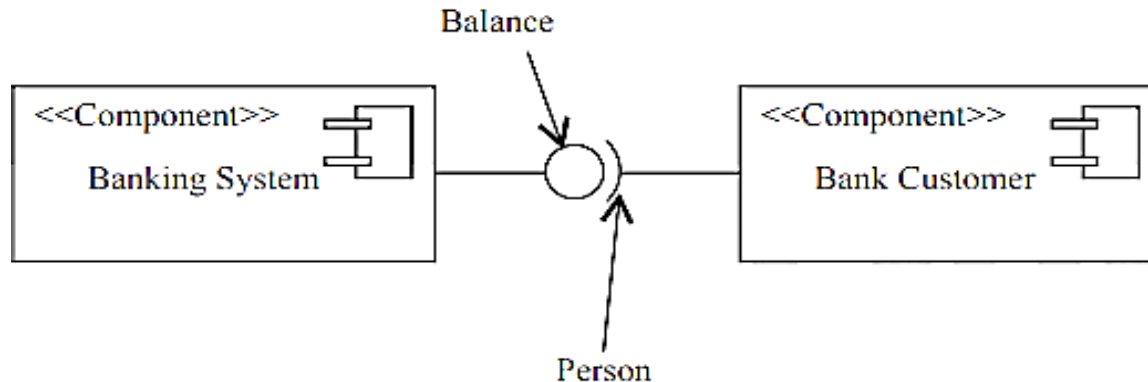
Implementation Diagram

- Components : In the component diagram, the component represents a reusable piece of software and is the physical and replaceable unit of the system



- Interfaces : Components of the system use interfaces to communicate with each other. It defines the component's interaction details. It is a collection of operations that are used to specify a service of a class or a component.

Interface :



Implementation Diagram

- **Ports :** Ports are used by the component for its interactions with its environment and also to control the implementation of all the operations in the component. Along the edge of the component, a port is shown as a small square
- **Connectors :** A connector is used to show a link between the ports of components. It also specifies the two or more classifiers.

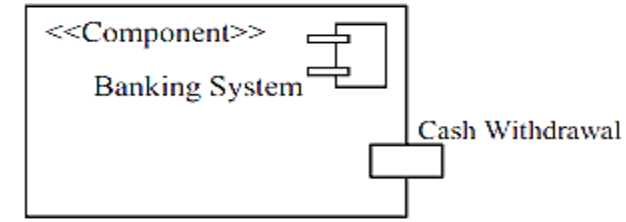
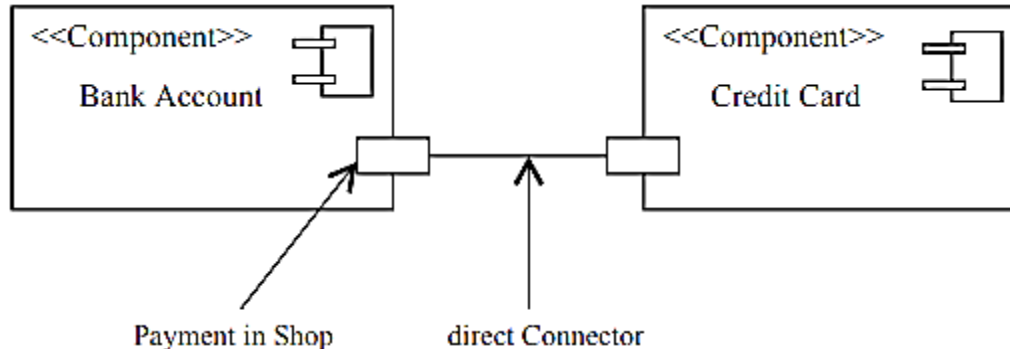
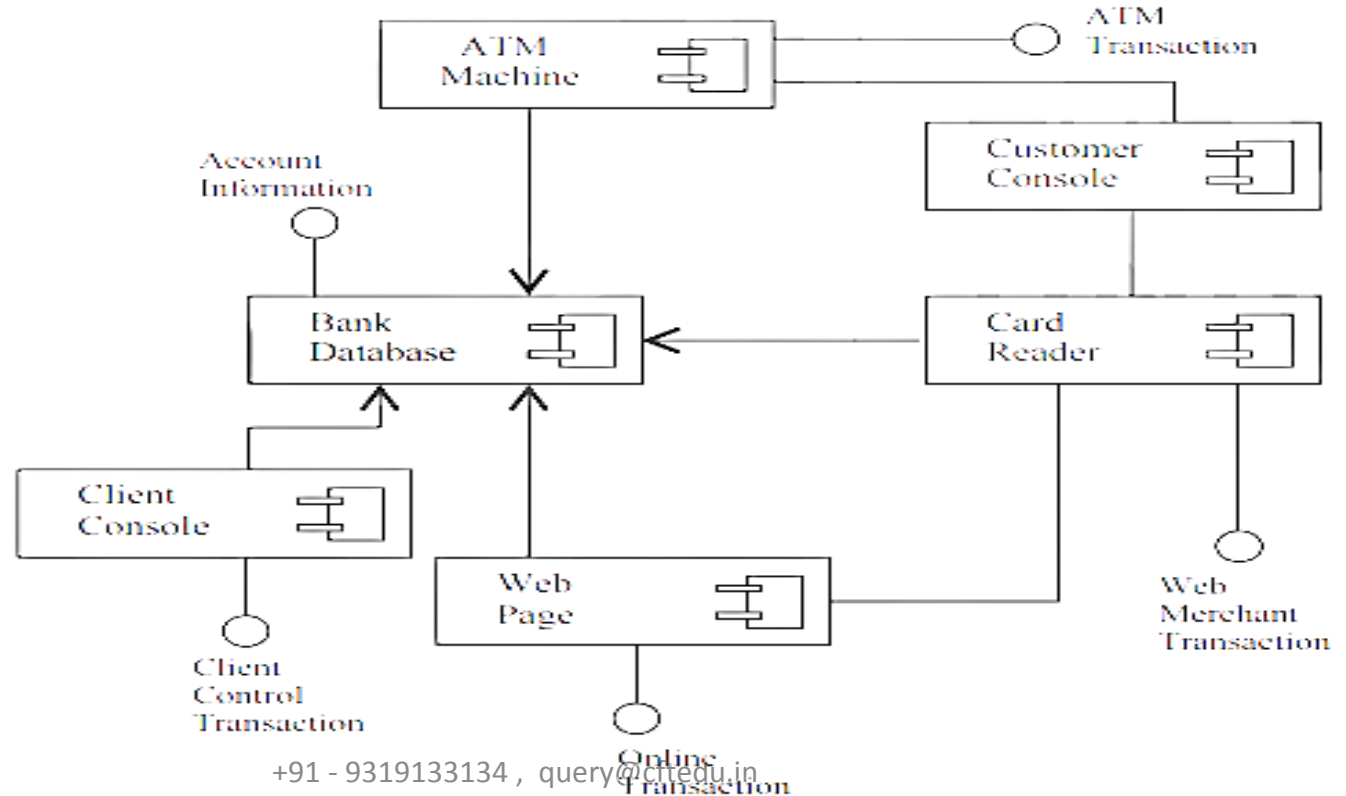


Figure 5.5: Port

Implementation Diagram

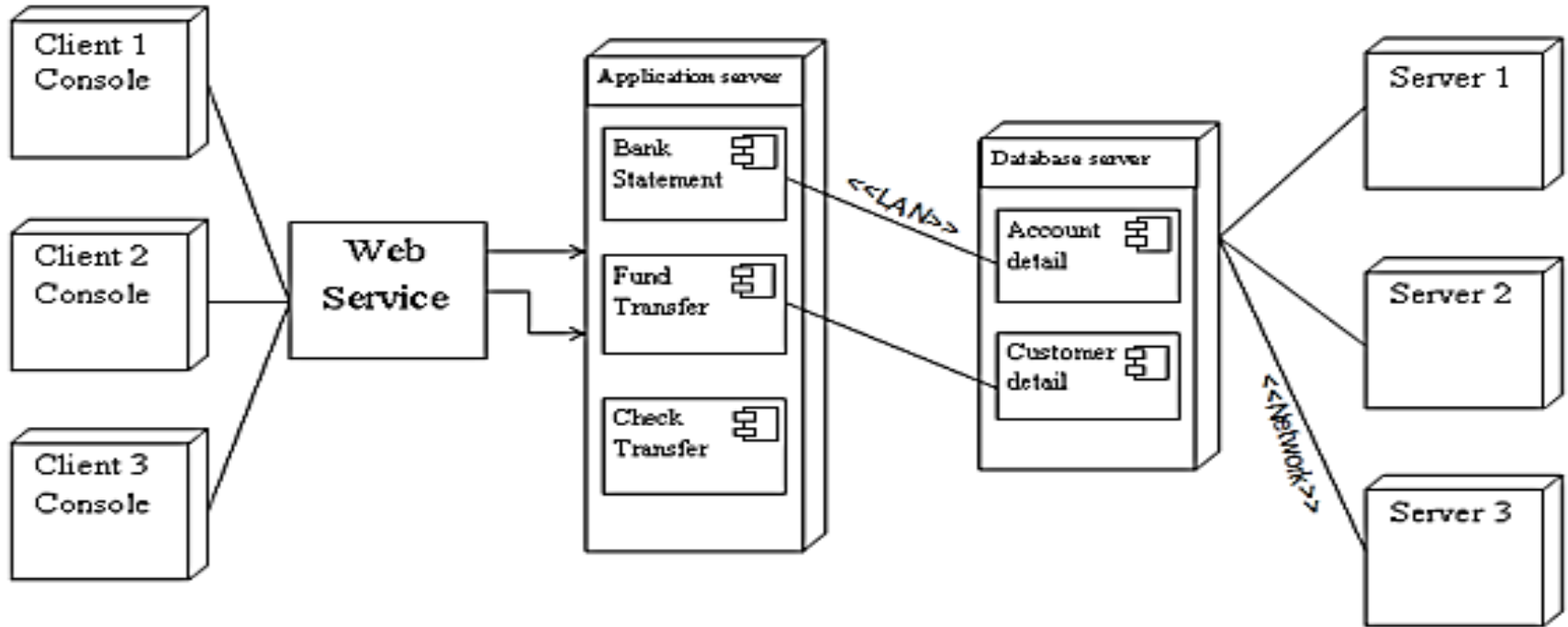
Component diagram of ATM System:



Implementation Diagram

- Deployment Diagram : To address the static deployment view of architecture, we use the deployment diagrams. Deployment diagrams are related to component diagrams in which a node typically encloses one or more components.
- In this diagram we use to indicate the physical collection of nodes that serve as the platform for execution of our system.

Implementation Diagram

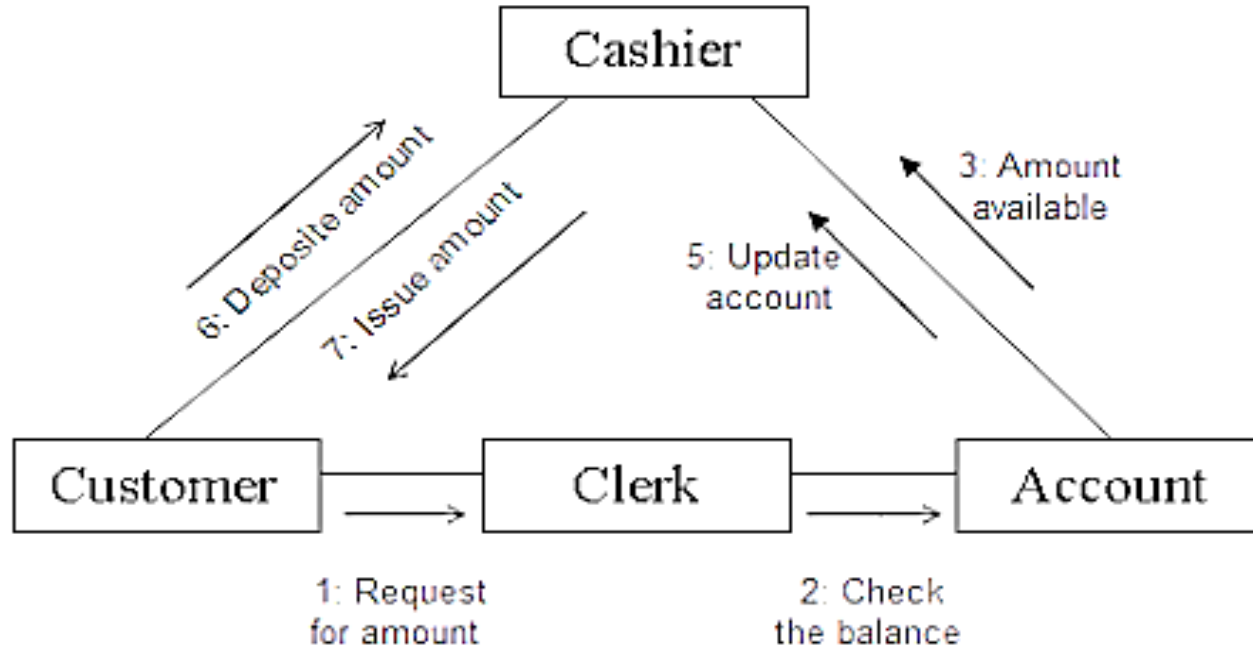


Example of Deployment diagram of Banking System

Collaboration

- Collaboration defines an interaction and describes how the elements work together to provide the system's behavioral requirements so that it must be larger than the combination of all elements.
- The combination of elements may be classes, objects, interfaces, components, nodes, use cases or other elements.
- We express the details about collaboration by using the collaboration diagram, which is part of the interaction diagram. Collaboration is represented in the form of the ellipse with dashed lines in its graphical form. It represents cooperating entities and connectors. Collaboration acts like a template in which roles are defined for classifiers, and during runtime, these classifiers are bound to their respective roles joined by the connectors.

Implementation Diagram



Example of Collaboration Diagram for Banking System

Important Topic

- Architecture Modeling
- Logical Architecture
- Physical Architecture
- Component Diagram
- Deployment Diagram

Summary

- The architectural model represents the complete framework and the outline of a complete software system that contains both structural and behavioral diagrams of the system.
- The logical model is a static view of the objects and classes that make up the design/analysis space and leans on patterns the most.
- The implementation diagram is very much helpful in the physical implementation of the design
- Component diagram models the physical aspect of the system, including software components, source code, runtime code, & executable
- Deployment diagrams are related to component diagrams in that a node typically encloses one or more components
- Collaboration acts like a template in which roles are defined for classifiers, and during runtime, these classifiers are bound to their respective roles (instances) joined by the connectors.

*Thank
You*



Object Oriented Analysis and Design

Block-2 Unit-6

Object Modeling

Topics to be Covered

- Introduction
- Advanced Modeling Concepts
- Multiple Inheritance
- Generalization and Specialization
- Meta Data and Keys
- Integrity Constraints
- An Object Model
- Important Topics
- Summary

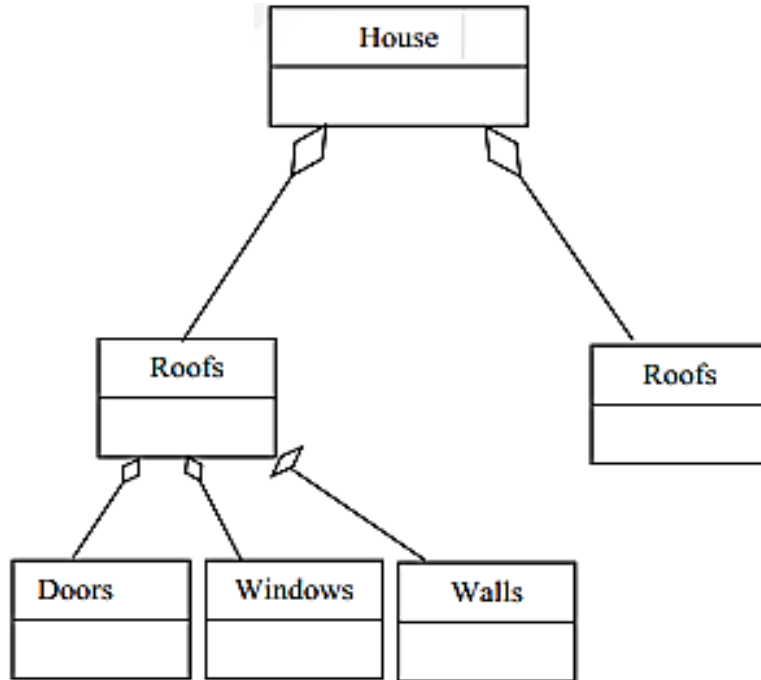
Introduction

- In the previous session we have learned the basics of OOA and OOAD. Also, we have seen how to draw and use different UML diagrams for visualizing, specifying, constructing, and documenting
- In this session, we will discuss Advanced Modeling Concepts, including, Aggregation & Abstract Class, Generalization & Specialization, Multiple Inheritance and designing of an Object Model.

Advanced Modeling Concepts

- Aggregation
 - Aggregation represents the has-a or part-of relationship. Aggregation is tightly coupled form of association. An aggregation depicts a complex object as an assembly of many object components.
 - These components are also objects, you may characterize a house in terms of its roof, floors, foundation, walls, rooms, windows, etc. A room may, in turn, be, composed of walls, ceiling, floor, windows, and doors, as represented in Figure.

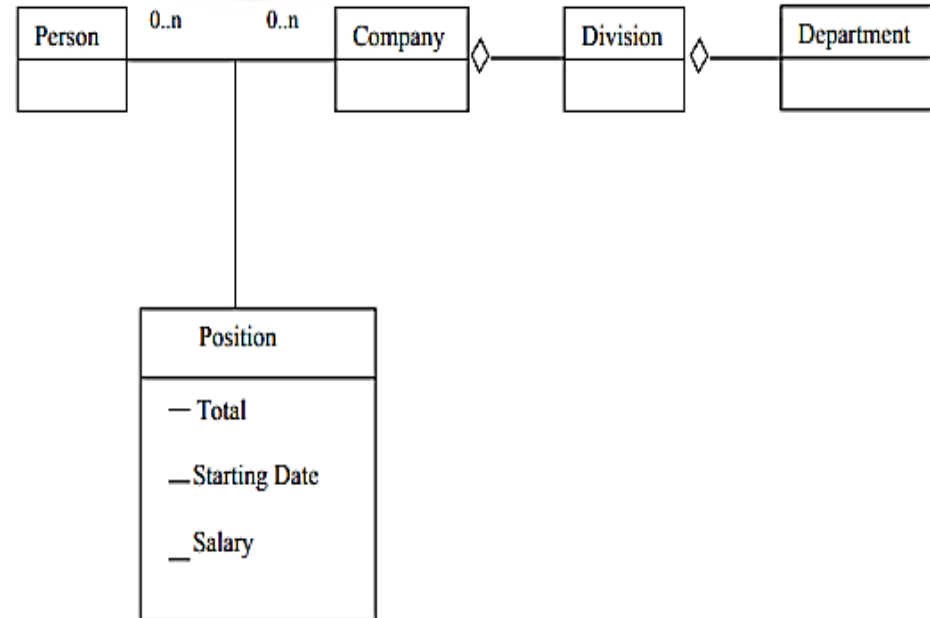
Advanced Modeling Concepts



- Hence a house can be seen as an aggregation of many objects.
- Object aggregation is very helpful in describing models of the real-world that are composed of other models.
- Aggregation is a special kind of association, adding additional meaning in some situations.

Advanced Modeling Concepts

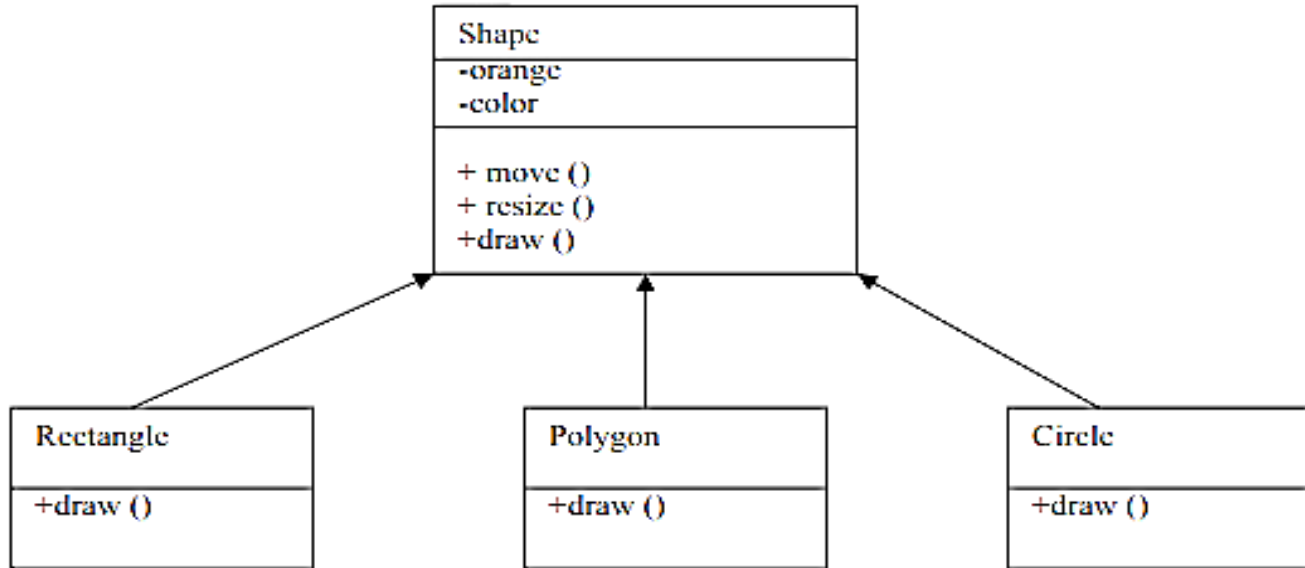
- Composition:
 - A stronger form of aggregation is called composition.
 - Composition means parts of the system may be created after a composite system is created.
- Remember that in such systems parts destruction of the composite.
- In UML, filled diamonds, as shown in Figure.



Advanced Modeling Concepts

- **Abstract Class** : An abstract class is used to specify the required behaviors (operations or methods) of a class without providing their actual implementations. In other words, we can say that methods without implementation (body) are part of abstract classes
- An abstract object class has no occurrences. Objects of abstract classes are not created but have child categories containing actual occurrences.

Advanced Modeling Concepts



How abstract and concrete classes are related. Consider the example of shapes.

Advanced Modeling Concepts

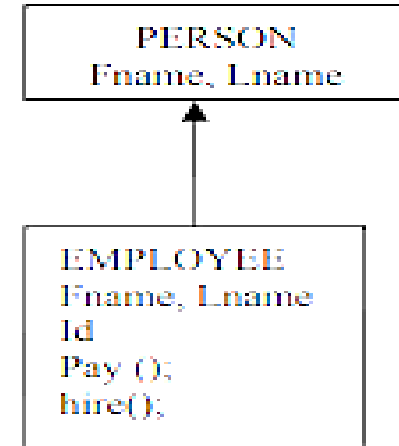
- Why are abstract classes created ?
 - To organize many specific subclasses with a superclass that has no concrete use,
 - An abstract class can still have methods that are called by the subclasses. We have to take this correct point into consideration in the case of abstract classes,
 - An abstract method must be overridden in a subclass; you cannot call it directly, and
 - Only a concrete class can appear at the bottom of a class hierarchy.

Multiple Inheritance

- Multiple inheritance is a feature in which an object or class can inherit features from more than one parent object or parent class.
- Inheritance gives several benefits such as:
 - Reduce duplication and increase reusability by building on what you have already created.
 - Organize the classes of the system in the ways it may appear in real-world situations in the problem domain.

Multiple Inheritance

- Multiple inheritance extends this concept to allow a class to have more than one parent class and inherit features from all parents. Thus, information may be mixed from multiple sources. It is a more complex form of generalization.
- Example of Inheritance in given diagram :



Multiple Inheritance

- Multiple inheritance provides greater modeling power for defining classes and enhances opportunities for reuse. Using multiple inheritance object models can more closely reflect the structure and function of the real world

Generalization and Specialization

- Generalization means extracting common properties from a collection of classes and placing them higher in the inheritance hierarchy in a superclass.
- Generalization and specialization are the reverse of each other. An object type hierarchy that models generalization and specialization represents the most general concept at the top of an object type: hierarchy as the parent and the more specific object types as children.
- When generalizing (as in the real world), care should be taken so that the property makes sense for every single subclass of the superclass. If this is not the case, the property must not be generalized.

METADATA AND KEYS

- **Metadata**
- Metadata can be defined as “such set of data which describes other data”.
- For example, when we have to describe an object, we need to have a description of the class from which the object is instantiated.
- **Keys**
- To identify an object, attribute or combination of attributes is used. This attribute or , combination of attributes called a key.

METADATA AND KEYS

- A primary key is an attribute (or combination of attributes) that uniquely identifies an object instance.
- For example, the combination of last name, first name, and middle initial might be used to identify a customer or employee object instance.
- A secondary key is an attribute (or combination of attributes) that may not uniquely identify an object instance but can describe a set of object instances that share some common characteristic.

Metadata And Keys

- **INTEGRITY CONSTRAINTS**
- Referential Integrity
 - Constraints on associations should be examined for referential integrity implications in the database models.
- Insert Rules
 - These rules determine the conditions under which a dependent class may be inserted, and they deal with restrictions that the parent classes impose upon such insertions. The rules can be classified into six types : -

Metadata And Keys

- **Dependent:** Permit insertion of child class instance only when the matching parent class instance already exists
- **Automatic:** Always permit insertion of a child class instance. If the parent class instance does not exist, create one
- **Nullify:** Always permit insertion of the child class instance.
- **Default:** Always permit insertion of a child class in séance.
- **Customized:** Allow child class instance insertion only if certain validity constraints are met.
- **No Effect:** Always permit insertion of the child class instance. No matching parent class instances may or may not exist. No validity checking is performed.

Metadata And Keys

- **Domain integrity**

- These integrity rules define constraints on valid values that attributes can assume. A domain is a set of valid values for a given attribute, a set of logical or conceptual values from which one or more attributes can draw their values.
- Domain characteristics include things such as:
 - Data type
 - Data length
 - Allowable value ranges
 - Value uniqueness
 - Whether a value can be null or not.

Important Topics

- Aggregation
- Abstract Class
- Multiple Inheritance
- Generalization and Specialization
- Meta Data and Keys
- Integrity Constraints

Summary

- This session explained basic aspects of object modeling, which includes a discussion on models based on objects (rather than functions) would be more stable over time hence the object-oriented designs are more maintainable. Objects do not exist in isolation from one another.
- There are different types of relationships (aggregation, composition, generalization) which are used to represent object models.

*Thank
You*



Introduction to Object Oriented Modeling

Block-2 Unit-7

Dynamic Modeling

Topics to be Covered

- Introduction
- Event
- State and State Diagram
- Elements of a State Diagram
- Advanced Concepts in Dynamic Modeling
- Concurrency
- Dynamic Model
- Important Topics
- Summary

Introduction

- The dynamic model shows the time-dependent behavior of the system and the objects in it. Events can be defined as “something that happened at a point of time”.
- The logical correctness of events depends on the sequences of interactions or events. The dynamic model is important for interactive systems
- we will discuss the basic concepts of dynamic modeling, covering events and states. The major dynamic modeling concepts are events, which represent external stimuli, and states, which represent the values of objects

Events

- We can say an event is some action-oriented result, such as mouse click, applying a brake on a car etc. We might have experienced that the appropriate action takes place whenever we click on a mouse.
- We may observe that an event has no specific time period duration. We can click on a mouse and keep it pressed as long as we want. So, as far as events are concerned, nothing is instantaneous. An event is simply an occurrence.

Events

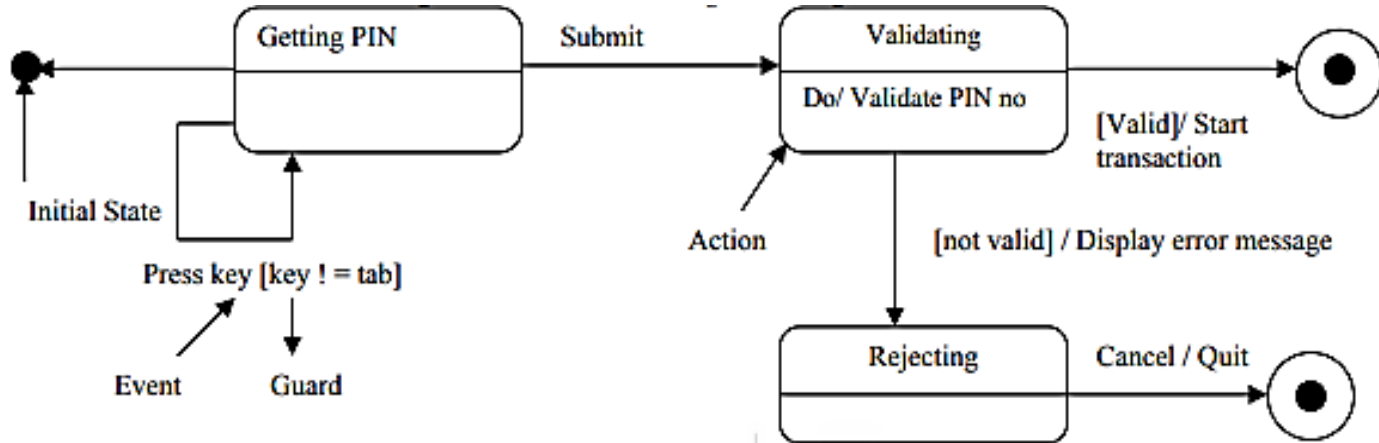
- Any two events may occur independently of each other or simultaneously. For example, two trains can depart simultaneously for two different places or depart from the same place, but one after the other. Any two events can be independent as well as dependent on each other.

State and State Diagram

- State : A state is a condition during the life of an object, or an interaction during which it satisfies some condition, performs some action, or waits for some event to occur. An object remains in a state for a finite (non-instantaneous) time.
- Actions are atomic and non-interruptible. A state may correspond to ongoing activity, such as activity being expressed as a nested state machine. Alternately, ongoing activity may be represented by a pair of actions, one that starts the activity on entry to the state and one that terminates the activity on exit from the state
- A state must have initial states and final states.

State and State Diagram

- The state chart diagram shown below represents the login part of an online banking system. Logging in consists of entering a valid social security number and personal id number and then submitting the information for validation.



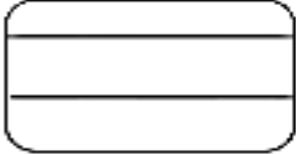
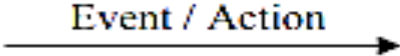
State and State Diagram

- Logging in can be factored into four non-overlapping states:
- Getting PIN, Validating, and Rejecting. A complete set of transitions comes from each state that determine the subsequent state.
- States are rounded rectangles.
- Transitions are arrows from one state to another.
- Events or conditions that trigger transitions are written beside the arrows.
- Our diagram has self-transition on Getting PIN.
- The initial state (black circle) is a dummy to start the action.

State and State Diagram

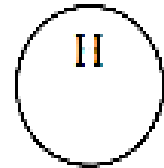
- Final states are also dummy states that terminate the action.
- The action that occurs as a result of an event or condition is expressed in the form of action.
- While in its Validating state, the object does not wait for an outside event to trigger a transition.
- **“The state machine is a graph of states and transitions that describes the response of an object of a given class to the receipt of outside stimuli. A state machine is attached to a class or a method”.**

Elements of a State Diagram

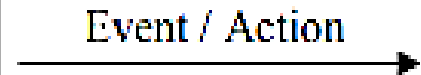
Elements and its Description	Symbol
Initial State: This shows the starting point or first activity of the flow. It is denoted by a solid circle. This is also called a pseudo state , where the state has no variables describing its further and no activities to be done.	
State: Represents the state of an object at an instant of time. There will be multiples of such symbols in a state diagram, one for each state of the object, denoted by a rectangle with rounded corners and compartments (such as a class with rounded corners to denote an object).	
Transition: An arrow indicating the object to transition from one state to the other. The actual trigger event and action causing the transition are written beside the arrow, separated by a slash. Transitions that occur because the state has completed an activity are called “triggerless” transitions.	

Elements of a State Diagram

History States: A flow may require that the object go into a trance or wait state, and on the occurrence of a certain event, go back to the state it was in when it went into a wait state — its last active state. This is shown in a state diagram with the help of the letter *H* enclosed within a circle.



Event and Action: A trigger that causes a transition to occur is called an event or action. Every transition need not occur due to the occurrence of an event or action directly related to the state that transitioned from one state to another. As described above, an event/action is written above a transition that it causes.



Elements of a State Diagram

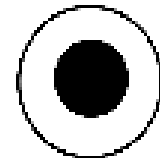
Signal: When an event causes a message/trigger to be sent to a state that causes the transition, then that message sent by the event is called a signal.

<<Signal>>
Event/Action

Event / Action



Final State: A bull's eye symbol shows the end of the state diagram, also called a final state. A final state is another example of a pseudo state because it does not have any variable or action described.



Advanced Concepts in Dynamic Modeling

- **Entry and Exit Actions**

- The following special actions have the same form but represent reserved words that cannot be used for event names:
 - ‘Entry’ ‘/’ action-expression: An atomic action performed on entry to the state.
 - ‘Exit’ ‘/’ action-expression: An atomic action performed on exit from the state.
 - ‘do’ ‘/’ machine-name (argument-list): The machine-name must be the name of a state machine that has an initial and final state. When this state is entered, after any entry action, the execution of the nested state machine begins with its initial state. If the nested machine has parameters, then the argument list must match correctly

Concurrency

- An expansion of a state into concurrent substrates is shown by tiling the graphic region of the state using dashed lines to divide it into sub regions. Each sub region is a concurrent substrate. Each sub region may have an optional name and must contain a nested state diagram with disjointed states
- Composite States : a state can be decomposed using and-relationships into concurrent sub states or using or-relationships into mutually exclusive disjoint substrates. A given state may only be refined in one of these two ways. Its sub states may be refined in the same way or the other way.

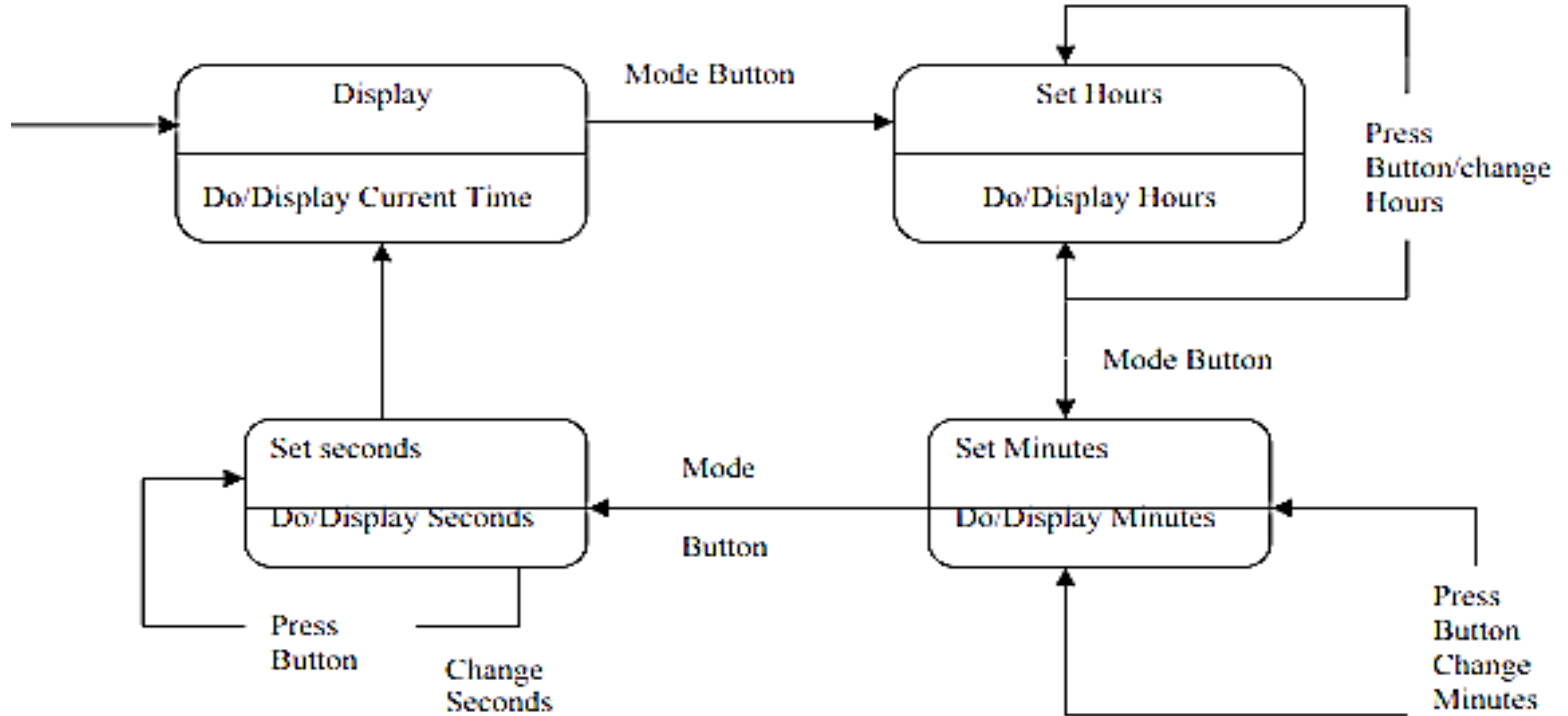
Dynamic Model

- The dynamic model describes those aspects of the system concerned with the sequencing of operations and time - events that cause state changes, sequences of events, states that define the context for events and the organization of events and states. The dynamic model captures control information without regard for what the operations act on or how they are implemented.
- The dynamic model is represented graphically by state diagrams.

Dynamic Model

- A state is an abstraction of an object's attribute values and links, where sets of values are grouped together into a state according to properties that affect the general behavior of the object.
- The state diagram should adhere to OMT's (Osteopathic manipulative treatment) notation and exploit the capabilities of OMT, such as transition guards, actions and activities, nesting (state and event generalization), and concurrency.
- Below is the state transition diagram for a digital watch.

Dynamic Model



Important Topics

- State and State Diagram
- Elements of a State Diagram
- Advanced Concepts in Dynamic Modeling
- Concurrency
- A Dynamic Model

Summary

- The dynamic model is the model which represents the control information : The sequence of events, states and operations that occur within a system of objects. It has scenarios to occur with meaningful constraints.
- An event is triggered by an instantaneous action. One kind of action is sending an event to another object.
- An external event, also known as a system event, is caused by something outside our system boundary.
- An internal event is caused by something inside our system boundary. States may be “nested.” A nested state usually indicates a functional decomposition within the “super” state. The term macro-state is often used for the super state. The macro-state may be composed of multiple micro-states.

*Thank
You*



Introduction to Object Oriented Modeling

Block-2 Unit-8

Functional Modeling

Topics to be Covered

- Introduction
- Functional Models
- Data Flow Diagrams
- Features of a DFD
- Design Flaws in DFD
- Sample Functional Model
- Functional Vs. Object Vs. Dynamic Model
- Important Topics
- Summary

Introduction

- The first Object Model, which is based on classes and objects and the features like encapsulation, inheritance, concurrency and abstraction.
- Next is Dynamic, which depicts states of objects, transition between states and events that trigger the transition from one state to another.
- The third and final model is the Functional model, which elaborates on concepts like how data flows between various objects called actors, processes, and data stores.
- The focus is on the functional model and all its features explained with the help of data flow diagrams up to level 3 decomposition

Functional Models

- A functional model is a graphical representation of the object-oriented analysis model and focuses on various processes to be performed on input data to obtain the desired output.
- The system is viewed as a black box that accepts inputs, transforms them by applying various processes/functions and then produces the outputs.
- A functional model does not show how the transformations occur, when they are performed and why there is a need to do the calculations. .

Data Flow Diagram

- The DFD is a simple graphical technique that represents an entire software system in terms of inputs fed to it, processing applied on these inputs and the outputs produced by the system, and various data stores used internally to store the data/results.
- The focus is on the flow of data input to a system by a source, transformed by the application of various processes and then sent to a destination as output.
- The sources that produce data and the destination that consumes the processed data are known as Actor objects.

Data Flow Diagram

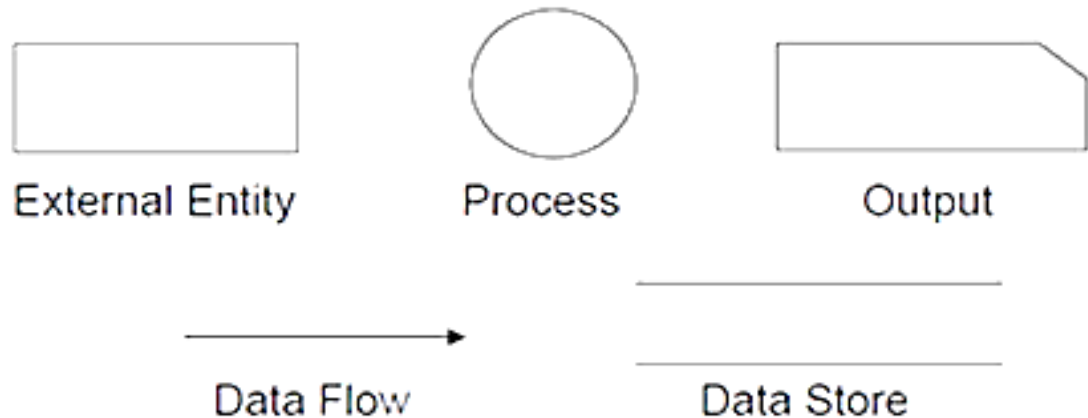
- It is very easy to understand the functionality of a system with DFD as there are very few symbols used in it, and the basis of DFD is to divide a complex problem into smaller components.
- DFD uses a hierarchical approach that starts from an abstract model that depicts high-level functions and further introduces details as we go down the hierarchy and create sub-functions.
- DFD is also called as a bubble chart.

Features of a DFD

- The features of DFD can be explained with the help of primitive symbols used. There are five different symbols used to draw a DFD :
 - Circle: Labelled circles called bubbles are used to show the processing or transformations applied on data. The name of the corresponding function is written inside the bubble.
 - Arrow: A labelled arrow is used to represent information and/or data flow occurring between two processes or between an external entity and a process.
 - Rectangle: It is used to show an actor who communicates with the software system. The actor may give input (source) to the system or consume the output produced (sink). An actor may be a human being e.g. accountant, or any hardware, maybe a sensor that is feeding signal as input to a process.

Features of a DFD

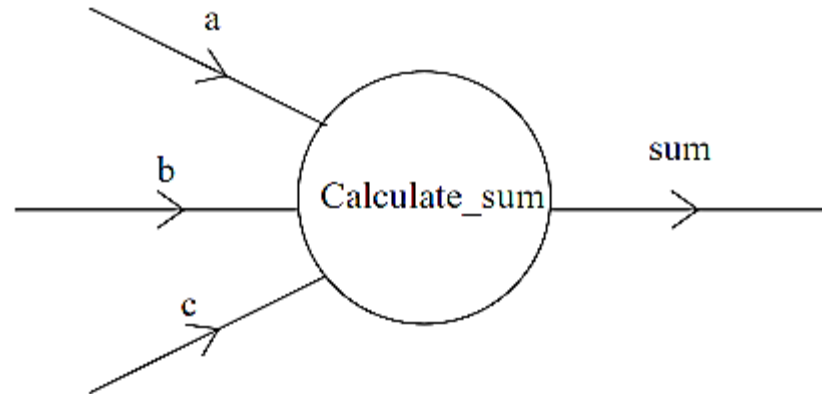
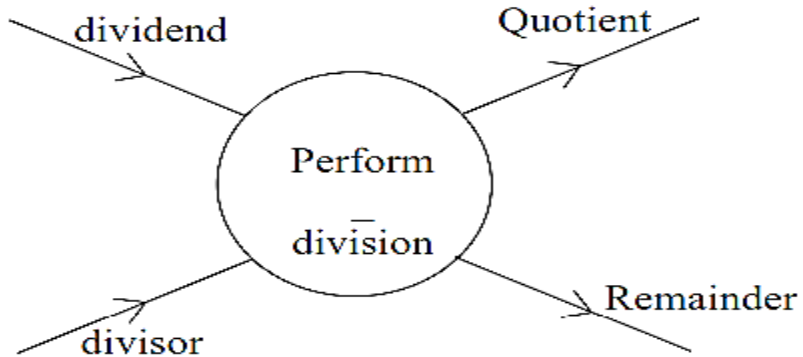
- Two parallel lines or rounded rectangle: It is used to represent a Data Store whose name is written between these two parallel lines. The data store is connected to a process that uses it.
- A rectangle with the right top corner chopped off represents the output in the form of a hard copy.



Features of a DFD

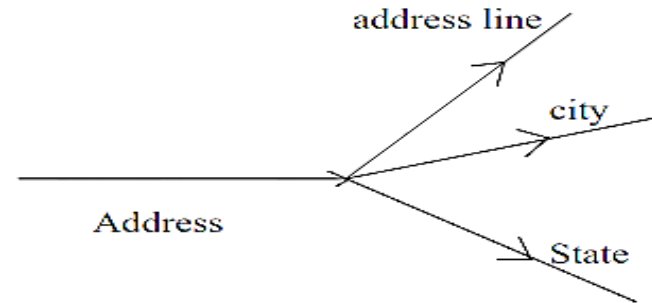
- Process :

- A circle represents a process and the name of function performed by it is written inside the circle. Process denotes a function that is to be performed on one or more inputs to produce one or more outputs. The purpose of the process is to transform data. Example :



Features of a DFD

- Data Flows :
 - It denotes the flow of data between any two processes or between a process and an actor, or between a process and a data store. It is represented by an arrow or a directed arc annotated with the name of the data item that it carries at that point in time. Data flow does not modify the content that it carries.

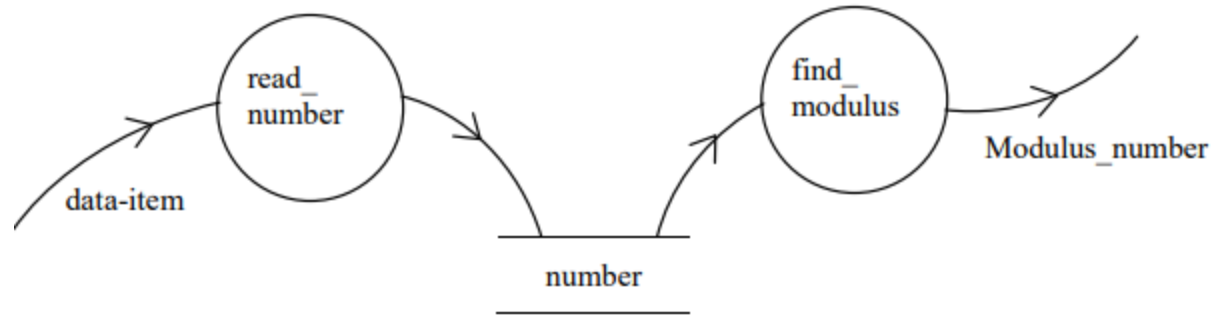


Aggregate value Address split in three components

Features of a DFD

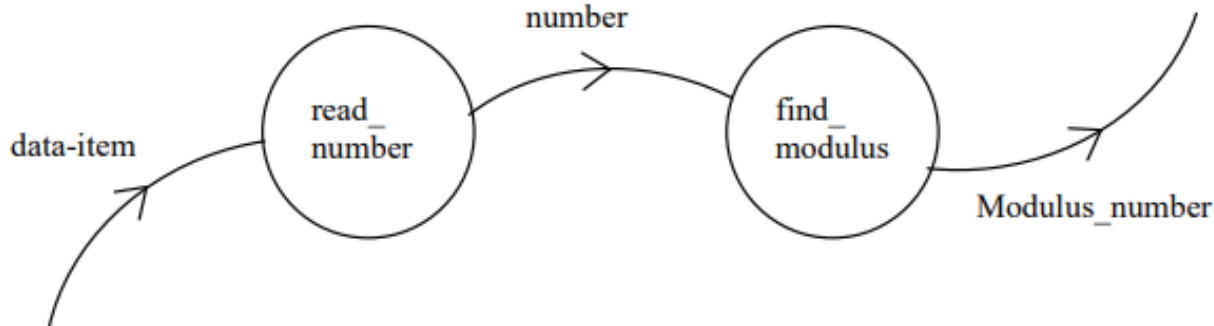
- Asynchronous Data Flow
 - As shown in Figure there sits a data source between these two processes. So, the first process produces the data stored in the data source. Later on, another process can use this as we can see that the two processes are independent of each other. This type of data flow is called asynchronous.

Asynchronous
data flow :



Features of a DFD

- Synchronous data flow :
 - In Synchronous data flow, the output produced by process “read_number” is immediately used by the process “find_modulus”, data flow arrow that directly connects both processes.

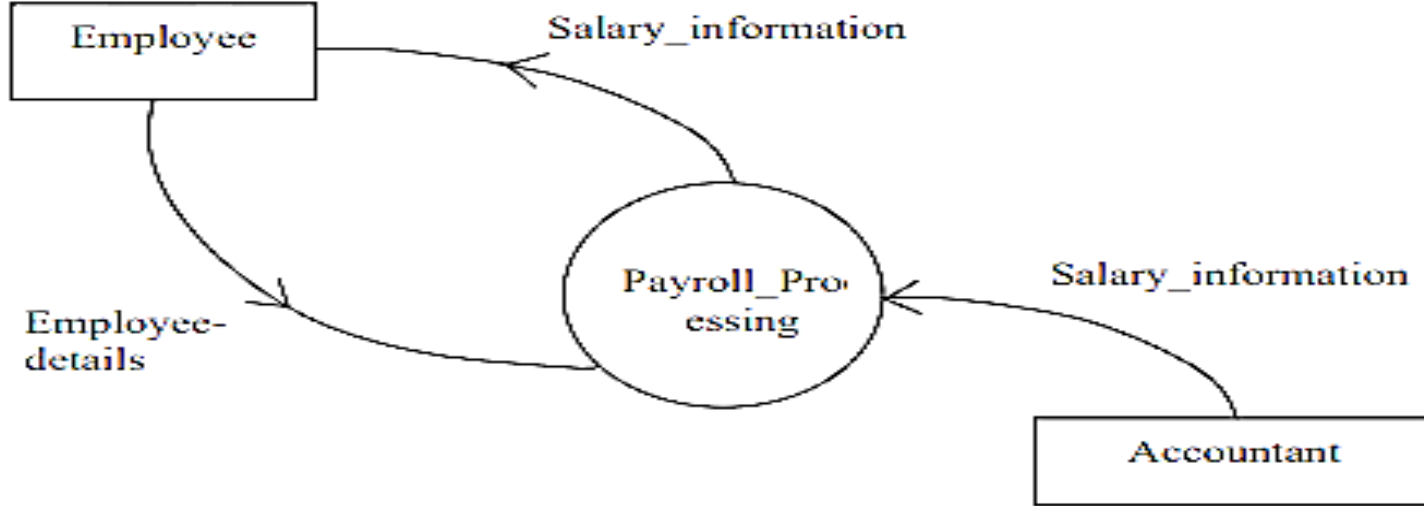


Synchronous data flow

Features of a DFD

- Actors :
 - An actor is represented using a rectangle.
 - An actor is an external entity that produces the data for the system or consumes the information produced by the system.
 - Actors lie at the boundary of the system.
 - They are also called terminators because the data flow is terminated at the input as a source or output as a sink of the system
 - Refer to below diagram from better understanding.

Features of a DFD

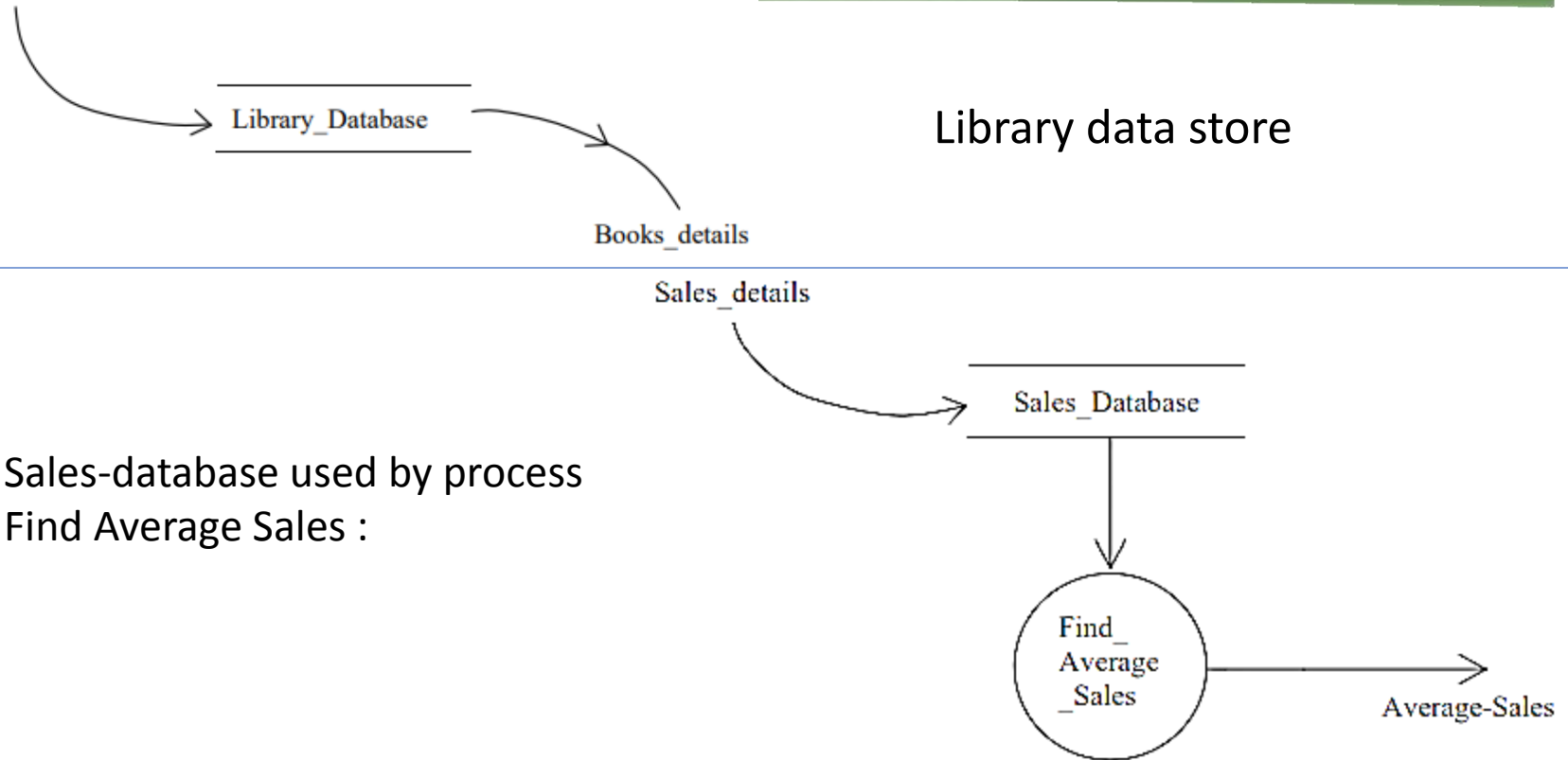


Actors interacting with a system

Features of a DFD

- Data Store :
 - A data store is a passive object that represents a repository of data in the software system and is represented using two parallel lines.
 - A data store symbol represents either a data structure or a physical file residing on the disk.
 - We need a data store when the system must store the data for future use by a process. Each data store is connected to one or more processes.
 - Data can be written/modified into the data store using the input arrow, or data may be retrieved from the data store represented using the output arrow.
 - For more clarification study below diagram.

Features of a DFD



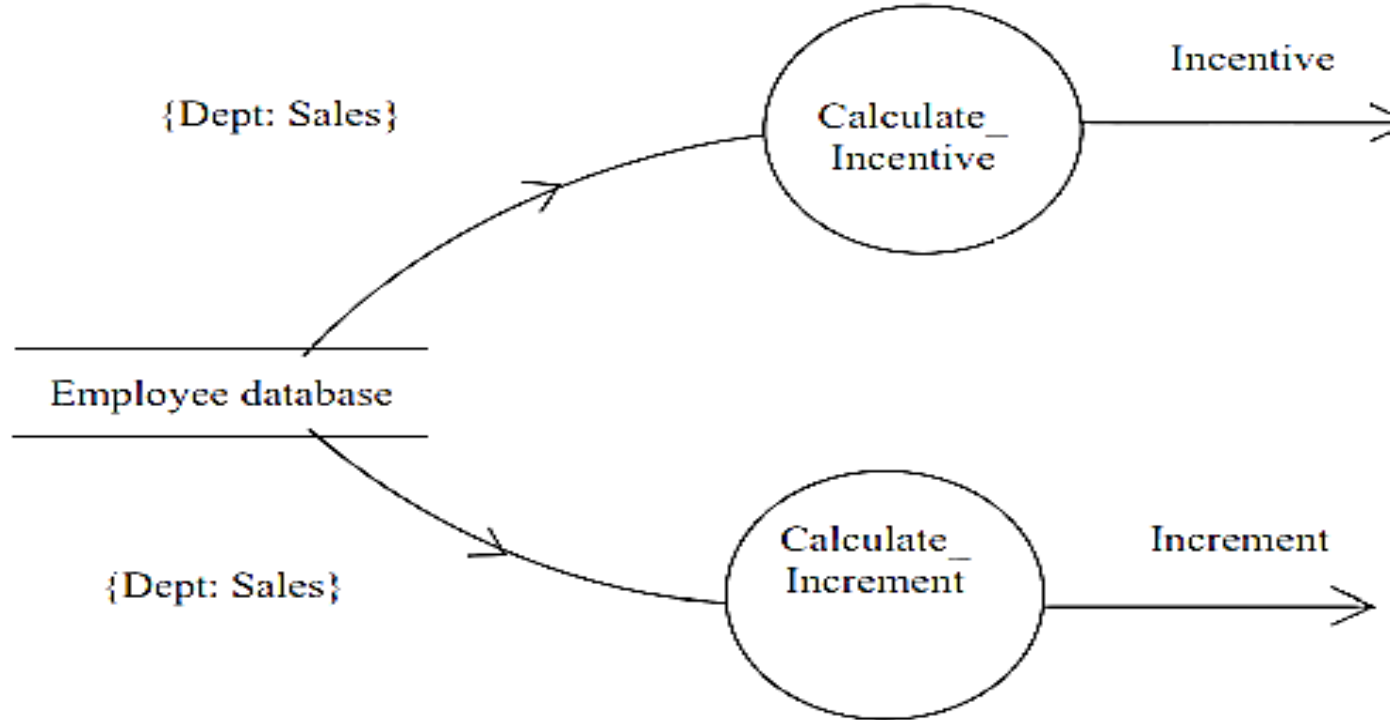
Features of a DFD

- Output Symbol :
 - Whenever a hardcopy is generated, we use the output symbol which is a rectangle with the top right corner chopped off.
- Constraints :
 - Constraint represents the relationship between two different objects at the same time or between different values of the same object at different times.
 - One can add new principles or modify the existing ones with the help of constraints.
 - All models of object-oriented analysis can have constraints in them, as explained below: -

Features of a DFD

- 1. Object Modelling: Here, constraints show the relationship between objects. In this type of modeling, constraints represent the relationship between the different values that an object may take at different times.
- 2. Dynamic Modelling: Here, the constraints define the relationship between the states and events of different objects.
- 3. Functional Modelling: In this, the constraints define the limitations on the transformations and computations that can be done on objects.
- Figure below shows a DFD in which, depending on the department of an employee, either he is given an incentive or an increment. As it is visible, if a department is Sales, then the incentive is calculated, and if the department is HR, then an increment is calculated.

Features of a DFD



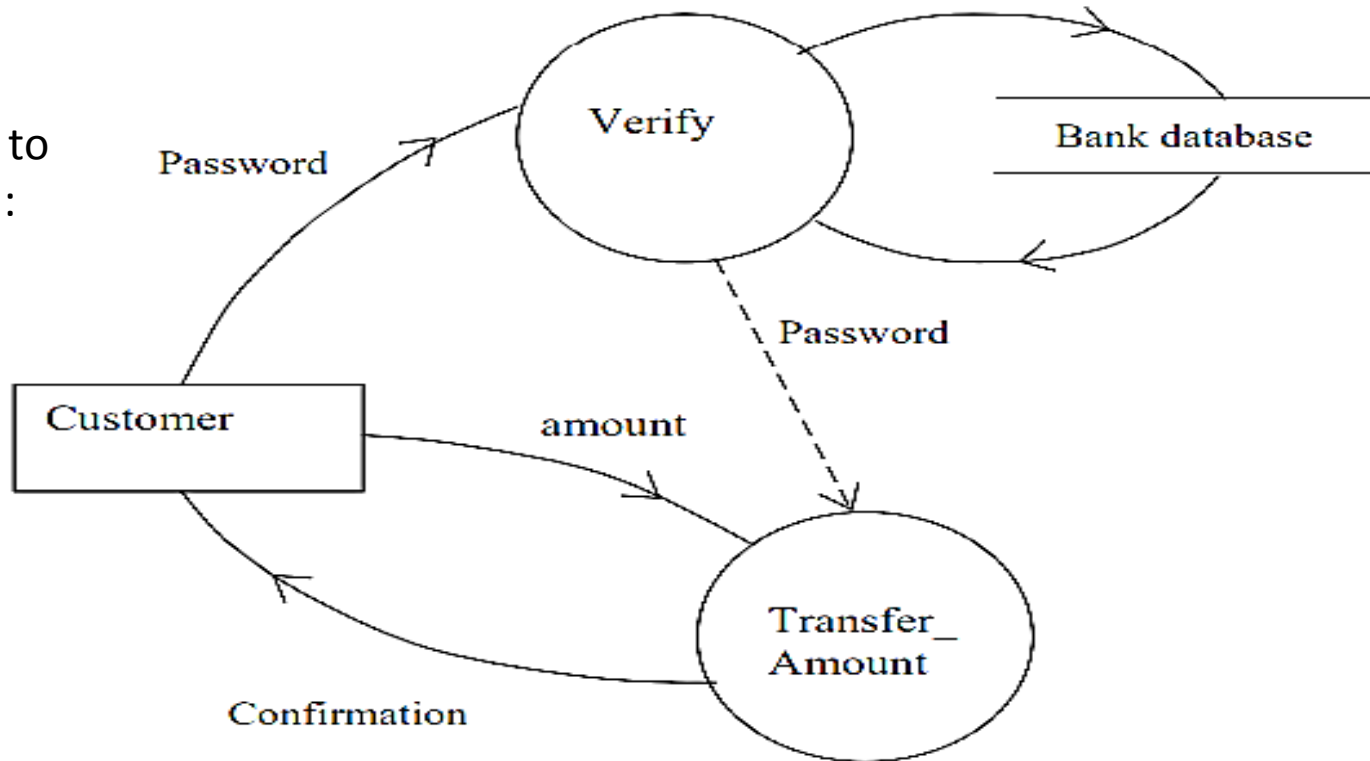
Features of a DFD

- Control Flows

- Control flow signal is generated by a process so as to inform other processes to start or stop its processing. This is typically a Boolean value passed to another process on the basis of which the other process decides its course of action. It is denoted by a dotted line.
- An example of control flow is shown in Figure given below. The customer enters the password in a net banking website, which is verified. Only after that, he can transfer money to somebody else's account.

Features of a DFD

Control How to
inform Transfer
amount process to
start processing :



Design flaws in DFD

- There are certain limitations of the DFD, which may lead to a few design flaws. The most disturbing limitation is that with DFD, we cannot visualize the complete picture of the software system and hence some of the essential physical entities are left out. There are different categories of design flaws:
 - Flaws committed by beginners:
 - The context diagram must have just one bubble that represents the complete software system at an abstract level and has arrows of input and output interactions corresponding to data flows with the world. Many designers draw more than one bubble in the context diagram, which must be avoided.

Design flaws in DFD

- External entities are drawn only in the context diagram. However, many designers represent the external entities at all levels of DFD, which is not correct.
- Each process/ bubble should be decomposed into between 3 to 7 bubbles, but sometimes designers do not follow this principle. Different levels of DFD must be balanced. Balancing of DFD means that the data flow into or out of a process symbol must match the flow at the next higher Level of the DFD.
- Representing control information in a DFD:
 - One must clearly understand that DFD is not a flow chart. DFDs cannot capture the following important features (When, Why and How): -

Design flaws in DFD

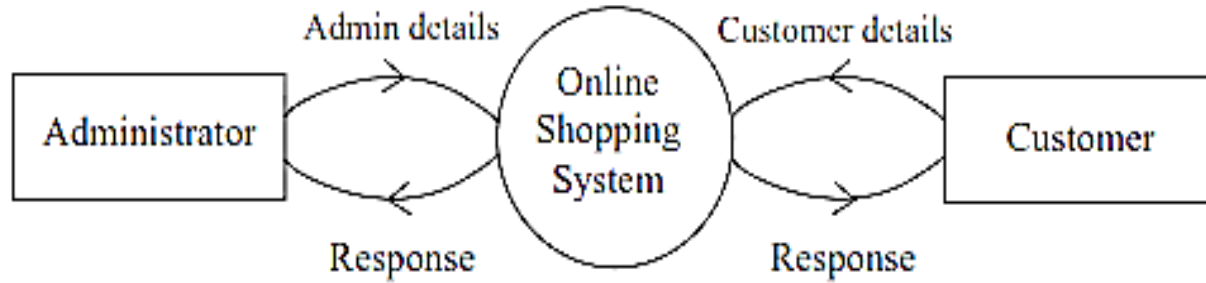
- When: As the Time-dependent behavior of a software system cannot be shown in a DFD, therefore the transformations are done by processes that cannot be shown in a DFD.
- Why: DFDs cannot specify why the value of an object has changed.
- How: DFDs cannot mention the frequency of transformations done on an object.
- If a bubble X invokes either the bubble Y or the bubble Z depending upon some conditions, we need only to represent the data that flows between bubbles X and Y or bubbles X and Z and not the conditions depending on which the two modules are invoked.

Design flaws in DFD

- Mistakes arising out of poor Diagramming approach: Black holes, miracles and grey holes. Such design flaws occur when the outputs from a process do not match its inputs.
- Illegal data flows with data store:
 - It is very important to realize that a store can only be connected to bubbles using data arrows. A data store can never be connected to another data store or to an external entity.

Sample functional model

- Level- 0 DFD –
 - Level- 0 DFD /Context diagram for Online Shopping System



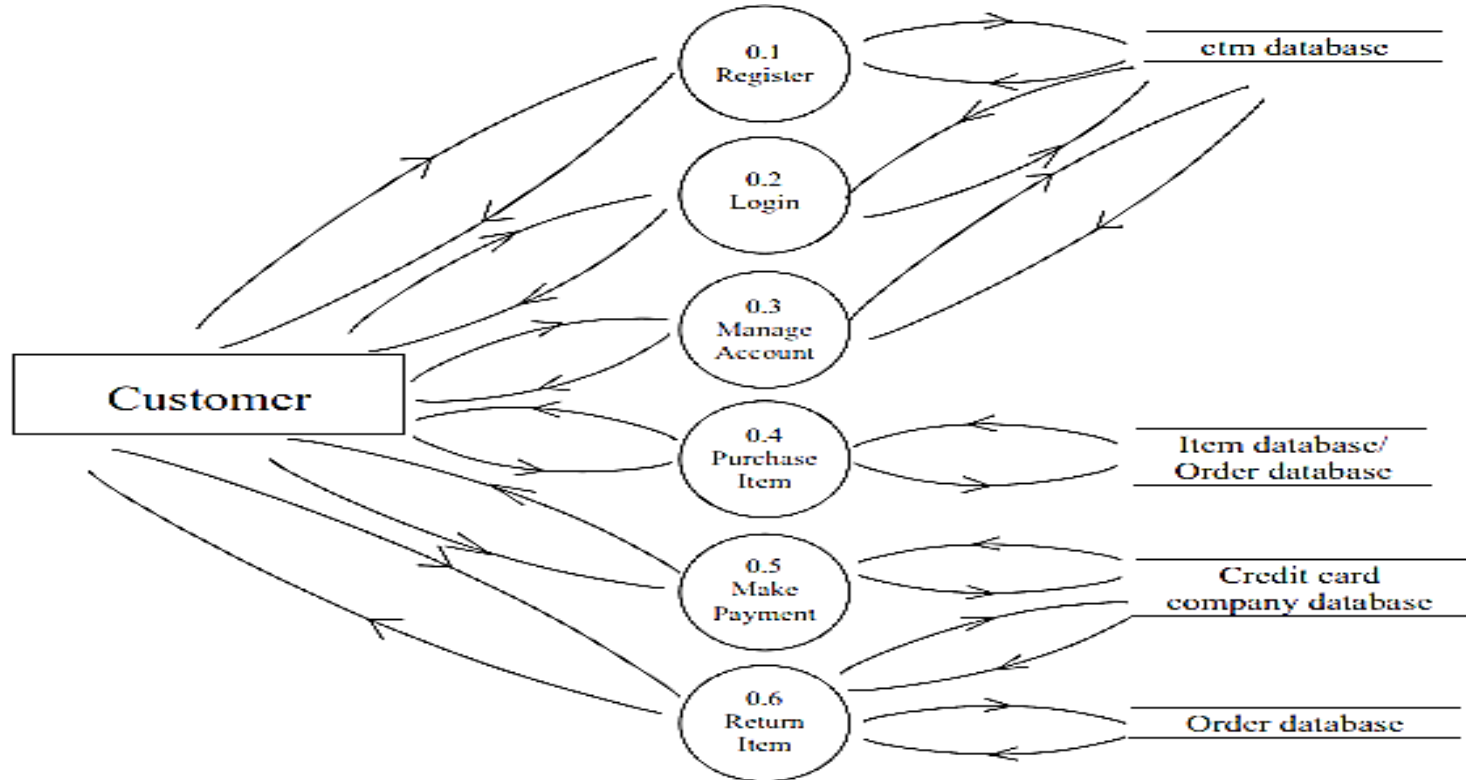
Sample functional model

- Level- 1 DFD :

- The context diagram is further decomposed in a hierarchical way to explain the system's entire working. This is done by converting the Level-0 bubble into three to seven bubbles which general.
- Since there are many functions pertaining to the customer side and administrator side, the Level- 1 DFD for both is shown separately.
- | | | |
|-------------------|------------------|-------------------|
| 1. Registration | 2.Customer Login | 3. Manage account |
| 4. Purchase items | 5. Make payment | 6. Return item |

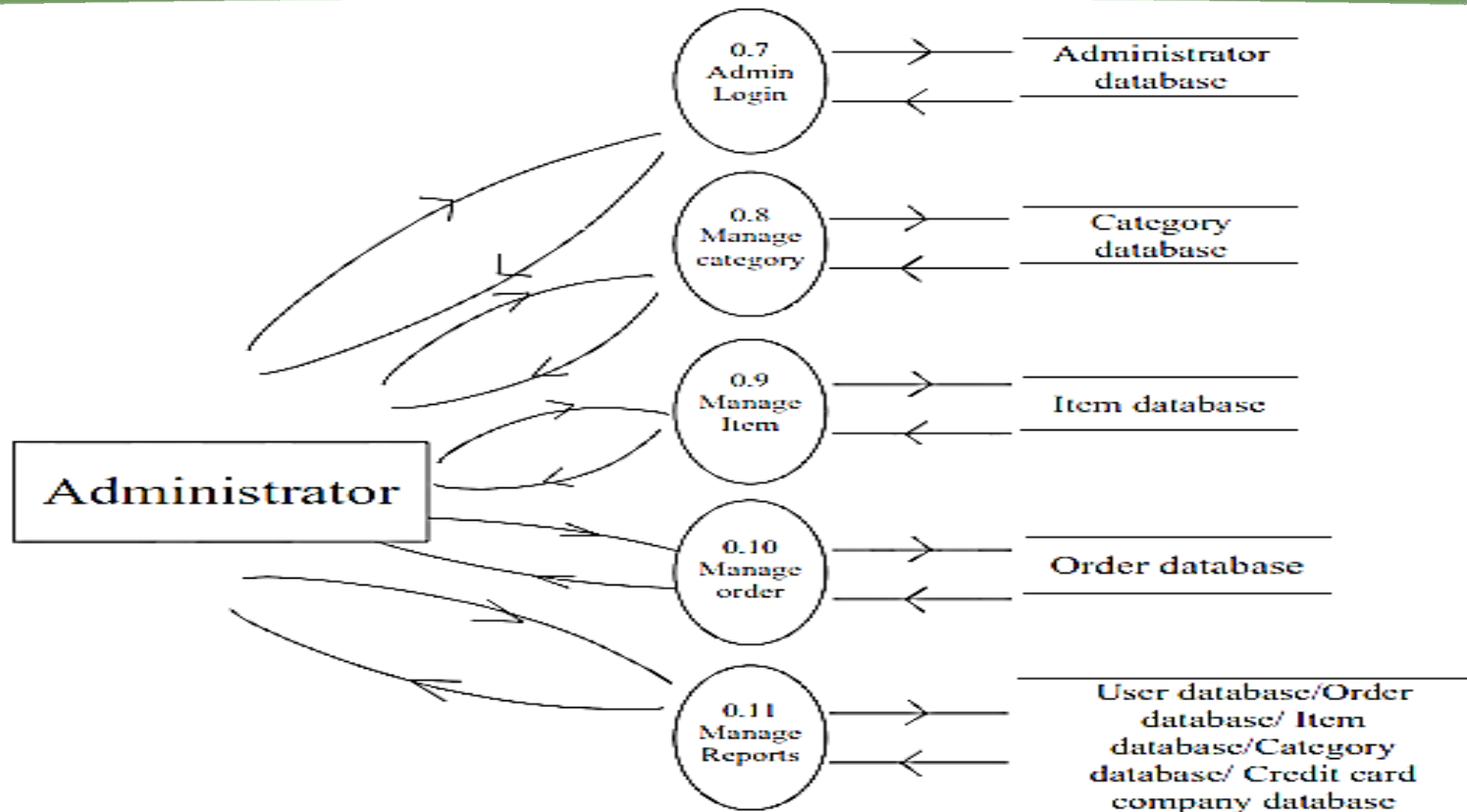
Sample functional model

Level 1
DFD for
costumer
side :



Sample functional model

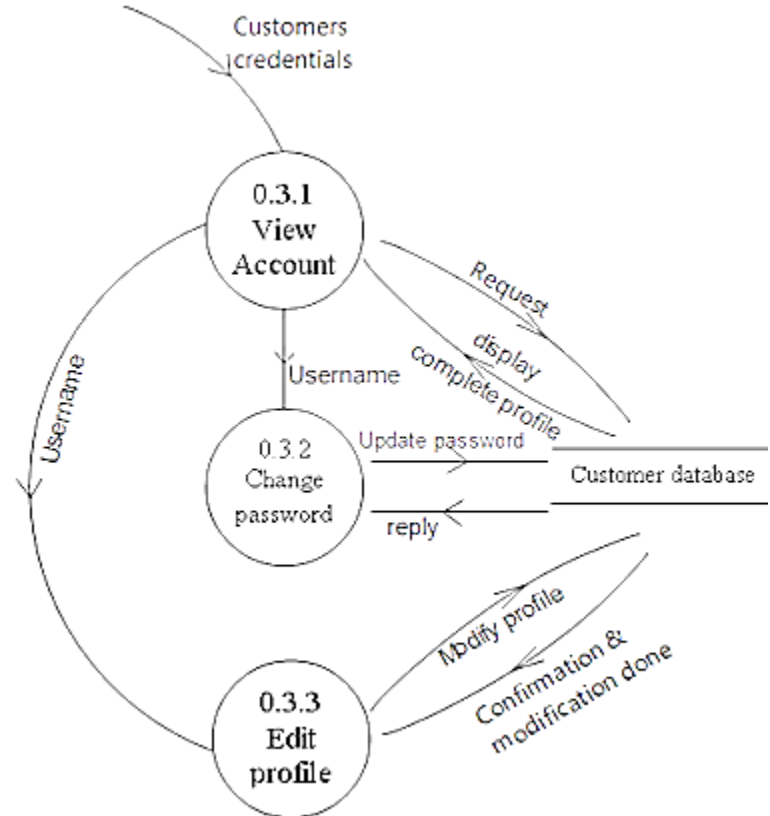
Level 1 DFD
for
administrat
or side :



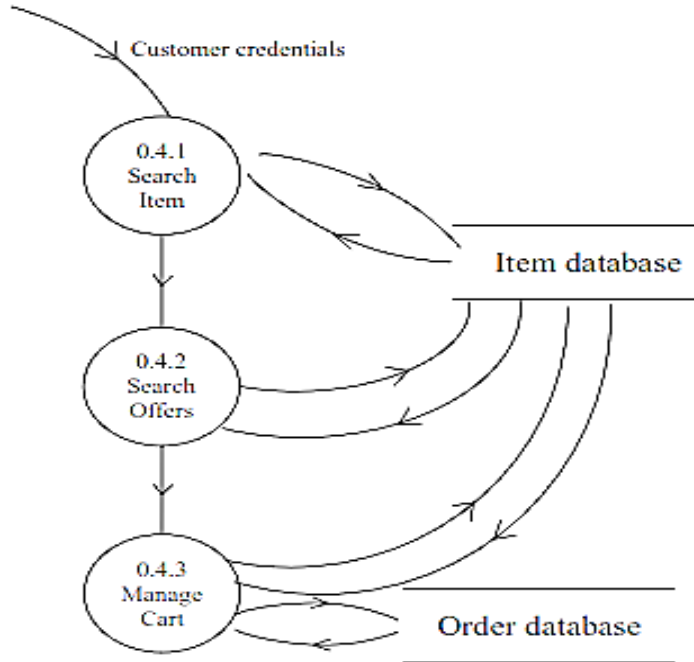
Sample functional model

- Level- 2 DFD :
 - Again, each of the bubbles of level-1 DFD maybe decomposed into three to seven more bubbles so that the functions can be explained at an atomic level. This expansion generates level 2 DFD.

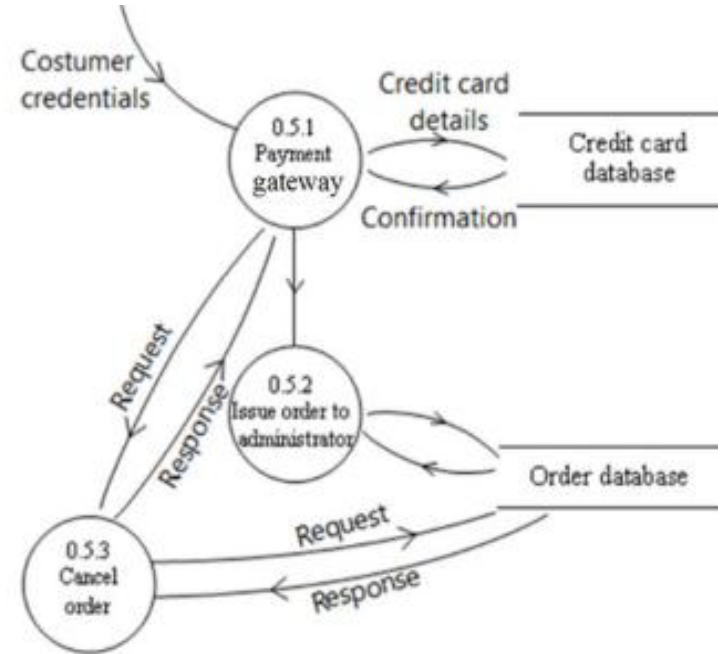
Level 2 DFD of “Manage Account” process of customer side :



Sample functional model



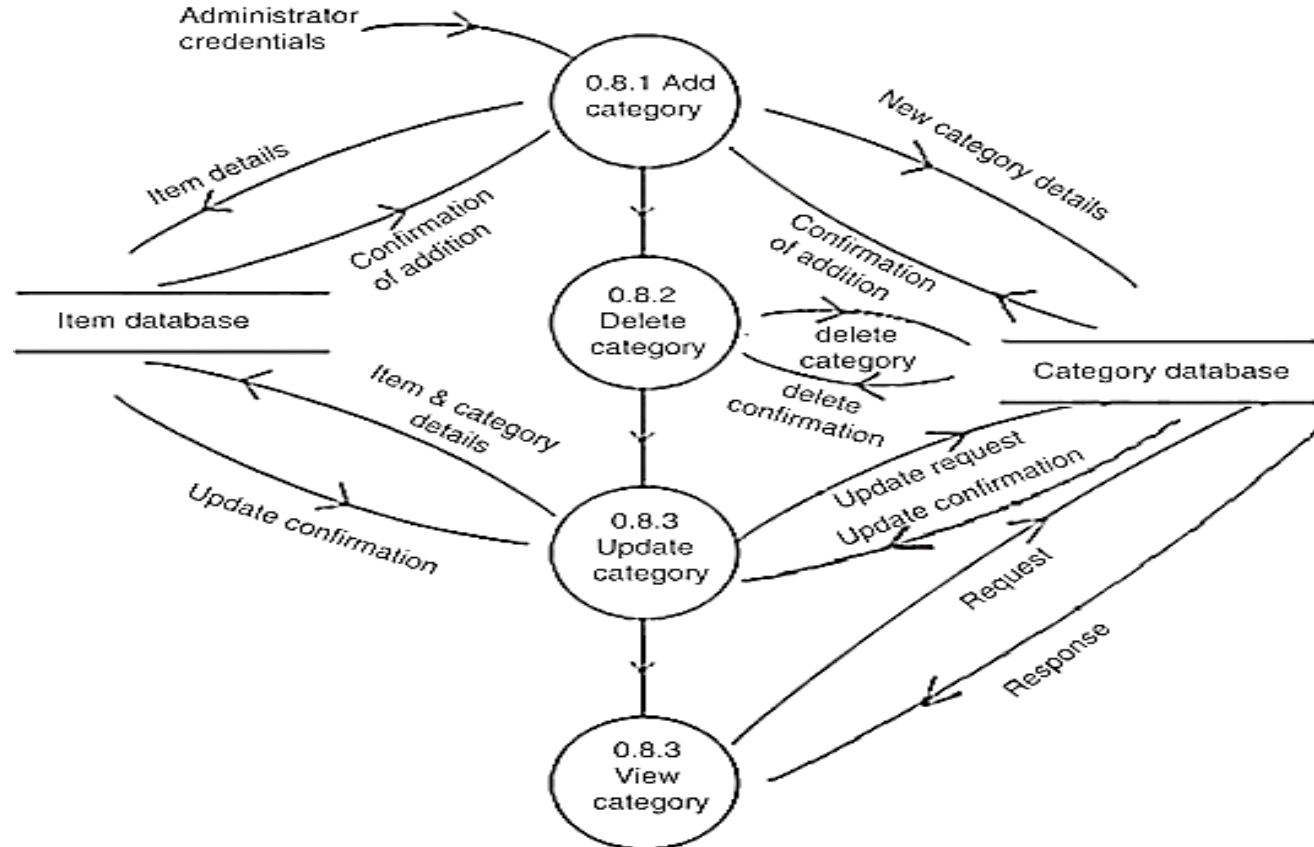
Level 2 DFD of “Purchase item” of customer side



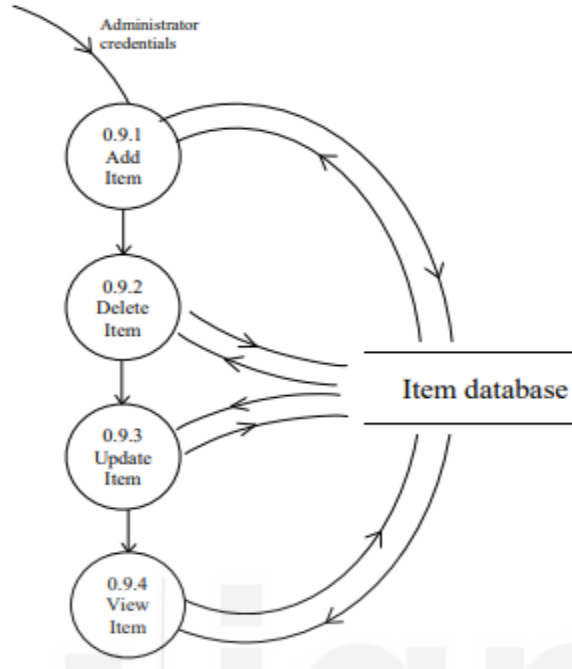
Level 2 DFD for “Make payment” (0.5) of customer side

Sample functional model

- Administrator side
- Level 2 DFD
- “Manage category”
of administrator
Side :

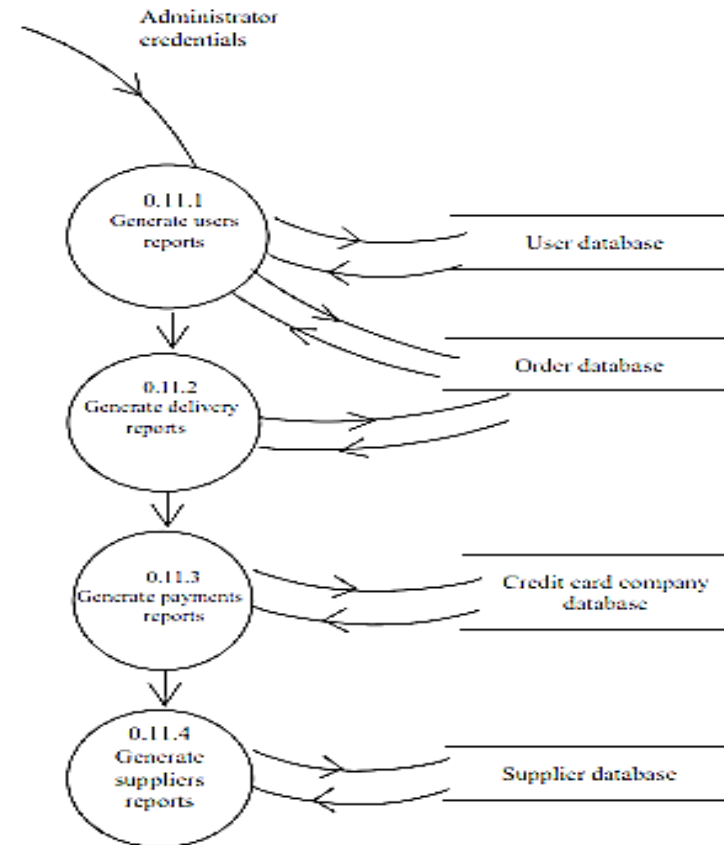


Sample functional model

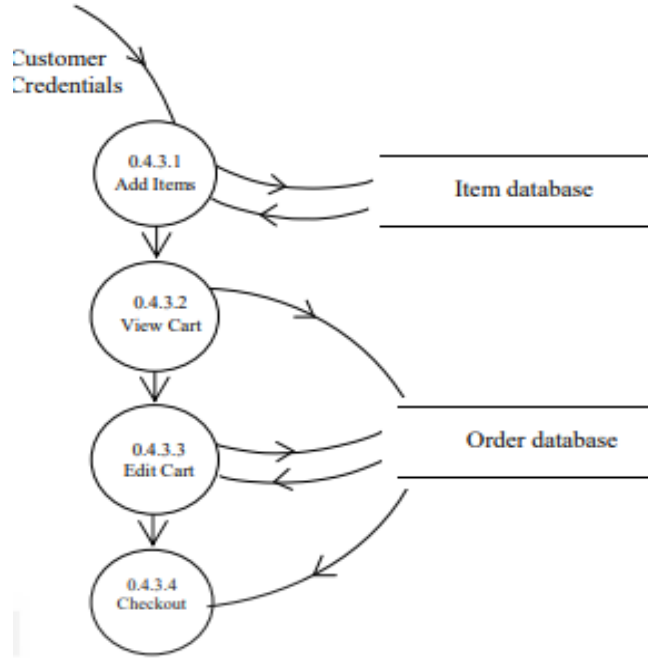


Level 2 DFD for "Manage item"
of administrator side

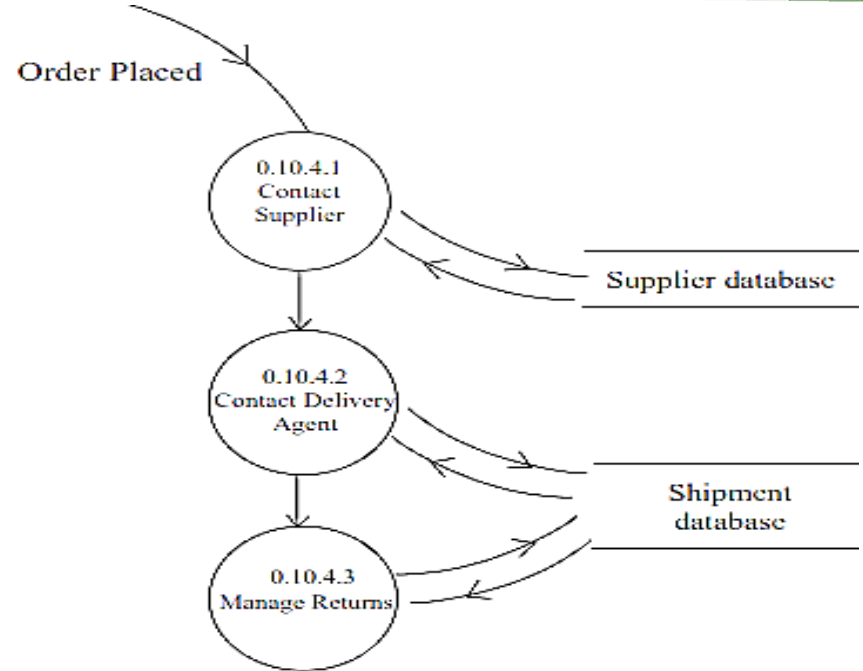
Level 2 DFD for
"Manage Reports" of
Administrator side:



Sample functional model



Customer side, for "Manage Cart"
Module of Level 2 DFD



Level 3 DFD after decomposing
"dispatch order" bubble of Level
2 of administrator side

Functional vs. Object vs. Dynamic model

- Object Model:
 - The purpose of this model is to show the static aspect of the software system, including depicting the attributes and operations that an entity can perform, i.e. Here, the focus is on how an object can be implemented and what operations an object is capable of performing.
 - Diagrams: In an Object model, we use object diagram and class diagram.
- Dynamic Model:
 - The purpose is to show the control aspect, as well as the temporal and behavioural nature of the software system. The sequencing of operations is shown in the dynamic model. The focus is on when the operations are done in response to an external stimuli.

Functional vs. Object vs. Dynamic model

- Diagrams: We model the states, transitions, events and actions with the help of interaction diagrams (like sequence and collaborative diagrams) and state transition diagrams.

Functional Model:

- This model is used to depict the functional aspects by showing various processes that include many sub-processes within them, along with various transformations applied to data as it moves in the system. This model focuses on what the operations do in terms of transformations on objects.
- Diagrams: We use data flow diagrams to show collaboration between different objects to achieve the system's desired behaviour.

Important Topics

- Functional Models
- Data Flow Diagrams
- Features of a DFD
- Design Flaws in DFD
- A Sample Functional Model
- Functional Vs. Object Vs. Dynamic Model

Summary

- In this session we have seen the design and working of the functional model in order to address the question “What” the software system is doing, and we have used Data Flow Diagrams to explain this.
- We have discussed the features of a DFD in terms of different symbols used along with their purpose. The symbols are used to denote process, data flow, actor, data store, constraints and control flows. Next, the design flaws that can occur while making a DFD have also been explained in detail in this unit.
- Further for the practical understanding of the topic, generation of DFDs up to Level-3 has been done for an Online Shopping System by decomposition of processes as we go from Level-0 to Level-3.
- Finally, an attempt has been made to throw some light on the relation between the functional model and the relation of the functional to the dynamic model.

*Thank
You*



Object Oriented Analysis and Design

Block-3 Unit-9

Basics of System Design

Topics to be Covered

- Introduction
- OOA To OOD
- System Design: At Object Oriented Approach
- Breaking Into Subsystems
- Concurrency Identification
- Management Of A Data Store
- Controlling Events Between Objects
- Handling Boundary Conditions
- Important Topics
- Summary

Introduction

- We will discuss the basics of object oriented design and the steps involved in designing.
- In this session, we will also learn how systems are broken into subsystems, concurrency identification, management of data store and boundary conditions.

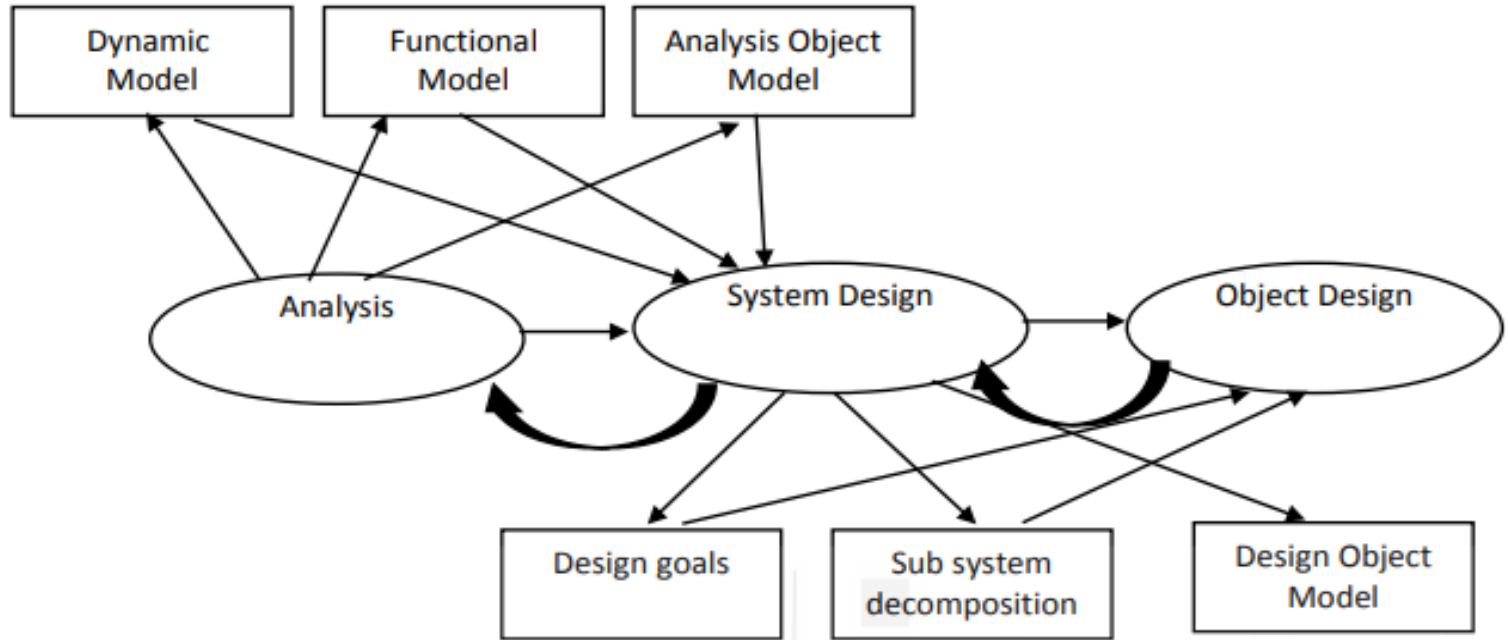
OOA to OOD

- Object-oriented analysis (OOA) is used to apply object-modeling techniques to analyze the functional requirements of a system.
- Object-oriented design (OOD) explains and expands the analysis models to create implementation specifications.
- OOA point on what the system does and OOD focuses on how the system does it.

System Design: an Object Oriented Approach

- The aim is to capture the structure and behavior of information systems into small modules that combines both data and process.
- There are primarily three tasks performed during the system design phase
 - Laying down the Design Goals
 - Decomposition of the System
 - Decomposition of the System

System Design: an Object Oriented Approach



The relationship of system design with other software engineering activities.

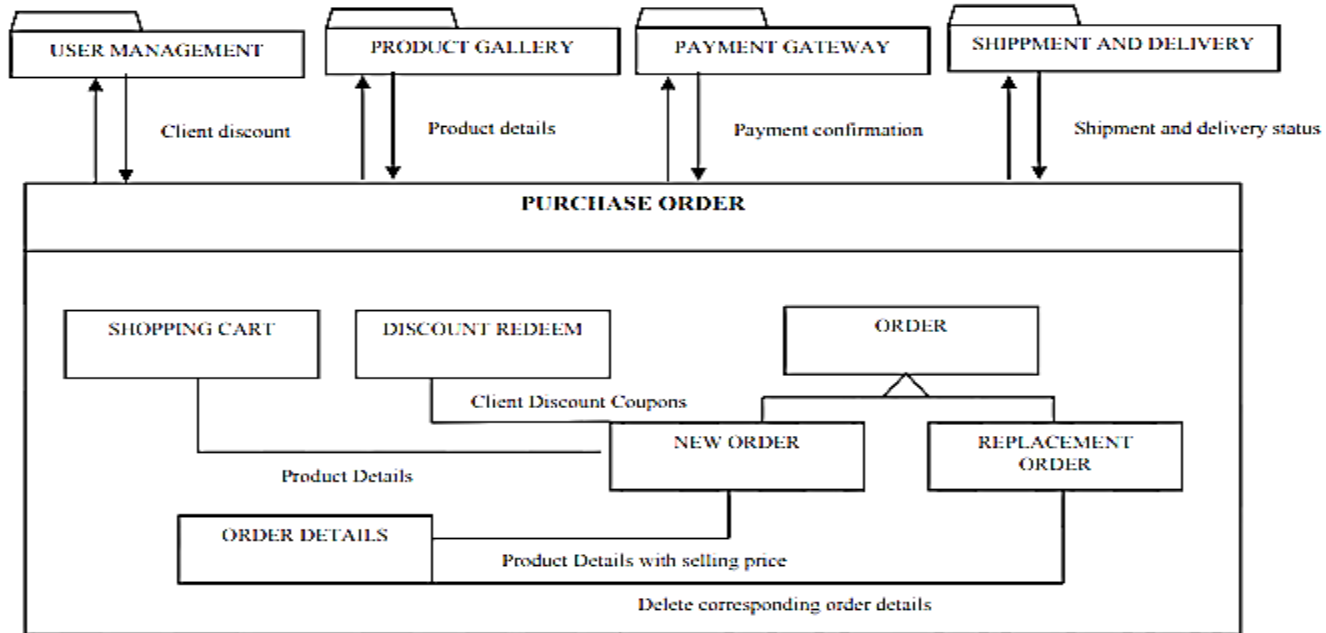
Breaking into Subsystems

- Subsystems and Classes
 - The first step is to simplify the system's complexity by dividing it into smaller subsystems.
 - A subsystem may be defined as a single class or a group of classes that share a common objective and performs multiple services as a single cohesive unit.
 - System decomposition is an iterative process, and each design goal contributes to more refinement of a subsystem.
 - A large system is usually decomposed into smaller subsystems using a vertical or horizontal partition

Breaking into Subsystems

- Coupling and Cohesion
 - Cohesion shows the independence of a subsystem, and coupling defines the dependencies between subsystems.
 - Cohesion reflects the interdependencies within a subsystem.
 - While decomposing subsystems, there is always a trade-off between cohesion and coupling.

Breaking into Subsystems

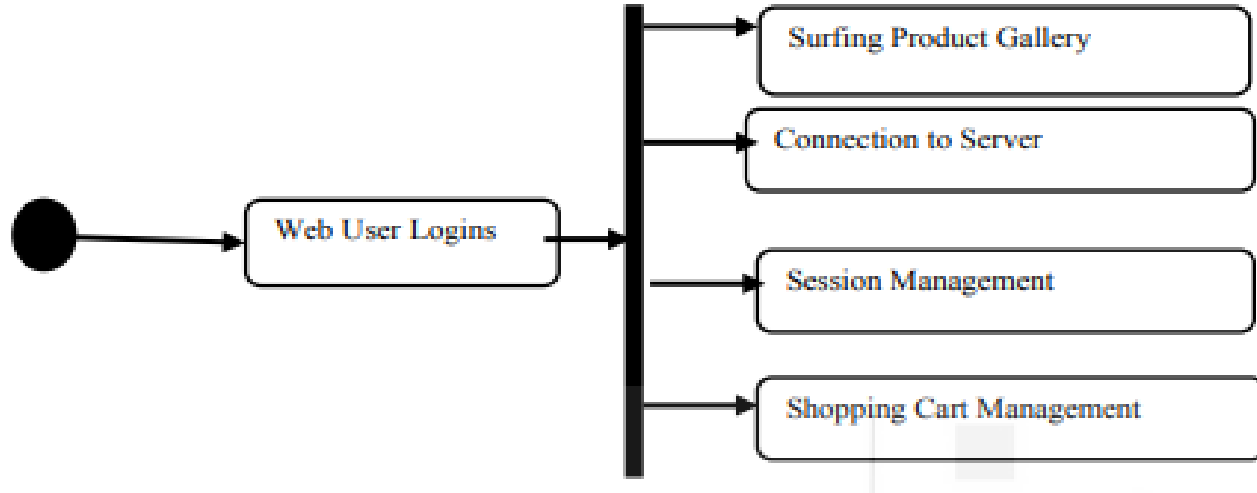


Example of PURCHASE ORDER subsystem of Online Shopping System showing cohesion and coupling

Concurrency Identification

- Concurrency allows two or more objects to be executed at the same time. It allows multiple tasks to be executed, interacted, and collaborated simultaneously to achieve global functionality of the system.
- synchronization algorithms help to avoid situations like
 - Deadlock : is a condition when two or more threads/processes are holding some resource and waiting for some other resource to be released by another thread or process
 - Starvation : occurs when a thread/process is made to wait for the resource till some other thread/process is using it. But instead of allocating the resource to the waiting thread, the resource is allocated to some other high priority thread/process, resulting in prolonged waiting, leading to starvation.

Concurrency Identification



Concurrent task in Online Shopping System

Management of a Data Store

- Persistent Objects : The object which must exist even after the system shutdown is called a persistent object.
- The Data Storage Strategy : Once the persistent objects have been identified, the system designer must decide the storage strategy to be used.
- Relational Database:
 - It provides a higher level of data abstraction.
 - Data is stored in the form of tuples in a relation. The data can be retrieved from multiple tables using complex queries

Management of a Data Store

- Object-Oriented Database: These databases provide similar services as relational databases. They follow the properties of OOPS viz., inheritance and abstract data type. It's advantages are –
 - It stores the data in the form of objects and classes.
 - It allows complex data set to be stored and retrieved much faster.
 - It allows the real world data to be mapped directly into the object oriented database.
 - These databases are capable of storing different types of data viz., pictures, voice, video, text, numbers and so on.

Management of a Data Store

Comparison of OODBMS and RDBMS

Features	OODBMS	RDBMS/DBMS
<i>Definition</i>	An object-oriented database management system is a DBMS where data is represented in the form of objects and classes.	Relational database management systems (RDBMSs), stores data in the form of tables/relations having rows and columns.
<i>Data Type</i>	Object Oriented Database supports complex data, for example, objects of class type.	RDBMS supports basic data types i.e. integer, date, string, Boolean etc.

Management of a Data Store

<i>Data</i>	i) In OODBMS, data are modelled as objects. An object has a finite set of attributes and behavioural properties. ii) Each object has a unique object identifier which is independent	i) In RDBMS, data is modelled as relations. A relation has a finite set of columns or attributes. ii) Each tuple or record in a relation has a unique
	of the values of its attributes. iii) All objects with the same set of attributes and methods are grouped into a class.	identifier called the primary key. iii) Each tuple represents an entity, and a group of similar entities forms an entity set.

Management of a Data Store

<i>Queries</i>	OODBMS supports complex queries, query optimization, backup and recovery.	RDBMS supports very high-level queries, query optimization, transactions, backup and crash recovery.
<i>Object Oriented Features</i>	i) OODBMS supports inheritance, which allows a class to inherit the attributes and methods of a superclass and is represented in the form of a tree. ii) OODBMS also supports multiple inheritance, where a class can have more than one direct superclass and is represented in the form of a lattice.	i) RDBMS does not allow the concept of inheritance. ii) RDBMS supports the concept of cardinality in the relationships between entities of the database system.

Controlling Events between Objects

- The order of execution of a program is called control flow.
- During analysis, the flow of a program is not decided, as all the objects/classes are assumed to be executing simultaneously at any time.
- In the design phase, the system designer must formulate the overall control flow of the system as well as control flow within classes and subsystems.

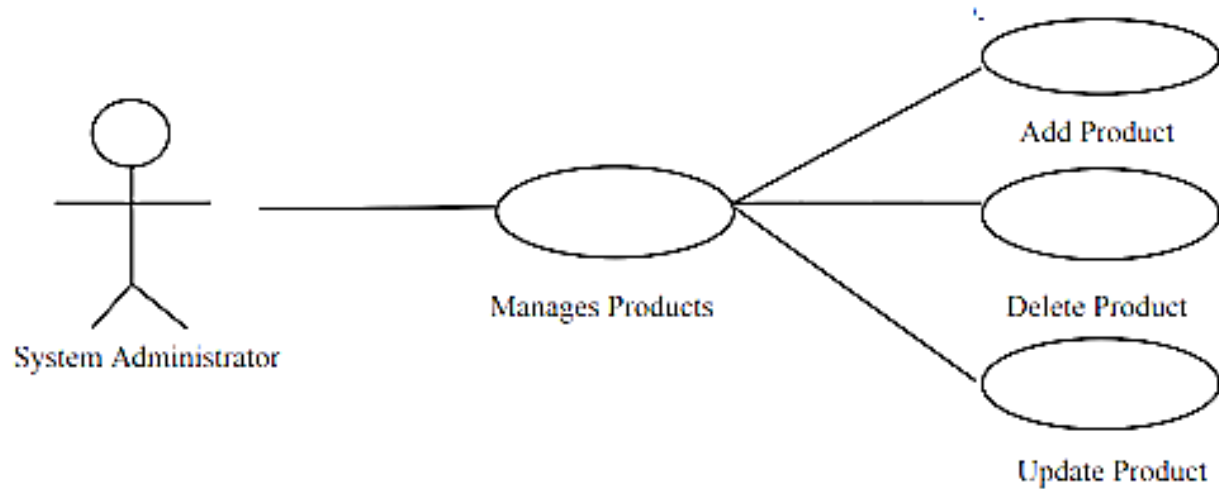
Handling Boundary Conditions

- We need to figure out how the system will behave if failure occurs due to error, network failure, database failure, system crash, etc.
- The use case defined to explain these conditions are called boundary conditions use cases.
- Boundary conditions are the necessary constraints used to find solution of a boundary value problem : -

Handling Boundary Conditions

- Persistent Objects/Classes - The system designer must examine the use cases where persistent objects are created and destroyed.
- Start and Shutdown – The system designer must add three use cases to start, shutdown and configure each subsystem.
- Exception Handling – The system designer decides how the system will behave in case of failure of any subsystem.

Handling Boundary Conditions



System Administrator use case diagram to manage products in Online Shopping System

Handling Boundary Conditions

- Start and Shutdown – The system designer must add three use cases to start, shutdown and configure each subsystem.
- Exception Handling – The system designer decides how the system will behave in case of failure of any subsystem. The designer needs to document the handling of each failure condition in the system.
- For example, in Online Shopping System, the order processing must not start if the user has not selected any product in the shopping cart and triggers the Order processing. In that case, the system must raise an exception and show the relevant message to the user rather than shutting down.

Important Topics

- OOA To OOD
- System Design:
- Breaking Into Subsystems
- Subsystems And Classes
- Coupling And Cohesion
- Concurrency Identification
- Management Of A Data Store
- Controlling Events Between Objects
- Handling Boundary Conditions

Summary

- The boundary conditions are analyzed looking from the end user perspective for effective start and shut down of the system.
- Concurrency Identification

*Thank
You*



Object Oriented Analysis and Design

Block-3_Unit-10

Object Design

Topics to be Covered

- Introduction
- Object Design for Processing
- Object Design Steps
- Create Links between High-Level Requirements to Low-Level Services
- Realization of Use Cases Using Operations
- Designing Algorithms
- Design Optimisation
- Implementation of Control
- Adjustment of Inheritance
- Design of Associations
- Important Topics
- Summary

Introduction

- The benefit of using object-oriented methodology is that it applies equally well in the domains of real-world mapping specifications and computer-based implementations. When the class design seems to be ready at one level of abstraction, it means now we need to iterate the process for lower levels.
- We will cover the aspects of object design on realizing use cases, designing algorithms, design optimisation and adjustment of the inheritance hierarchy.

Object Design for Processing

- The design model is driven by the requirement to produce a finished good, and outcomes are dependent on the experience of the team, reuse opportunities and personal preferences of the designer.
- The designer studies both the requirements and analysis artifacts to produce an effective solution.

Object Design Steps

- For designing objects, the following steps are involved:
 - Creating links between high-level requirements to low-level services.
 - Realization of use-cases using operations.
 - Formulating an algorithm
 - Designing operations that assist high-level methods.
 - Refactoring for a clean design.
 - Optimisation of data access paths.
 - Realize behavior that must be deployed.
 - Adjustment of classes to specify inheritance.
 - Organize classes with their associations.

Create links between High-level requirements to Low-level services

- Creating a bridge between the high-level requirements specified in the problem domain and the actual available resources.
- The high-level requirements or features are the realization of use-cases, implementing services and operations of the application and its corresponding commands.
- On the other hand, the available resources are operating system, class libraries and the previously used application.

Realisation of use cases using operations

- We will understand realization of Use case with the help of Example: Withdrawing money from ATM, This would involve several responsibilities such as :
 - Reading the debit card.
 - Asking amount from the customer.
 - Check the account balance.
 - Verify the transaction limit.
 - Verify the sufficiency of cash in the ATM.
 - Release money.
 - Debit bank account.
 - Print the receipt.
 - Release the card.

Designing Algorithms

- Steps for designing an algorithm should cater to the following conditions given below:
 - Choosing Algorithms : Algorithm with smallest time complexity should be used, so that minimum time is used while execution.
 - Selecting Data Structure : Algorithms work on data structures. In the design phase, we need to compose data structures in a manner such that the algorithms can run efficiently.
 - Defining Internal Classes and Operations : clearly indicates that mapping of attributes and relationships of analysis classes into their design counterparts would result in the extensive creation of messages

Designing Algorithms

- Assigning Operations to classes : It is easy to assign operations to a class when it is a reflection of real-world scenarios. During design, new internal classes emerge that do not correspond with the aspects of the real-world scenario.
- Receiver of action: Operations can best be associated with the target class rather than the class which initiated that operation.
- Focal class: It is a class in the class diagram which forms a center or a star to other classes and their associations. The center class is the target for the operation.

Design Optimization

- The designer tries to optimize performing tasks given below:
 - Look for access paths that are efficient
 - Rearranging computing tasks for greater efficiency
 - Saving intermediate results for avoiding recomputation.

Account
+CurrentAmount
/balance

AmountWithdrawn

{balance = CurrentAmount – AmountWithdrawn}

Derived Attribute is shown with a '/' in front.

Implementation of Control

- The consistency of state diagrams is validated against classes and sequence diagrams while building the application. To implement an applications state model, the following steps are performed:
 - Find the respective classes for states.
 - Find the triggering events.
 - Merge state diagrams into code.
 - Examining against other state diagrams.
 - Examining against other class diagrams.
 - Examining against sequence diagrams.

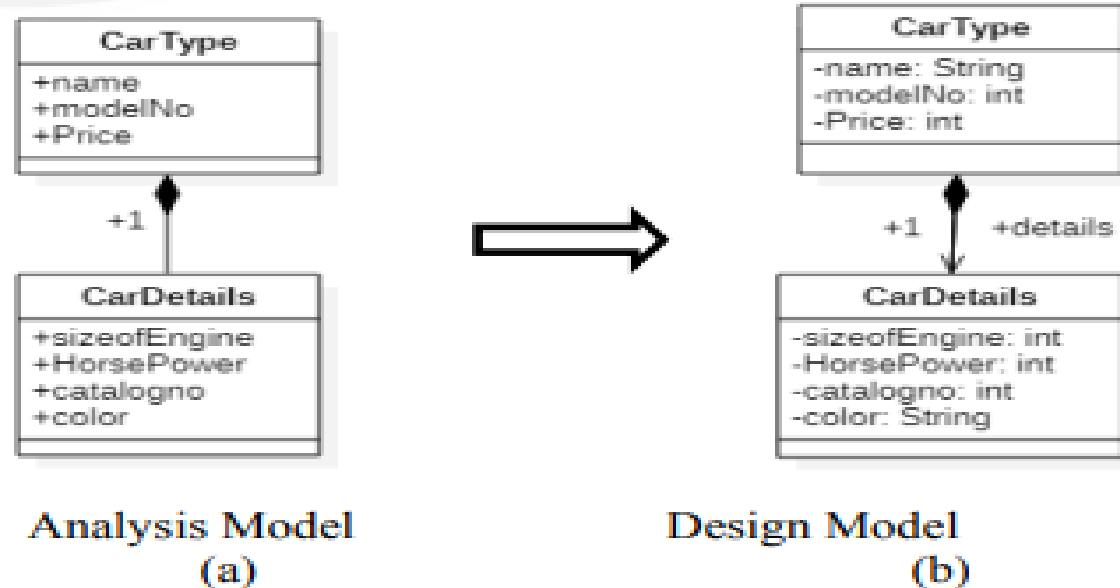
Adjustment of Inheritance

- It would be convenient to create inheritance hierarchies between classes as we proceed further with the design. The following steps guide understanding the adjustment of classes and operations.
 - Rearrangement of classes and operations – When we examine classes during design, we find that several classes define the same operations. In such a case, creating an inheritance hierarchy using common ancestors is common practice
 - Abstraction of Common Behavior - While adding new classes and operations during design, the opportunity of adding inheritance hierarchy can also be explored. An inheritance hierarchy is easily recognisable amongst two classes with repetitive operations and attributes. They could possibly be specializations of the same thing giving a higher level of abstraction.

Design of Associations

- The diagrammatic representation of classes in UML is differentiated from its instances using an underline below their names. While progressing towards design, there are three major issues related to associations that must be determined.
 - Identifying classes to be implemented
 - Identifying types of attributes
 - Mapping composition relationships

Design of Associations



Representation of attributes and compositions from analysis model into the design model

Important Topics

- Design for Processing
- Object Design Steps
- Create Links between High-Level Requirements to Low-Level Services
- Realisation of Use Cases Using Operations
- Designing Algorithms
- Implementation of Control
- Adjustment of Inheritance

Summary

- The first step involved enhancing the classes in the design phase by adding operations identified from use cases. New operations might also be added to deliver the new behaviours discovered.
- Defining abstract super classes for retaining the common behaviours and subclasses to assimilate the specialized attributes and operations.
- .As class design progresses, classes have the same common features found, and these classes now need to be adjusted and modified to create inheritance relationships between them.

*Thank
You*



Object Oriented Analysis and Design

Block-3 Unit-11

Advance Object Design

Topics to be Covered

- Introduction
- Control and its Implementation
- Inheritance Adjustment
- Association: Design
- Object Representation
- Design Optimization
- Design Documentation
- Important Topics
- Summary

Introduction

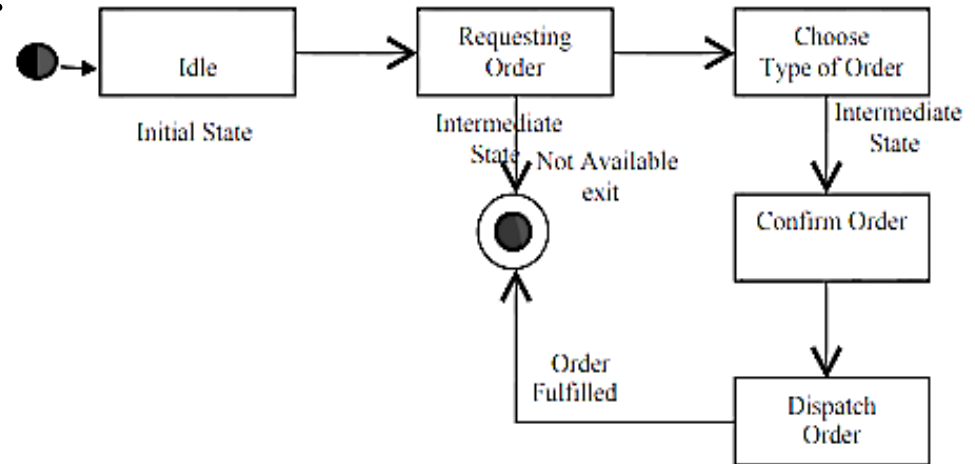
- An object is an important element of Object Oriented Technique and can be represented as:
 - a physical entity such as car, truck etc.,
 - a conceptual entity like any chemical reaction or process or the software entity, for example, linked list, stacks etc.
- In this session, we will learn concepts and diagrams used in object design.

Control and its Implementation

- Control means how the program change state from one to another after giving the input.
- State Machine- Object behavior can be modelled by using static machine modeling. Any event that affects the object state, causing some occurrence of actions. State Transition Diagram(STD) is used to represent it graphically.
- State - A curved edge rectangle is used to represent states in STD. During its lifecycle object goes through different states based on its attributes and function.

Control and its Implementation

- Transition- The transition from one state to another is shown by an arrow from an old state to a new state. When an object satisfies certain applied conditions and after an event which performs various activities results in a new state.



Order Management Process represented as State Transition Diagram :

Control and its Implementation

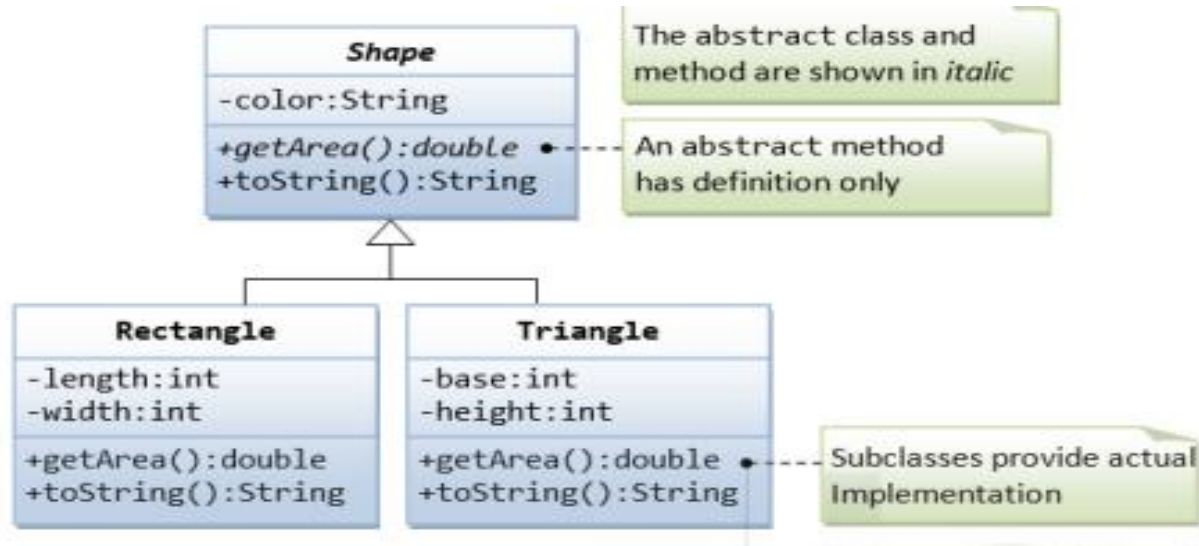
- The various parts of transition are:
 - Source State: It is the initial state of the object before any transition.
 - Events: It changes the state of an object from the source to the intermediate state after it satisfies certain conditions. Examples include - mouse click
 - Condition: If the required condition is satisfied, then it returns true and otherwise false; and no state change in state.
 - Action: It is an outcome because of transition. A series of actions represent a certain scenario.
 - Final State: It occurs as a result of the last transition and action.

Inheritance Adjustment

- Inheritance is the way to create new classes from the existing ones.
- To increase inheritance, the following steps should be performed:
 - Reorganize classes and their associated behaviour to increase the inheritance-
 - Some operations may have optional arguments.
 - Operations contain fewer arguments and are special cases of general arguments.
 - Some operations may have optional arguments. – Operations contain fewer arguments and are special cases of general arguments.

Inheritance Adjustment

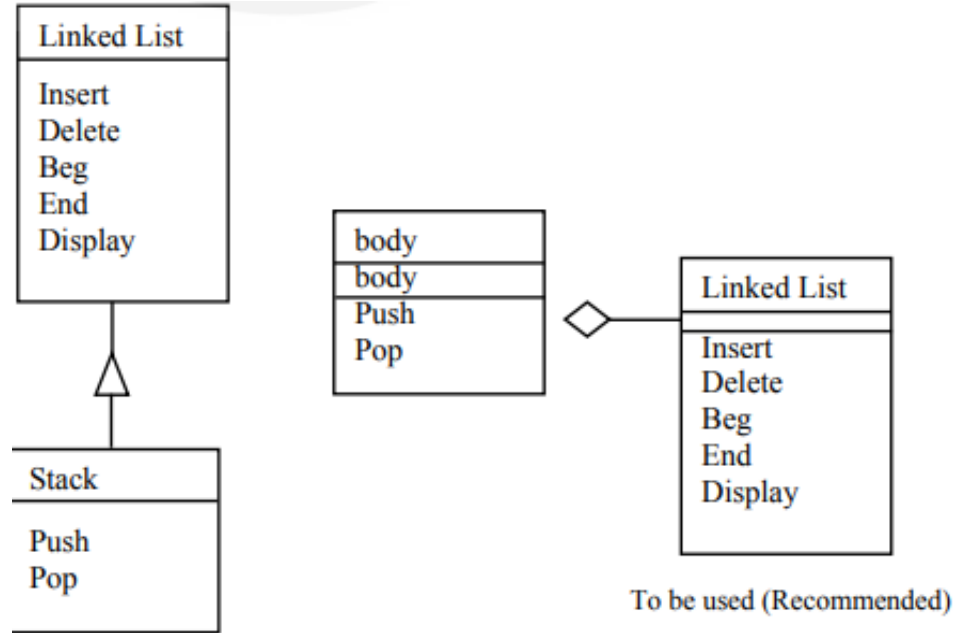
- Abstract the common behaviour in classes



Abstract class and subclasses

Inheritance Adjustment

- Use delegation to share behaviour when inheritance is semantically invalid

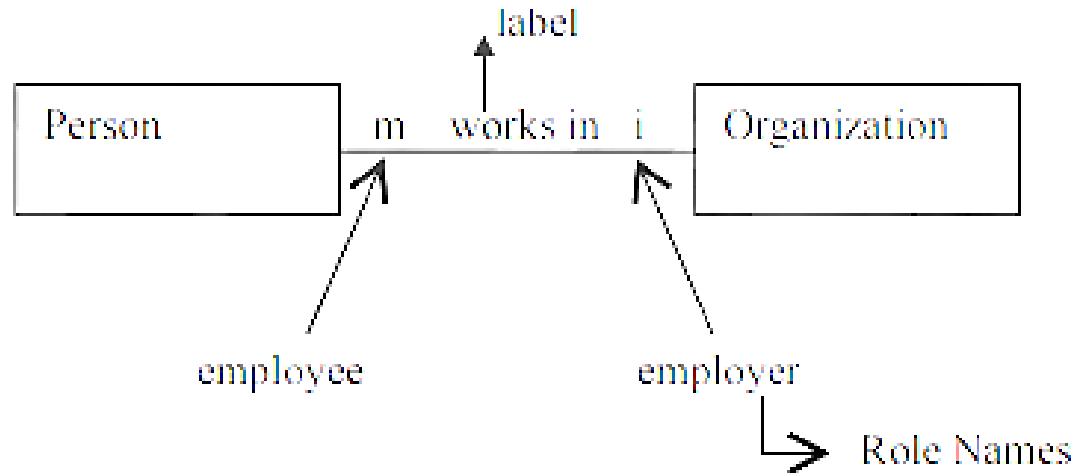


Avoid inheritance

To be used (Recommended)

Association Design

- Association is defined as the relationship that exists among objects of different classes. A link is used to represent this relationship among objects.



Association Design

- Aggregation represents the special kind of association between the whole and its parts.
- Composite Association: In case there is a strong dependency between class 1 and class 2, as a result, if class 1 is deleted, class 2 is deleted automatically

Association Design

Difference between Association, Aggregation and Composition

Association	Aggregation	Composition
Denoted by arrow	Denoted by empty Arrowhead.	Denoted by filled Arrowhead.
Can be classified on 1:1, 1:M, M:N	Weak Association.	A strong type of Association.
Association can be in between two or more objects.	Linked Objects can remain in the scope of a system without each other.	Objects cannot remain in the scope without each other.
Objects are Linked.	Objects are not dependent on other objects.	Objects are strongly dependent on other objects.
Example: Relationship between Student and Teacher Object.	Example: Vehicle requires wheel but not every time of the same type.	Example: relationship between file and folder, if folder deleted all files in the folder also deleted automatically.

Association Design

- Unidirectional Associations :
 - For implementing unidirectional associations, care should be taken so that uni-directionality is maintained.
 - Optional Associations: In this type of association, the class may or may not be linked to each other. It may be possible that customer have a current account or may not be. Consider the case of Customer association with its Current Account.



Association Design

- One-to-one Associations: In 1:1 Association, one class instance is connected to just only one instance of the other class. Consider a situation where Manager is related to its Department in 1:1 relationship.



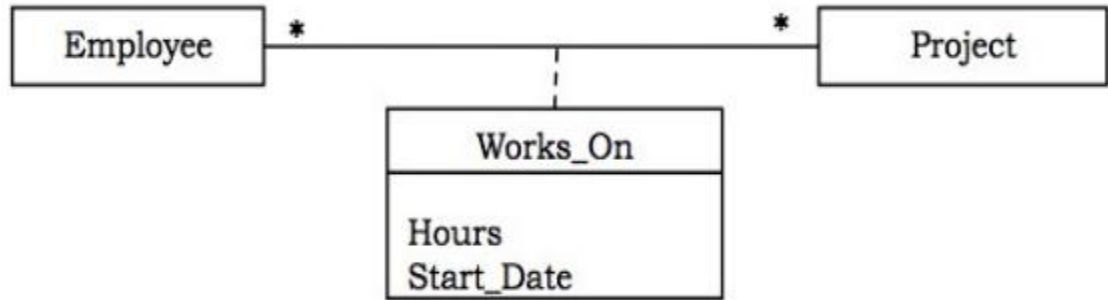
- One-to-many Associations: In this association, single occurrence of a class is linked to many occurrences of other class. In the example below, one department has many employees associated with it.



Association Design

- Bidirectional Association
 - It is used to relate associations that communicate in both directions regularly on constantly, or it is also defined as an association that relates two objects in both direction.
 - A bidirectional relationship provides navigational access in both directions.

Bidirectional
Association
Between two
Classes :-

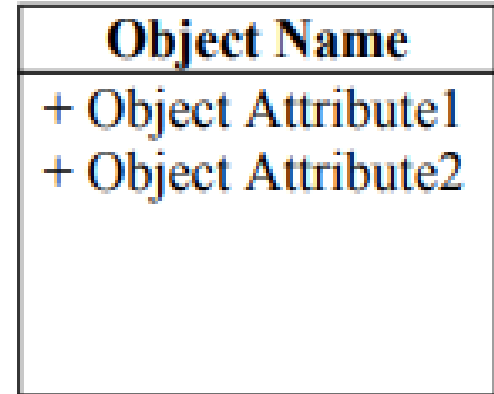


Object Representation

- Representing objects using an object model.
- Objects represent the static view of object-oriented systems, and their behaviour is defined only in class.
- Their behaviour is defined only in class.

Object Representation

- Rules for representing objects by using the diagram -
 - Analyze the system and identify all the objects.
 - Important instances should be identified.
 - It should have some heading.
 - Shows some important associations between objects.
 - Add comments or notes wherever necessary.



Design Optimization

- It considers various design models constructed and trade-offs between time, space, cost and other metrics but does not leave behind other criteria viz. ease of maintaining, implementation, extension, ease of understanding, reusability and many more.
- Some important points related to design optimization are:
 - Often a small part of the code is responsible for most of the time or space costs.
 - Optimizing critical areas is the best approach to work on all areas evenly.
 - Optimized system is more ambiguous, and there is a little possibility of its reusability.
 - It is good to maintain the correct balance between efficiency and transparency.

Design Documentation

- Design documentation is the information that completely describes the design process.
- Documentation should be precise in explaining the capabilities of already constructed components. It may be divided into various types
 - User Documentation
 - System Documentation
 - Project Documentation

Important Topics

- Inheritance Adjustment
- Association: Design
- Object Representation
- Design Optimization
- Design Documentation

Summary

- The purpose of OOA is to identify all objects of problem domain, their relationship and behaviour and during design their attributes, functions and how they all collaborate or interact.
- All design decisions related with object design should be documented.
- Various techniques are used in OOD to make the best model for implementation.
- Explained designing controls, association and design optimization.

*Thank
You*



Object Oriented Analysis and Design

Block-4 Unit-12

Implementation Strategies-1

Topics to be Covered

- Introduction
- Mapping Design to Code
- Creating Class Definition from Class Diagram
- Implementing Associations
- Unidirectional Implementations
- Bi-directional Implementations
- Important Topics
- Summary

Introduction

- After the designing stage, the implementation process gets start.
- The task of converting an object design into source code is a straightforward process.

Mapping Design to Code

- Mapping Designs to Code is a transformation process from UML class diagrams to class and interface definitions as well as method definitions.
- Many UML tools have the facilities to generate code for the programming language.
- They can generate basic codes but cannot generate your ideas and interpretation of the diagram.

Creating Class Definition from Class Diagram

- Class diagram gives you an overview of your application software by depicting classes and their attribute, operations (methods) and relationships among objects.
- If you draw your class diagram with the help of UML software tool, it generates the basic class definition from the diagrams.

Creating Class Definition from Class Diagram



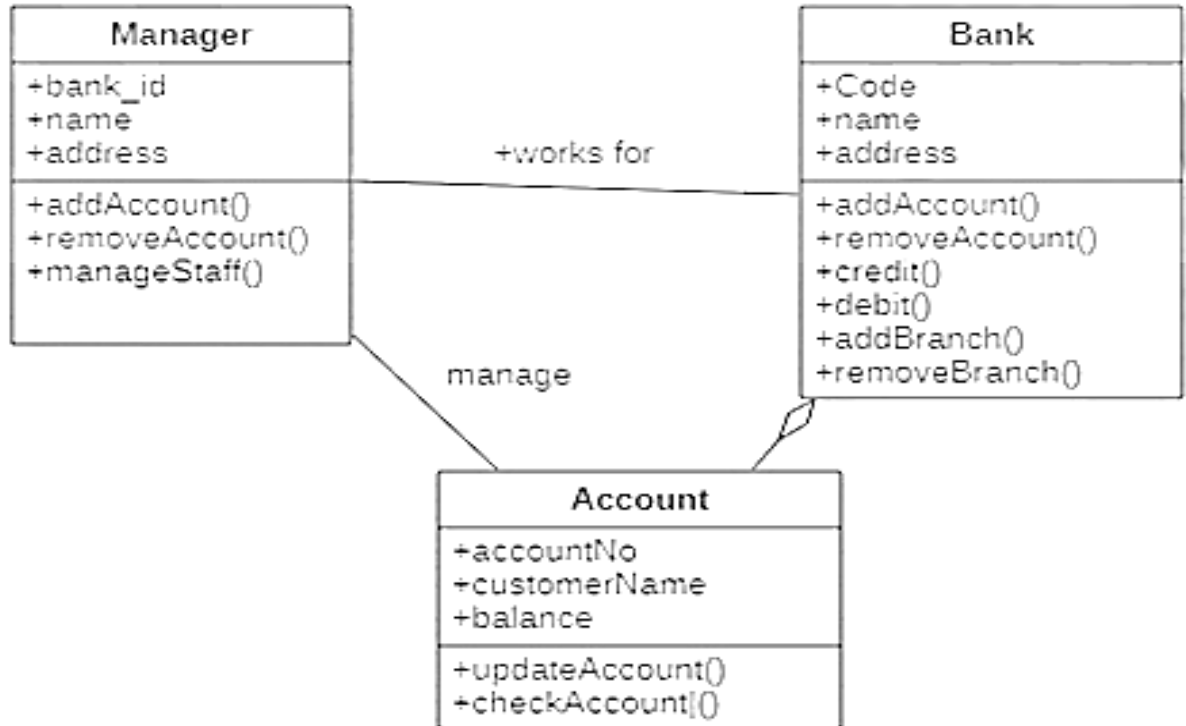
Class diagram for Teacher and Student relationship

Implementing Associations

- Associations tells structural relationships between classes that become links between the instances of these classes. These links shows inter-object navigation that is to say the fact that one instance can retrieve another instance through the navigable link.
- The implementation of association is applied either in unidirectional or bi-directional. As the name implies that unidirectional is a one-way association, and bi-directional is a two-way association.
- Each association may be either optional, one-to-one, one-to-many association.

Implementing Associations

Relationship between
Manager and Bank :

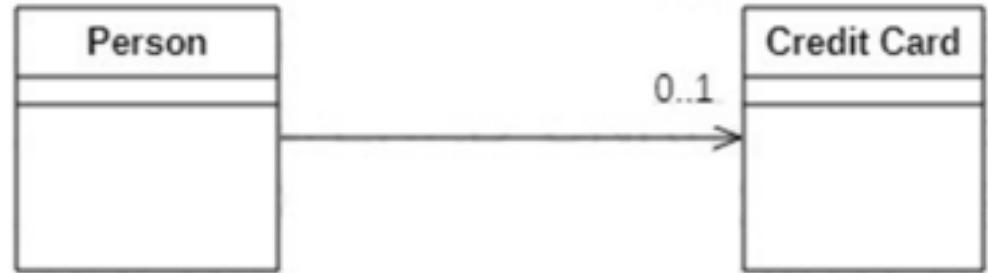


Unidirectional Implementation

- This type of association will be supported in one direction only. It means that one class object is linked to another object of associated class in one direction.
- The direction of traversal of an object can be represented in a class diagram by putting an arrow on the association. It is of three types –
 - Optional
 - One-to-one
 - Association with Multiplicity

Unidirectional Implementation

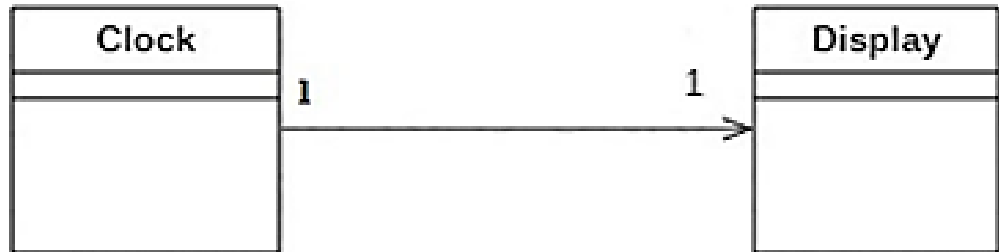
- Optional Association : In optional association, a link between the participating objects may or may not exist. This type of association is implemented in one direction only. Example - Every person may or may not have a credit card.



Example of Optional Association

Unidirectional Implementation

- One-to-One Association : The one-to-one association is a simplest form of association to implement between two classes. In one-to-one association, one occurrence of a class is related to exactly one occurrence of the associated class. Ex – As we know that each person has only one saving account in a bank. So, this is a One-to-one Association. For example, one-to-one association exists in two classes: Clock and Display



Unidirectional Implementation

- Association with Multiplicity ‘Many’ : This type of association exists in those cases where one instance of a class is related to many instances of associated class. Example - In a school, each teacher is responsible for teaching a number of students. A teacher object could be associated with many student objects in this association.

An association with multiplicity ‘many’ :



Bi-directional Implementations

- Bi-directional relationship provides navigational access in both directions, so that we can access the entity of the other side without explicit requests.
- Each entity has a relationship property or field that refers to the other entity.
- The following subsections define bi-directional implementation of association for different multiplicity is as follows:
 - One-to-one and Optional Associations ,
 - One-to-many ,
 - Immutable Association

Bi-directional Implementations

- One-to-One and Optional Associations : In One-to-one bi-directional association, you can traverse from an occurrence of one class to an occurrence of the associated class and vice versa. Example, we have to create two classes: Person and Account. Both the classes contain a link through which it is associated. Each class needs an attribute that contains a reference to an object of the associated class.
- Example of one-to-one bi-directional association -



Bi-directional Implementations

- One-to-Many Association : In One-To-Many associations, one source is connected with multiple targets. For example, many students can register for one program. Consider the following example; here we have models two classes: Person and Account. There is a One-to-Many relationship between them, which means that one Person has many Accounts.



Bi-directional Implementations

- Immutable Association : In object-oriented programming, an immutable object is unchangeable object, and its state cannot be modified after it is created, whereas mutable objects are changeable objects which can be modified after their creation. Example - An association between account and debit card are traversed in both directions. It is assumed that there is a restriction for the debit card; it can only be issued to person per account.



Important Topics

- Implementing Associations
- Unidirectional Implementations
- Optional Associations
- One-to-One Associations
- Associations with Multiplicity 'Many'
- One-to-One and Optional Associations
- One-to-Many Associations
- Immutable Association

Summary

- In this session, you have learned mainly about implementation strategy, which transforms design into code in a programming language.
- Association creates a relationship between two separate classes through their objects. This relationship can be unidirectional or bi-directional
- Unidirectional and Bi-directional association may be either one-to-one, one-to-many or many-to-many relationships.

*Thank
You*



Object Oriented Analysis and Design

Block-4 Unit-13

Implementation Strategies - 2

Topics to be Covered

- Introduction
- Creating Methods from Collaboration Diagram
- Implementing Constraints
- Implementing State Charts
- Persistency
- Important Topics
- Summary

Introduction

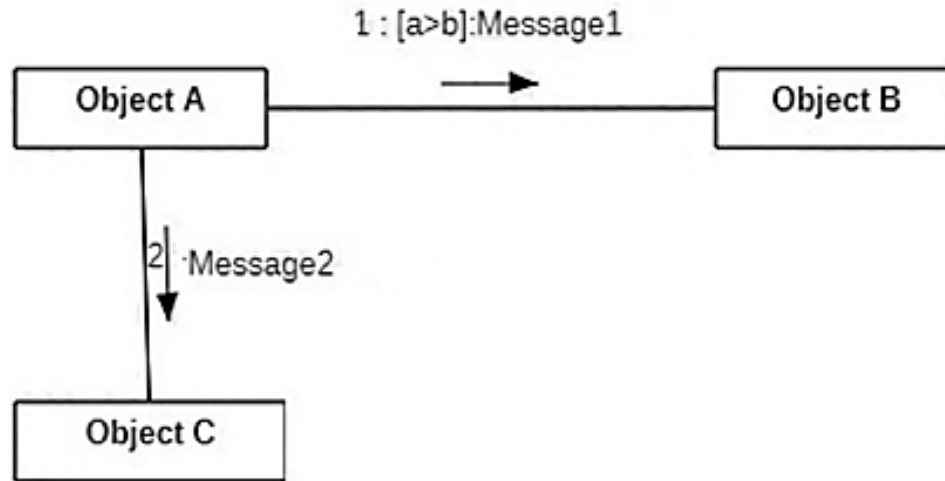
- This session will teach us implementation strategy for the dynamic behavior of object-oriented system.
- We will know about the persistency concept.

Creating Methods from Collaboration Diagram

- The key purpose of creating this diagram is to illustrate the relationship between objects and works together to perform a particular task.
- In an application system, there are one or more collaboration diagrams to find all the eventuality of a complex behaviour.
- While designing the collaboration diagram, it contains the elements such as actors, objects and their communication links, and messages.

Creating Methods from Collaboration Diagram

- Collaboration Diagram showing If-else condition



Creating Methods from Collaboration Diagram

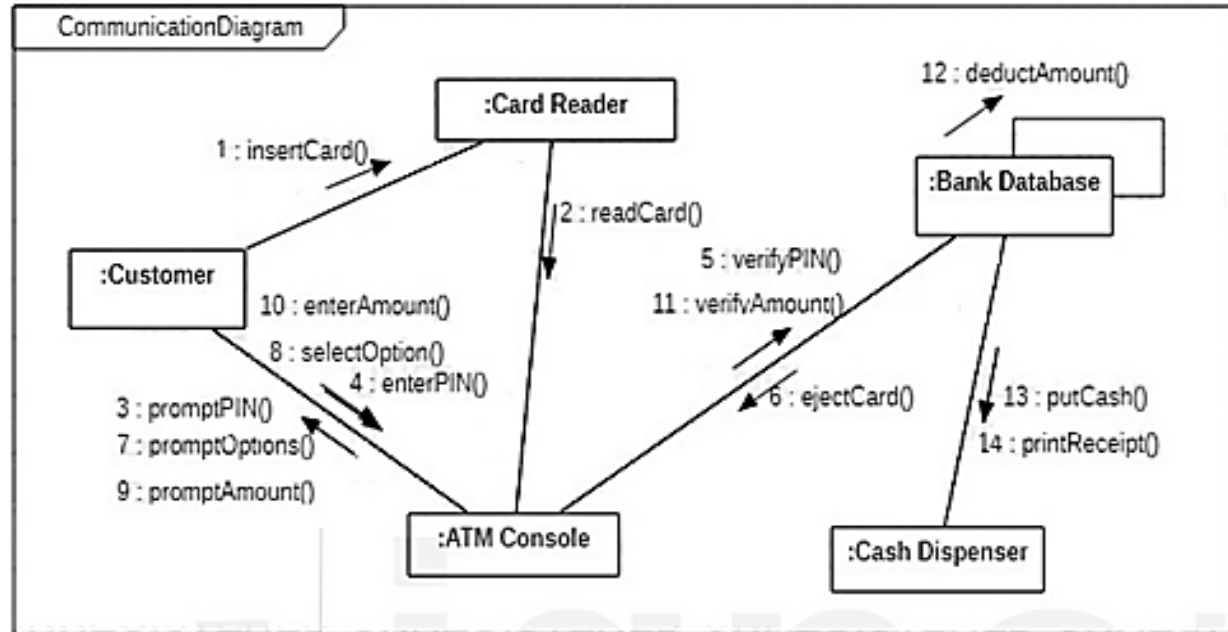
- Let us consider a collaboration diagram for ATM transactions. For this example, first, we will create a Use Case diagram and then draw a collaboration diagram. Use Case Diagram –
 - Customer can insert ATM card into the card reader of the ATM console.
 - The card reader reads and verifies the card.
 - ATM console displays a message and prompts the customer to enter PIN.
 - Customer enters PIN.
 - ATM console verifies PIN with bank database and confirms the PIN is valid.
 - If PIN is invalid, ATM console notifies the customer with a message and ejects the card.

Creating Methods from Collaboration Diagram

- If PIN is valid, ATM console represents a prompt with options such as deposit, withdraw and transfer money.
- Customer selects withdraw option.
- ATM console presents prompt to the customer for an accurate amount to be withdrawn.
- The customer enters the amount.
- ATM console verifies with the bank database whether the customer account has sufficient funds. Otherwise, it shows an insufficient funds message.
- ATM console deducts money from customer's account, puts the requested amount in cash dispenser, and prints a receipt.
- ATM console rejects the card and becomes ready for another transaction

Creating Methods from Collaboration Diagram

- Using the above flow of messages, we can construct the collaboration diagram-



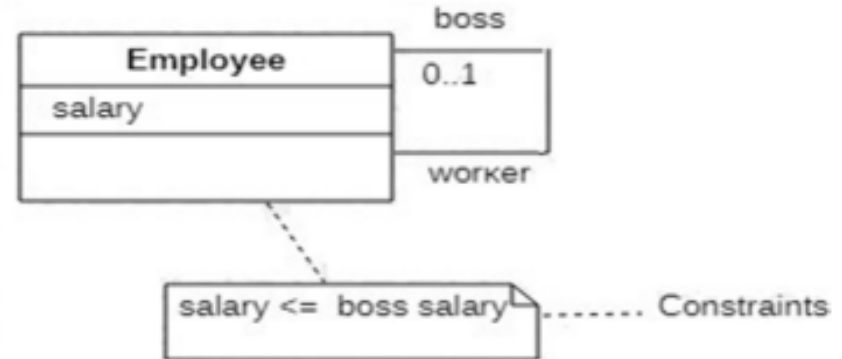
Implementing Constraints

- A constraint is a condition that restrains the semantics of an element and it must Implementation always be true.
- For example, a person can only get a loan from a bank if the person has an account with the bank.
- Class diagram demonstrates an example of applying constraints on the attributes.
- The constraints are used to represent the semantics of object-oriented model elements such as objects, links and attributes.

Implementing Constraints

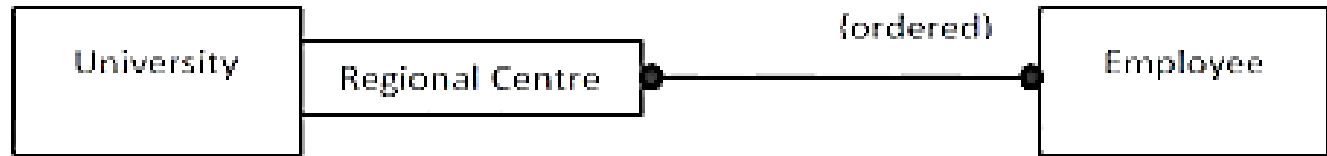
- Constraints on Objects

- The following class diagram demonstrates an example of applying constraints on the attributes. Employee could be as a boss or the worker. Worker's salary cannot exceed the salary of boss.



Implementing Constraints

- Constraints on Links
 - As we know, the multiplicity specifies how many objects of a class can be coupled with an object of the other association class. The multiplicity constraints restrict the number of objects associated with the given object. Object modeling notations define the syntax for displaying multiplicity values like [0, 1] and exactly 1

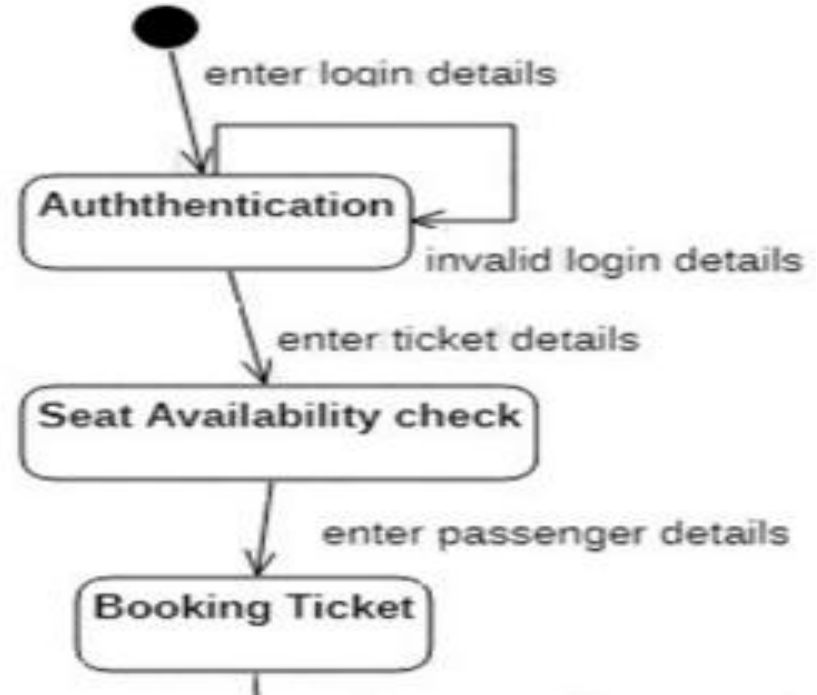


Implementing State Charts

- State diagram is used to represent the condition of the system or part of the system at a specific period of time.
- We use it to state the events responsible for change in state.
- We use it to model the dynamic behavior of the system .
- We can say that, a state diagram is used to model the dynamic behavior of a class in response to time and changing external stimuli.

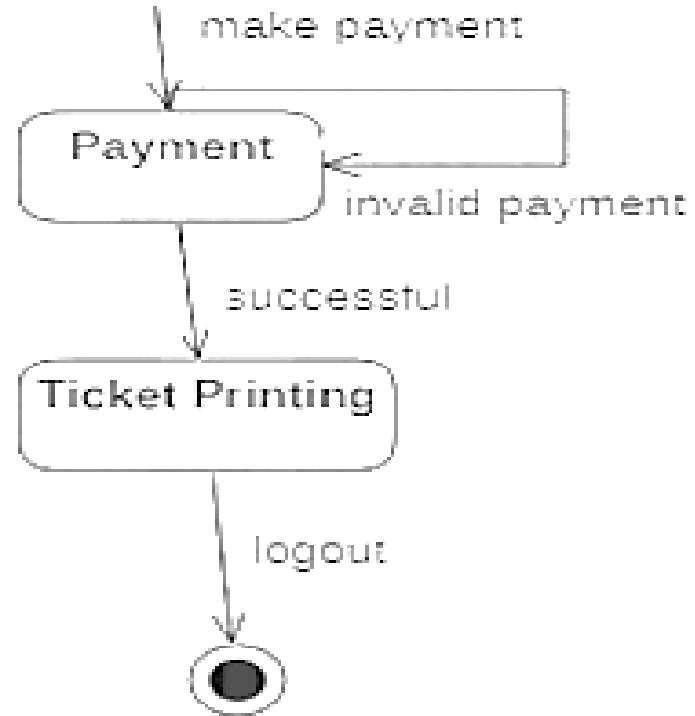
Implementing State Charts

State diagram for Booking Tickets :



Implementing State Charts

State diagram for Booking Tickets :
(Continued)



Implementing State Charts

- Implementing State chart using Helper Object –
 - The helper object denotes the current state of the domain object and specifically implements the behavior of the current state. The domain object transfers all external messages to its helper object, which reacts to the messages on account of the domain object.



Persistency

- In general meaning, persistent means preservation. It occurs over a prolonged period.
- This is the property by which an object stays even after its initiator concludes (close the program) is known as persistence.
- Persistence is important for enterprise applications because it gives required access to relational databases.

Persistence

- **Data Persistence** : The most important tasks the application performs are to save and restore data. Persistent data may give useful mechanism for passing data between programs and permits the program to resume processing after a while
- **Process Persistence** : Persistence represents the lifetime of a process or an object. Referring to the computer threads and processes, a persistent process is a process which cannot be killed or shut down by other processes until the user kills them.

Persistence

- **Object Persistence** : In connection with data, persistence means an object should not be removed except it is actually erased. This requires appropriate storage and certain procedures which permit the data to persist.
- **Identifying Persistent Data** : While designing models, models may not always provide a clear picture of the data, and this data should be persistent or transient. There are no limitations to defining permanent data in models, so a single model can bring together persistent and transient data.

Persistence

- Persistent data is steady and restorable and is accessible after closing the application. This type of data must be saved into a database or internal or external memory, whereas Non- persistence data is not accessible after closing the application
- **Serialization :** Serialization is the process of transforming an object into byte streams to store the object to memory or a database.
- The serialization's main objective is to save an object's state so that it recreates when needed.

Important Topics

- Creating Methods from Collaboration Diagram
- Implementing Constraints
- Implementing State Charts
- Persistency

Summary

- The key purpose of creating collaboration diagram is to illustrating the relationship between objects and to explain how objects work together to perform a particular task.
- Implementation of behavioral features of model through the state chart.
- Finally, persistency concepts have been discussed concerning object oriented databases and relational databases.

*Thank
You*



Object Oriented Analysis and Design

Block-4 Unit-14

Objects Mapping with Databases

Topics to be Covered

- Introduction
- Relational Database Schema for Object Models
- Object Classes to Database Tables
- Extended Three Schema Architecture for Object Models
- Mapping Associations to Tables
- Mapping Generalizations to Tables
- Interfacing to Databases
- Important Topics
- Summary

Introduction

- The relational databases are widely used database software in organizations.
- This session introduces you to how to implement the object model in the relational model.
- This session starts with some introductory concepts on general database as well as relational DBMS concepts.

Relational Database Schema for Object Models

- A database is a group of interconnected data files with structures. It is designed to come across the various information necessities of the organization.
- General DBMS Concepts:
 - A DBMS is a software program which is used for creating, retrieving and controlling access to permanent data. There are various motives for using DBMS, such as Integrity, Security, Crash Recovery, Data Distribution and Sharing between users as well as applications

Relational Database Schema for Object Models

- Database Schema :
 - Database Schema is the design of a database. It is the frame of the database that signifies the structure (table names and their fields/columns name).
- Data Constraints
 - Sometimes you want to place certain restrictions on the type of data that can be inserted in column(s) of a table. This is done by defining one or more constraints on that column(s) while creating the tables.

Relational Database Schema for Object Models

- Data Dictionary:
 - The database schema along with numerous constraints on the data is stored in a database catalog or dictionary. It is called meta-data. A meta-data is data about the data.
- Database Design:
 - The database design is an important and tough task for any business database application. Database design describes the database structure used for creating, storing and maintaining information.

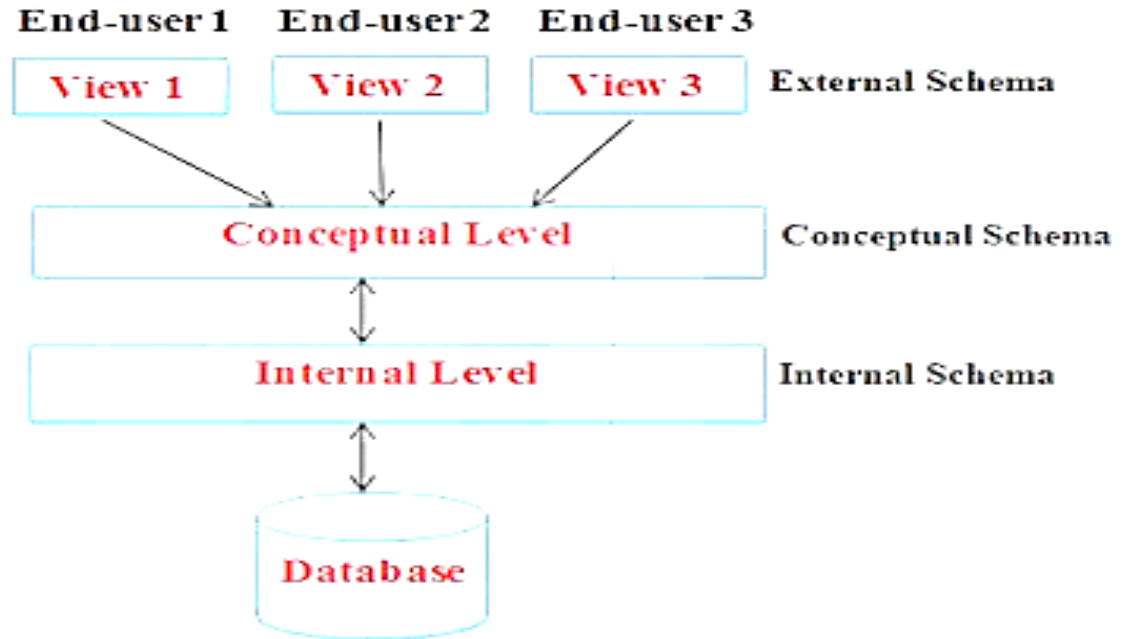
Relational Database Schema for Object Models

- **Three schema architecture**

- The database design contains three layers such as external, conceptual and internal schemas.
 - The external schema is external view which represents some portion of the entire database. It is a higher order abstraction which is seen by the user.
 - The next level is conceptual view which represents data logically. It defines what data is stored really in the database.
 - The third level of this architecture is an internal schema which shows the lowest level of abstraction and closest to physical storage.

Relational Database Schema for Object Models

Diagram of three
schema
architecture :



Relational Database Schema for Object Models

- **RDBMS Integrity** For maintaining the quality of the information in the database, relational DBMS provides integrity rules. Data integrity means that the data is accurate and consistent in the database. There are two rules of integrity :
 - Integrity Rule 1(Entity integrity): It defines that each table have exactly one primary key. A primary key is a set of one or more attributes that can uniquely identify each row in a relation. Integrity
 - Rule 2 (Referential integrity): It needs that a foreign key must have a matching primary key. Referential integrity constraint ensures that relationships between records in related tables are valid.

Relational Database Schema for Object Models

- A foreign key is a primary key of one table that is embedded in other table. The primary, as well as foreign keys, are shown in the following tables.

CourseCode	CourseName
MCS-218	Data Communication and Computer Networks
MCS-219	Object Oriented Analysis and Design
MCS-220	Web Technologies

RollNo	Name	Age	CourseCode
1	Ram	20	MCS-218
2	Mohan	25	MCS-219
3	Govind	30	MCS-220
4	Narayan	28	MCS-218

CourseCode is the primary key of the Course table, and RollNo is the primary key of the Student table. CourseCode is a foreign key in the Student table.

Relational Database Schema for Object Models

- Normalization : The normalization is used to reduce the repetition of data.
- A table is in first normal form (1NF) if and only if all underlying domains of the table contain atomic values.
- A table is in second normal form (2NF) when it satisfies the first normal form and every non-key attribute fully depends on the primary key.
- A table is in the third normal form (3NF) when it satisfies the second normal form, and every non-key attribute is non-transitively dependent on the primary key

Object Classes to Database Tables

- In the previous section, we have gone through the general DBMS and relational DBMS concepts that are most useful during the database design process. This section provides you with how to transform an object model into a relational database. This section focuses on extended three schema architecture for the object models, use of object IDs, and mapping object classes to tables.

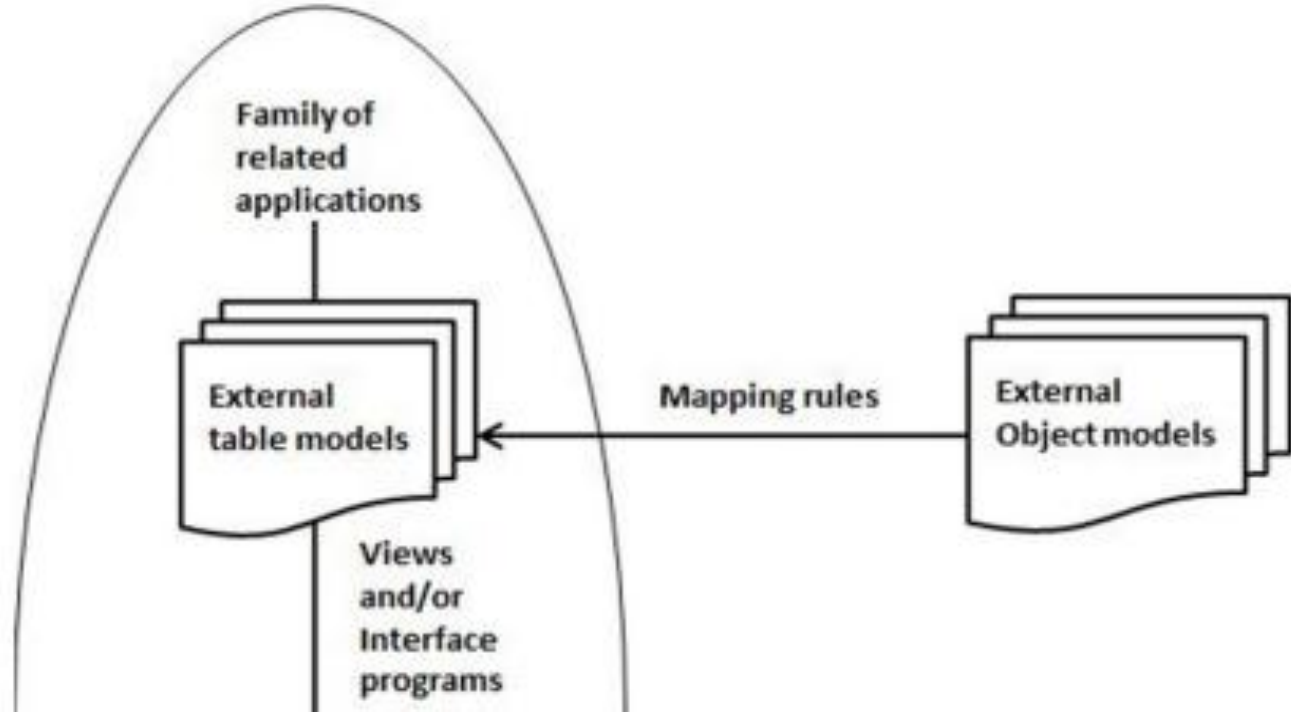
Extended Three Schema Architecture for Object Models

- The three schema architecture is used for designing the structure of database systems. This architecture is used to separate the user applications and physical databases. The three schema architecture covers three levels :
 - External Schema: It designates high-level data model and closest to the user.
 - Conceptual Schema: It defines how the database looks in front of users.
 - Internal Schema: It defines the physical storage structure of the database.

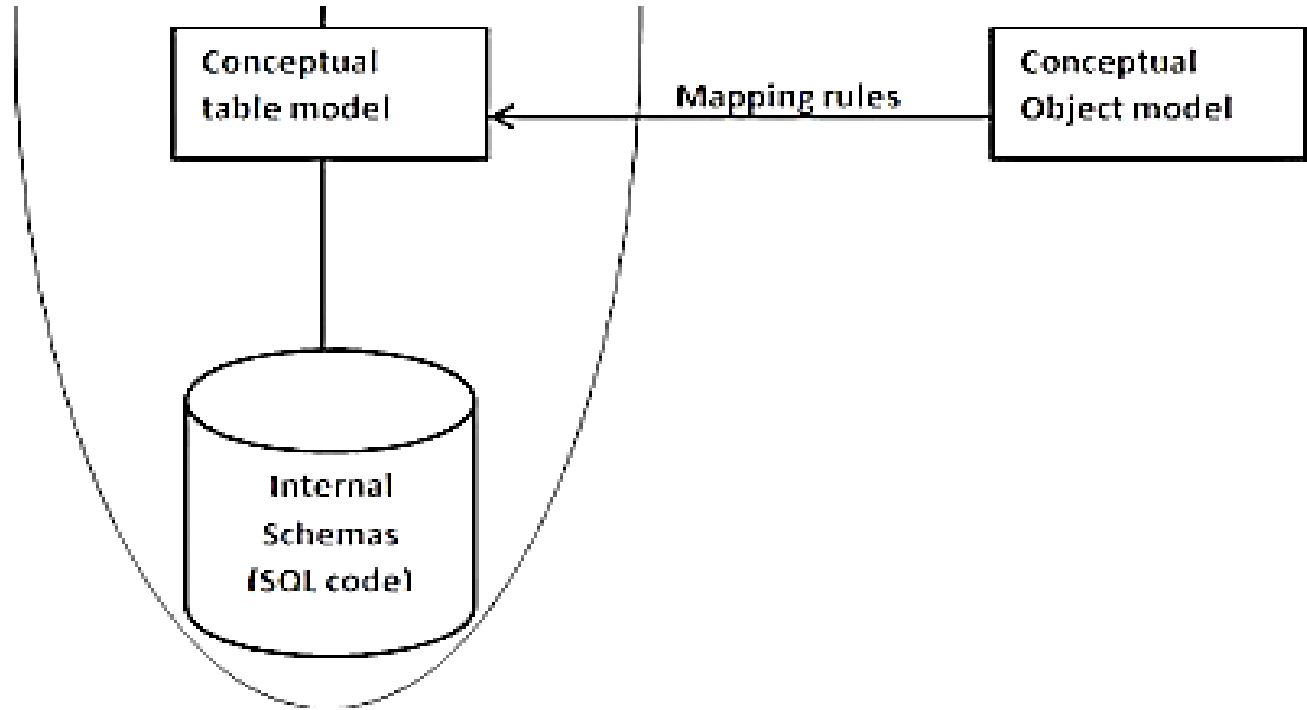
Extended Three Schema Architecture for Object Models

- The **object model** defines the static structure of the objects in a system and their relationships. It emphasizes on logical data structures.
- The **internal schema** of the three schema architecture contains SQL commands which are used to create the tables, attributes, assign constraints, and keys.

Extended Three Schema Architecture for Object Models



Extended Three Schema Architecture for Object Models



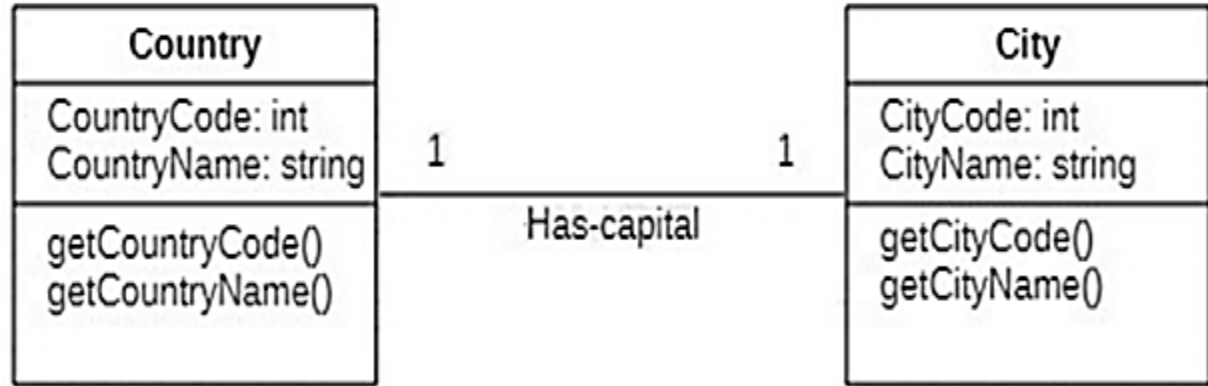
Mapping Associations to Tables

- Association is a relationship between two separate classes through their objects. Associations are generally denoted as continuous lines between the participants' classes in the relationship.
- An association may be
 - one-to-one, one-to many,
 - many-to-many or
 - a ternary association.

Mapping Associations to Tables

- In one-to-one Association, one occurrence of a class is related to exactly one occurrence of the associated class.

The following class diagram shows one-to-one association between Country and City :



Mapping Associations to Tables

University Table Model

Attribute Name	Null	Domain
UniID	N	ID
UniName	N	Name
UniAddress	Y	address

Employee Table Model

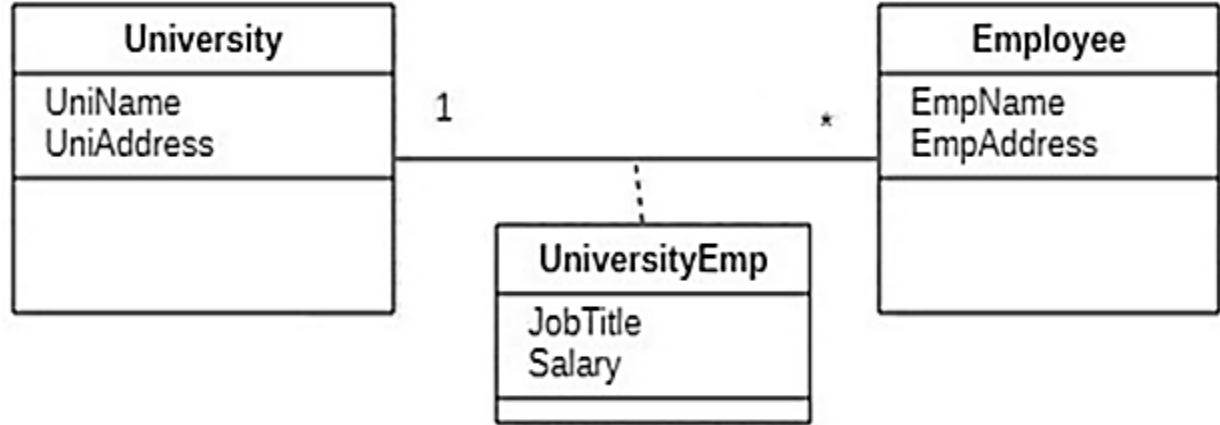
Attribute Name	Null	Domain
EmpID	N	ID
EmpName	N	Name
EmpAddress	Y	Address
UniID	N	ID
Job title	Y	Title
Salary	Y	Salary

Table model for one-to-many Association(using foreign key)

Mapping Associations to Tables

- In one-to-many association, one occurrence of a class is related to many occurrence of the associated class.

The class diagram shows one-to-many association between University and Employee classes



Mapping Associations to Tables

UniversityEmp

Attribute Name	Null	Domain
UniID	N	ID
EmpId	N	ID
Job title	Y	Title
Salary	Y	Salary

University Table Model

Attribute Name	Null	Domain
UniID	N	ID
UniName	N	Name
UniAddress	Y	address

Employee Table Model

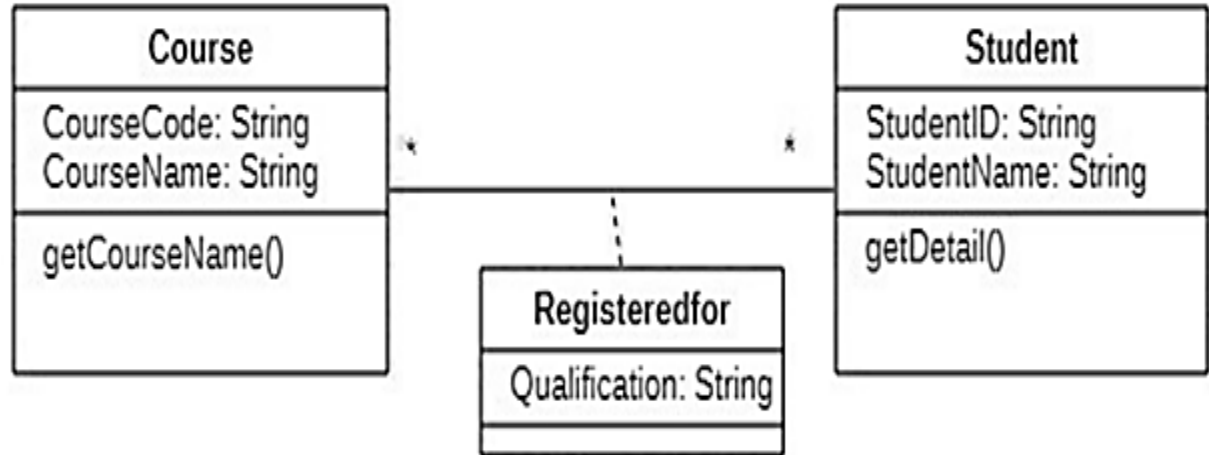
Attribute Name	Null	Domain
EmpID	N	ID
EmpName	N	Name
EmpAddress	Y	Address

Table model for one-to-many association – using distinct table

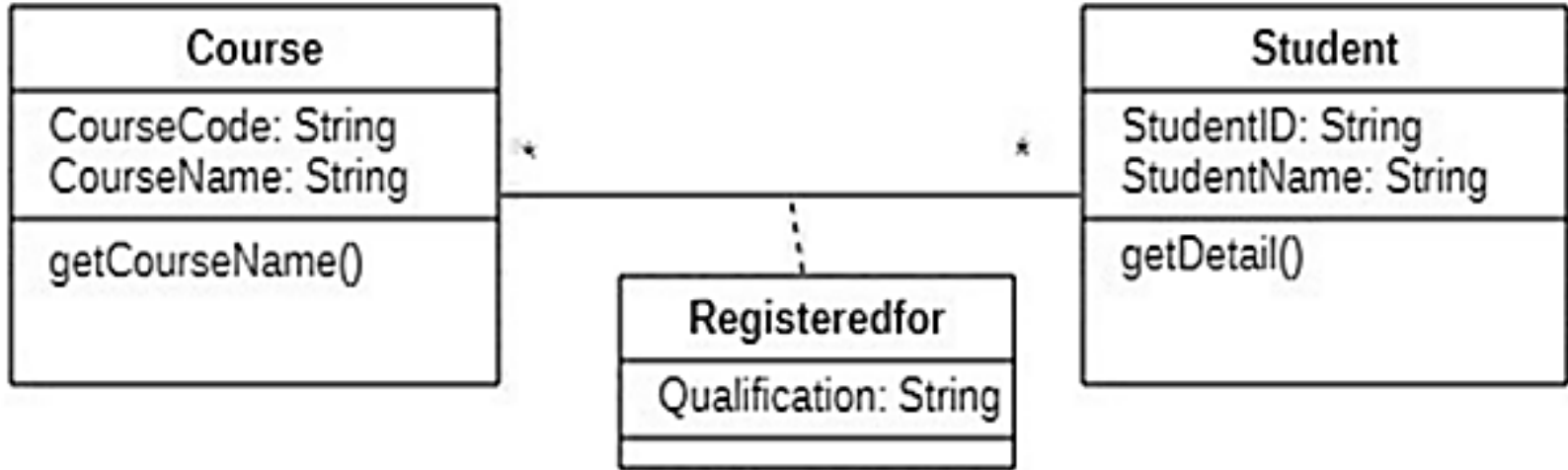
Mapping Associations to Tables

- In many-to-many association, M occurrence of a class is in a relationship with N occurrence of the associated class.

Object model
for many-to-
many
Association :



Mapping Associations to Tables



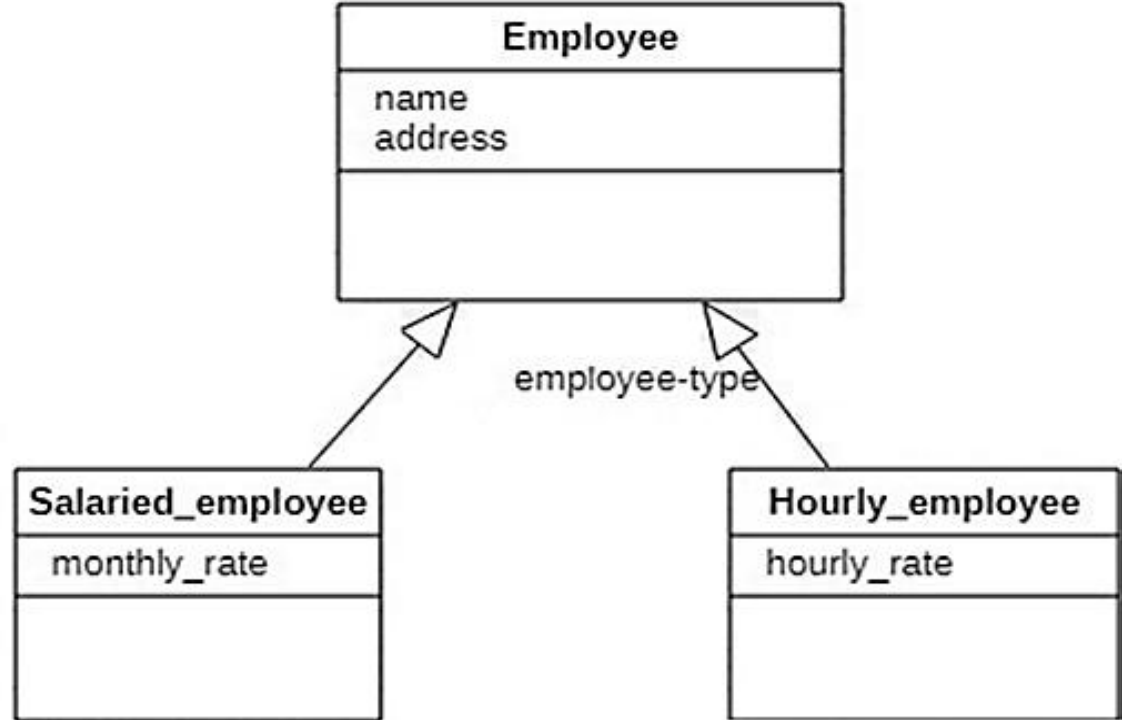
Object model for many-to-many Association

Mapping Generalizations to Tables

- Generalization is a superclass - subclass type of relationship.
- The Employee superclass is a generalization of the entity subsets such as salaried or hourly employees.

Mapping Generalizations to Tables

Object Model for
Generalization :



Interfacing to Databases

- A DBMS is a software program which is used for creating, retrieving and controlling access to permanent data.
- Once we have transformed object oriented data model to relational database, we shall write the interfacing code responsible for the application's functionality.
- This code must contain commands capable of reading and writing data from the database and interpreting it wherever requisite in terms of the model used by the program.

Important Topics

- General DBMS Concepts
- Relational DBMS Concepts
- RDBMS Logical Data Structure
- Extended Three Schema Architecture for Object Models
- Mapping Associations to Tables
- Mapping Generalizations to Tables

Summary

- Learned about the basic and relational concepts of RDBMS as well as extended 'three schema architecture' for object model, which is the base of database design. The three schema architecture provides an abstract view of data to users.
- This Unit has typically about the object mapping to database. You have learnt about mapping the object classes to tables.
- We have also studied the mapping generalizations to tables. In this type of mapping, we have learnt different tactics or strategies. Towards this unit's end, the database interfacing has been described.

*Thank
You*